

Intelligent Security Compliance Management System (ISCMS)



Project Id: IT-F25-26

MUHAMMAD ALI BUKHARI
01-135222-096

MUHAMMAD HAMMAD CHAGHTAI
01-135222-014

Supervisor: Dr Faisal Bashir

Bachelor of Science in Information Technology

Department of Computer Science
Bahria University, Islamabad

May 2026

Certificate

We acknowledge the work done in the report titled “Intelligent Security Compliance Management System (ISCMS)” by authors Mr. Muhammad Ali Bukhari and Mr. Muhammad Hammad Chaghtai as an affirmation of the standard of completion for the partial fulfillment of the degree of Bachelor of Science in Information Technology.

Approved by :

Supervisor: Dr. Faisal Bashir (Dean Engineering Sciences)

Internal Examiner: Name of the Internal Examiner (Title)

External Examiner: Name of the External Examiner (Title)

Project Coordinator: Dr. Kabeer Ahmed Bhatti (FYP Coordinator, IT Department)

Head of the Department: Dr. Muhammad Khurram Ehsan (Head of Department,
Computer Science)

May, 2026

Abstract

Manual security auditing and static compliance mapping leave gaps that expose enterprise infrastructure to configuration drift. Existing Governance, Risk and Compliance (GRC) tools ignore endpoint telemetry, which raises administrative costs and produces unreliable security assessments. This project presents the Intelligent Security Compliance Management System (ISCMS), a platform that automates continuous controls monitoring. The system integrates with the Wazuh Security Information and Event Management (SIEM) engine to ingest Security Configuration Assessment (SCA) logs. A deterministic mapping algorithm translates raw technical events into verifiable regulatory controls, enabling continuous compliance tracking against standards such as ISO/IEC 27001. The system follows an N-tier architecture. A React Single Page Application (SPA) serves the presentation layer and communicates with a Django REST Framework API. To avoid blocking the main thread during SIEM polling and PDF report generation, the system offloads I/O operations to a Celery and Redis worker tier. PostgreSQL stores all data using a hybrid schema that pairs relational constraints for Role-Based Access Control (RBAC) with JSON fields for raw telemetry. Authentication combines stateless JSON Web Tokens (JWT) with Time-based One-Time Password (TOTP) two-factor verification. Testing confirmed the platform operates correctly. The asynchronous tier processed concurrent endpoint synchronizations with an average API latency under 150ms, and the mapping engine preserved data integrity during collision attempts. These results show that replacing periodic manual audits with continuous SIEM integration produces a verifiable compliance posture and cuts the overhead of organizational audits.

Acknowledgments

In the name of Allah, the Most Gracious, the Most Merciful. All praise is due to Allah for the strength and guidance to complete this work.

We thank our supervisor, Dr. Faisal Bashir, for his mentorship and technical direction throughout the project. We also appreciate the lab staff who assisted us and the faculty members who taught us over the last four years. We are thankful to the Department of Computer Science and Bahria University for providing the resources needed to complete our degree.

We are grateful to our parents, families, and friends for their support and patience. We also thank each other for the collaboration that carried this project through to completion.

Finally, on a personal note, I (Muhammad Ali Bukhari) dedicate this to my pet bird, Kiroo. Though he is no longer with me, his memory brought me comfort and joy throughout this journey.

Muhammad Ali Bukhari

Muhammad Hammad Chaghtai

Contents

Certificate	i
Abstract	ii
Acknowledgments	iii
List of Figures	x
List of Tables	xiii
List of Abbreviations	xv
1 Introduction	1
1.1 Project Background	1
1.1.1 The Temporal Blind Spot of Audits	1
1.1.2 The Shift to Continuous Monitoring	1
1.2 Problem Description	2
1.2.1 Telemetry Correlation Delays	2
1.2.2 Algorithmic Failures in Legacy Systems	2
1.3 Project Objectives	2
1.4 Methodology	3
1.4.1 Agile Development Lifecycle	3
1.4.2 Sandbox Testing and Evaluation	3
1.5 Project Scope	4
1.5.1 Compliance Framework Boundaries	4

1.5.2	Technical Implementation Boundaries	4
1.6	Feasibility Study	4
1.6.1	Technical and Economic Feasibility	4
1.6.2	Operational Feasibility	5
1.7	Areas of Application	5
1.7.1	Financial Technology (FinTech)	5
1.7.2	Healthcare Sector	5
1.8	Report Organization	5
2	Literature Review	7
2.1	Literature Review	7
2.1.1	Evolution of Governance, Risk, and Compliance (GRC) Platforms . . .	7
2.1.2	Security Information and Event Management (SIEM) Integrations . . .	7
2.1.3	Performance Bottlenecks in Telemetry Aggregation	8
2.1.4	Modern Access Control Paradigms	9
2.1.5	Containerization and Defense-in-Depth Topology	9
2.1.6	Cognitive Load and HCI in Security Dashboards	10
2.2	Analysis of Existing Systems	10
2.2.1	Manual Spreadsheet Auditing	10
2.2.2	Vanta (Commercial SaaS)	10
2.2.3	Drata (Commercial SaaS)	11
2.2.4	AuditBoard (Enterprise GRC)	11
2.3	Comparative Analysis	11
2.4	Summary of Advantages	12
3	Requirement Specification	13
3.1	Existing System	13

3.2	Proposed System	13
3.2.1	Technology Stack Selection	13
3.2.2	High-Level Architecture	14
3.2.3	Dockerized Infrastructure and Network Isolation	15
3.3	Requirements Specification	16
3.4	Functional Requirements	16
3.5	Non-Functional Requirements	17
3.6	User Requirements	17
3.7	System Models and Components	18
3.7.1	Context Diagram	18
3.7.2	Data Flow Diagram	18
3.8	Use Cases	19
3.8.1	Use Case Diagram	19
3.8.2	UC-01: User Authentication	20
3.8.3	UC-02: Manage Wazuh Telemetry Synchronization	21
3.8.4	UC-03: Map Framework Controls to Wazuh Rules	22
3.8.5	UC-04: Monitor Endpoint Compliance Status	23
3.8.6	UC-05: Resolve Compliance Anomalies	24
3.8.7	UC-06: Export Audit Reports	25
4	Design	27
4.1	System Architecture	27
4.1.1	Client and Presentation Tier	27
4.1.2	Application and Middleware Tier	27
4.1.3	Asynchronous Processing Tier	27
4.1.4	Data Persistence Tier	28

4.2	Design Constraints	28
4.2.1	Technical Constraints	28
4.2.2	Infrastructure and Operational Constraints	29
4.3	Design Methodology	29
4.4	High Level Design	30
4.4.1	Component Diagram	30
4.4.2	Deployment Diagram	30
4.4.3	System Activity Diagrams	31
4.4.3.1	Authentication and IAM Routing	31
4.4.3.2	Asynchronous Telemetry Ingestion	32
4.4.3.3	Compliance State Evaluation	33
4.4.3.4	Audit Report Generation	34
4.4.3.5	Framework Rule Mapping and Validation	35
4.4.3.6	Real-Time Alerting and Notification	36
4.4.4	System Sequence Diagrams	37
4.4.4.1	SD-01: User Authentication	37
4.4.4.2	SD-02: Manage Wazuh Telemetry Synchronization	38
4.4.4.3	SD-03: Map Framework Controls to Wazuh Rules	39
4.4.4.4	SD-04: Monitor Endpoint Compliance Status	40
4.4.4.5	SD-05: Resolve Compliance Anomalies	41
4.4.4.6	SD-06: Export Audit Reports	41
4.5	Low Level Design	42
4.5.1	UML Class Diagram	42
4.6	Database Design	43
4.6.1	Physical Data Model (Core ERD)	43
4.6.2	Extended Entity Relationship Diagram (EERD)	44

4.7	Graphical User Interface (GUI) Design	45
4.7.1	Authentication and Identity Management	45
4.7.2	Superadmin Configuration Portal	46
4.7.3	Auditor Workspace (RBAC Enforcement)	48
4.7.4	Automated Audit Artifacts and Notifications	48
4.8	External Interfaces	50
4.8.1	Wazuh SIEM REST API	51
4.8.2	SMTP Notification Gateway	51
4.8.3	Outbound Webhook Integration	51
4.8.4	Interface Specification Matrix	51
5	Implementation	52
5.1	System Architecture	52
5.1.1	Presentation Layer Implementation (Frontend)	54
5.1.2	Middleware and API Gateway (Backend)	54
5.1.3	Asynchronous Processing Engine	55
5.1.4	Data Persistence Layer	55
5.1.5	Architectural Technology Stack	55
5.2	System Internal Components	56
5.2.1	Identity and Access Management (IAM) Module	57
5.2.2	Governance and Compliance Data Engine	57
5.2.3	SIEM Telemetry and Synchronization Module	57
5.2.4	Automated Audit and Reporting Module	58
5.2.5	Frontend State and Visualization Layer	58
5.3	Tools and Technologies	58
6	System Testing and Evaluation	60

6.1	Critical Appraisal and Limitations	60
6.2	Testing Environment and Methodology	60
6.3	Functional System Test Cases	61
6.3.1	TC-01: User Authentication	61
6.3.2	TC-02: Manage Wazuh Telemetry Synchronization	65
6.3.3	TC-03: Map Framework Controls to Wazuh Rules	69
6.3.4	TC-04: Monitor Endpoint Compliance Status	71
6.3.5	TC-05: Resolve Compliance Anomalies	73
6.3.6	TC-06: Export Audit Reports	74
6.4	Non-Functional System Testing	75
6.4.1	Performance and Load Testing	76
6.4.2	Security and Penetration Testing	76
6.4.3	Usability and Accessibility Testing	77
6.4.4	Compatibility and Deployment Testing	77
7	Conclusions	79
A	User Manual	80
A.1	Purpose	80
A.2	System Requirements	80
A.3	System Access	80
A.4	Main User Steps	81
A.5	Logout	88
A.6	Support	88
A.7	Note	89
	References	90

List of Figures

3.1	Refined N-Tier System Architecture and Protocol Flow	15
3.2	Docker Service Mesh: Port Exposure and Health-Check Dependencies	16
3.3	Context Diagram: High-Level System Interactions	18
3.4	Level-1 Data Flow Diagram detailing internal compliance processes.	19
3.5	Advanced UML Use Case Diagram illustrating RBAC inheritance hierarchies, core compliance interactions, system boundaries, and integration with secondary external actors (Authenticator, SIEM, SMTP).	20
4.1	Tier-wise System Architecture of the ISCMS Platform	28
4.2	Conceptual View: UML Component Diagram	30
4.3	Physical View: UML Deployment Diagram	31
4.4	Activity Diagram: Authentication and IAM Routing	32
4.5	Activity Diagram: Parallel Asynchronous Telemetry Ingestion	33
4.6	Activity Diagram: Iterative Compliance Evaluation and Worst-Case Priority . .	34
4.7	Activity Diagram: Automated PDF Audit Report Generation	35
4.8	Activity Diagram: Framework Rule Mapping and API Validation	36
4.9	Activity Diagram: Real-Time Alerting and Notification Fan-out	37
4.10	SD-01-1: Valid Authentication Flow	38
4.11	SD-01-2: Invalid Authentication Flow	38
4.12	SD-02-1: Telemetry Synchronization Success	39
4.13	SD-03-1: Map Controls Success	39
4.14	SD-03-2: Map Collision	40

4.15 SD-04-1: Monitor Endpoint Compliance Status	40
4.16 SD-05-1: Resolve Compliance Anomalies	41
4.17 SD-06-1: Report Generation	42
4.18 UML Class Diagram: Core Object-Relational Models with Department Governance	43
4.19 Physical Database Schema: Department-Governed Relational Data Model . . .	44
4.20 Extended Entity Relationship Diagram (EERD) with Constraints and Attributes	45
4.21 GUI Design: Secure User Authentication Portal.	46
4.22 GUI Design: Time-based One-Time Password (TOTP) Verification.	46
4.23 GUI Design: Superadmin Global Compliance Dashboard (Dark Mode).	47
4.24 GUI Design: Superadmin Global Compliance Dashboard (Light Mode).	47
4.25 GUI Design: RBAC User and System Administrator Management Interface. . .	48
4.26 GUI Design: Security Auditor Read-Only Compliance View.	48
4.27 System Output: Automated PDF Audit Dossier Generation.	49
4.28 System Output: SMTP Alert for Rule Violation.	50
5.1 Implemented N-Tier System Architecture	53
5.2 Implemented Service Topology and Container Networking	53
A.1 ISCMS authentication portal.	81
A.2 Two-factor authentication (2FA) verification interface.	81
A.3 Password recovery interface for credential resets.	82
A.4 Policy configuration interface with department-level governance assignment. .	83
A.5 Detailed view of the regulatory framework requirements mapping interface. .	83
A.6 Interface for raw scan results and endpoint telemetry mapping.	84
A.7 Report generation interface featuring Framework and Department filtering. . .	85
A.8 Generated PDF dossier highlighting department-specific compliance summaries.	86

A.9	Department-scoped compliance detail page from the generated PDF.	87
A.10	System Settings interface for managing global configurations and thresholds. .	88
A.11	Expanded user management interface for configuring strict RBAC hierarchies. .	88

List of Tables

2.1	Comparative Feature Matrix: Analyzing GRC Platforms	12
3.1	UC-01: User Authentication	21
3.2	UC-02: Manage Wazuh Telemetry Synchronization	22
3.3	UC-03: Map Framework Controls to Wazuh Rules	23
3.4	UC-04: Monitor Endpoint Compliance Status	24
3.5	UC-05: Resolve Compliance Anomalies	25
3.6	UC-06: Export Audit Reports	26
4.1	External Interface Technical Specifications	51
5.1	Architectural Technology Stack and Implementation Roles	56
5.2	Summary of Tools, Technologies, and Development Stack	59
6.1	TC-01-1: Valid Superadmin Authentication (Positive)	62
6.2	TC-01-2: Invalid Credential Rejection (Negative)	63
6.3	TC-01-3: UI RBAC View Segregation (Positive)	64
6.4	TC-01-4: Unauthorized API Write Attempt (Negative)	65
6.5	TC-02-1: Wazuh API Handshake and Token Retrieval (Positive)	66
6.6	TC-02-2: Wazuh Connection Timeout Exception (Negative)	67
6.7	TC-02-3: Asynchronous Telemetry Polling via Celery Beat (Positive)	68
6.8	TC-02-4: Unstructured JSON Log Ingestion (Positive)	69
6.9	TC-03-1: Deterministic Framework Mapping (Positive)	70
6.10	TC-03-2: Duplicate Mapping Collision (Negative)	71

6.11	TC-04-1: Global Theme Toggling, Light/Dark Mode (Positive)	72
6.12	TC-04-2: Dynamic Compliance Score Recalculation (Positive)	73
6.13	TC-05-1: Asynchronous SMTP Alert Dispatch (Positive)	74
6.14	TC-06-1: Comprehensive PDF Audit Report Generation (Positive)	75
6.15	NFR-TC-01: System Load and Latency Benchmarking	76
6.16	NFR-TC-02: Asynchronous Processing & High I/O Load	76
6.17	NFR-TC-03: Perimeter Defense and Vulnerability Prevention	77
6.18	NFR-TC-04: Cognitive Load and Interface Usability	77
6.19	NFR-TC-05: Environment Portability and Container Resilience	78

List of Abbreviations

ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
CCM	Continuous Controls Monitoring
CI/CD	Continuous Integration and Continuous Deployment
CPU	Central Processing Unit
DOM	Document Object Model
DRF	Django REST Framework
ERD	Entity-Relationship Diagram
FedRAMP	Federal Risk and Authorization Management Program
FISMA	Federal Information Security Modernization Act
FK	Foreign Key
GRC	Governance, Risk, and Compliance
GUI	Graphical User Interface
HCI	Human-Computer Interaction
HIPAA	Health Insurance Portability and Accountability Act
HMAC	Hash-based Message Authentication Code
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
I/O	Input / Output
ISCMS	Intelligent Security Compliance Management System
IDE	Integrated Development Environment
IEC	International Electrotechnical Commission
ISMS	Information Security Management System
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
JWT	JSON Web Token
MIME	Multipurpose Internet Mail Extensions
NIST	National Institute of Standards and Technology

ORM	Object-Relational Mapping
OS	Operating System
PDF	Portable Document Format
PHI	Protected Health Information
PK	Primary Key
QA	Quality Assurance
RBAC	Role-Based Access Control
REST	Representational State Transfer
SaaS	Software as a Service
SCA	Security Configuration Assessment
SHA	Secure Hash Algorithm
SIEM	Security Information and Event Management
SMTP	Simple Mail Transfer Protocol
SPA	Single Page Application
SQL	Structured Query Language
SSL	Secure Sockets Layer
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TOTP	Time-based One-Time Password
UI	User Interface
UML	Unified Modeling Language
UX	User Experience
VPN	Virtual Private Network
WCAG	Web Content Accessibility Guidelines
XSS	Cross-Site Scripting

Chapter 1

Introduction

1.1 Project Background

1.1.1 The Temporal Blind Spot of Audits

Traditional security auditing relies on manual evidence collection and point-in-time assessments. Organizations compile spreadsheets and static configuration snapshots to satisfy regulatory requirements. This creates temporal blind spots because the security status is never continuously validated. Configuration drift after an audit is common and leaves infrastructure exposed to risk even while it appears technically compliant.

1.1.2 The Shift to Continuous Monitoring

To reduce these risks, the cybersecurity sector has adopted Continuous Controls Monitoring (CCM), which replaces manual sampling with automated control validation [1]. CCM tools connect directly to the computing environment and assess configurations and audit logs. Automation ties compliance status to the environment's current state rather than a past snapshot. NIST Special Publication 800-53 Revision 5 calls for a shift from periodic control assessments to continuous monitoring [2]. ISO/IEC 27001:2022 similarly requires continuous risk assessment within an Information Security Management System (ISMS) [3]. Both standards require solutions that connect system data to governance goals. Security Information and Event Management (SIEM) systems provide that data. SIEM engines collect telemetry across the enterprise and convert system events into compliance states. This project builds a mechanism that maps SIEM telemetry to regulatory controls.

1.2 Problem Description

1.2.1 Telemetry Correlation Delays

Converting large volumes of endpoint telemetry to regulatory controls is a practical challenge. SIEM systems generate technical notifications, such as failed SSH logins and file changes, that must be correlated to ISO 27001 or SOC 2 control maps. Manual correlation creates time gaps between events and compliance checks, opening windows of vulnerability. Multiple telemetry feeds can also report conflicting states, and manual processes have no reliable way to reconcile them.

1.2.2 Algorithmic Failures in Legacy Systems

Aggregating compliance metrics across distributed fleets exposes data handling limits in typical GRC systems. Standard relational database implementations are prone to the N+1 problem, where the application issues a separate database query for each security control. This produces a linear $O(N)$ retrieval pattern that degrades as agent counts grow. When response times exceed latency thresholds, the system can no longer support continuous monitoring. The deeper problem is that mapping raw telemetry events to regulatory controls often saturates memory bandwidth at the application level. Without query optimization to load only the necessary parts of relational objects, these systems overload CPU and memory, causing timeouts and data inconsistency. The result is that the system becomes too slow to ingest telemetry at the rate it arrives.

Traditional GRC user interfaces also increase cognitive load for analysts. Static dashboards with poor color palettes lead to alert fatigue. Empirical measurement of alert incidence rates in a Security Operations Center (SOC) shows that many triggered alerts come from benign actions, forcing analysts to wade through noise when triaging events [4]. This slows incident response and raises organizational risk. Dense data grids with no information hierarchy make signal-to-noise filtering a manual task. Over time, the analyst's sensitivity threshold rises until serious security events are overlooked.

1.3 Project Objectives

The ISCMS platform tackles these technology and operational challenges via four development priorities:

- **Automated Telemetry:** Automate the collection and mapping of Wazuh SIEM telemetry to ISO 27001 and SOC 2. The system queries the Wazuh Manager's Security Configuration Assessment (SCA) module, ingests check results, and maps them to a compliance

matrix. A Worst-Case Priority algorithm resolves conflicting evaluations so the compliance dashboard always reflects the worst observed security state.

- **Reduce Latency:** Avoid the N+1 database query problem by leveraging $O(1)$ set-based aggregations via the Django ORM. The system applies query annotation and conditional filtering using SQL to ensure that the dashboard will load in less than 200 milliseconds; filtering and aggregations run directly in the relational layer rather than in application loops. It pre-calculates fleet-wide compliance scores to reduce data round trips and memory allocations on the application host.
- **Security Infrastructure:** Ensure infrastructure and session security via a Dockerized bridge network and isolated PostgreSQL data tier, and a Django REST Framework Role-Based Access Control and stateless JWT token rotation. The system uses a virtual bridge network to isolate the database so the persistence tier is inaccessible to external networks. This bridges with an Identity and Access Management (IAM) system that employs stateless JSON Web Token (JWT) rotation and custom permission classes.
- **UI/UX Engineering:** Manage alert fatigue using a modern user interface that uses flexible design to replace inflexible tables. The design enables rows to wrap and scale as the view is resized. The system leverages depth and gradient scaling to create interactive security cards over embedded telemetry that organizes raw data for decision-making.

1.4 Methodology

1.4.1 Agile Development Lifecycle

The ISCMS platform was designed and developed using the Agile Software Development Lifecycle. Work began with a literature review and structural analysis of the ISO/IEC 27001:2022 and SOC 2 Type II regulatory frameworks. The development team decomposed governance requirements into technical metrics and built a schema to translate operational telemetry to compliance requirements. The team then prototyped the API integration layers to extract the Wazuh SCA module capabilities and tested client connectors for polling and serializing endpoint states, while preserving the central SIEM infrastructure's health and performance.

1.4.2 Sandbox Testing and Evaluation

Backend architecture and implementation drove the subsequent development sprints. Testing showed that database aggregation strategies using set-based operations outperformed conventional relational approaches with synthetic telemetry datasets, which led to the adoption of $O(1)$ set-based query projections. During the evaluation phase, the platform transitioned to a containerized deployment model using Docker virtual bridge networks. This isolated environment

enabled security validation, performance profiling, and stress testing under simulated threat and compliance drift conditions.

1.5 Project Scope

1.5.1 Compliance Framework Boundaries

The ISCMS platform operates within the ISO/IEC 27001:2022 and SOC 2 Type II compliance frameworks. The scope covers the ingestion and mapping of endpoint security telemetry from the Wazuh Manager's Security Configuration Assessment (SCA) module. The system uses the Wazuh REST API to retrieve check results from active agents and correlate them to governance controls, so the compliance state reflects the actual technical configuration. Authenticated requests to the SCA module retrieve payloads describing the pass or fail status of configuration checks on monitored agents. Each check ID is cross-referenced against the ISCMS seed matrix, which maps technical identifiers to governance controls. This correlation translates technical events into regulatory violations.

1.5.2 Technical Implementation Boundaries

The implementation boundary limits this system to a containerized Docker-based deployment. This setup uses a private virtual bridge network for communication between the PostgreSQL database, Redis message broker, Celery workers, and the Django-React application stack. In this architecture, the data persistence tier is not accessible to external networks, providing a baseline for compliance telemetry. Several operational domains are outside the project scope. The system does not enable active Incident Response (IR) activities such as endpoint isolation or malicious process termination. It handles no network packet captures and does not automate configuration drift remediation. The system assesses and reports compliance state but does not alter endpoint configurations; infrastructure remediation remains the responsibility of IT operations teams.

1.6 Feasibility Study

1.6.1 Technical and Economic Feasibility

The ISCMS platform is technically and economically feasible. It uses the open-source Wazuh SIEM ecosystem alongside the Django and React frameworks, and these dependencies can handle telemetry streams at the required scale. Containerizing the stack with Docker decouples the application from the host operating system, so organizations can deploy it on existing infrastruc-

ture without specialized hardware. Economically, the platform avoids the costs of proprietary SaaS compliance solutions. The open-source technology stack eliminates recurring licensing fees and vendor lock-in risks, shifting the financial model from high operational expenditure (OpEx) to a maintenance cost.

1.6.2 Operational Feasibility

The interface and automated telemetry mapping reduce the labor hours required for evidence collection in Security Operations Centers (SOC).

1.7 Areas of Application

1.7.1 Financial Technology (FinTech)

The ISCMS platform suits industries with strict data privacy and governance mandates. In Financial Technology (FinTech), where transaction processing and infrastructure scaling outpace point-in-time auditing, the platform provides the assurance regulatory bodies require. FinTech startups processing payment data can use it to maintain adherence to SOC 2 Type II and internal security policies, reducing the risk of financial penalties or license revocation due to configuration drift in cloud environments.

1.7.2 Healthcare Sector

The platform is also applicable to the Healthcare sector, where organizations handle Protected Health Information (PHI). To prevent data exposure, healthcare organizations validate their infrastructure against standards such as ISO 27001 and HIPAA. The platform converts technical vulnerabilities into compliance violations to maintain a consistent posture across distributed hospital network endpoints. Because all telemetry is stored within the organization's private Docker network, the platform supports entities with strict confidentiality requirements.

1.8 Report Organization

This report consists of seven chapters detailing the design and implementation of the ISCMS platform. Chapter 2 evaluates literature regarding Governance, Risk, and Compliance (GRC) frameworks and Security Information and Event Management (SIEM) integration. Chapter 3 defines the functional and non-functional requirements and establishes the system architecture. Chapter 4 provides the system design, using Unified Modeling Language (UML) diagrams to

represent logical and physical data flows. Chapter 5 focuses on implementation, detailing back-end algorithms and containerized orchestration. Chapter 6 outlines testing protocols and evaluates performance against accuracy and latency metrics. Chapter 7 summarizes the project's contributions and proposes directions for future work.

Chapter 2

Literature Review

2.1 Literature Review

2.1.1 Evolution of Governance, Risk, and Compliance (GRC) Platforms

Traditional security auditing relied on point-in-time assessments that left gaps between evaluation periods. Organizations managed Governance, Risk, and Compliance (GRC) through manual evidence collection, and the compliance state was typically outdated soon after an assessment. Regulatory frameworks now require continuous controls monitoring instead. Continuous Controls Monitoring (CCM) addresses these audit gaps by maintaining an up-to-date compliance state [5]. NIST Special Publication 800-53 Revision 5 and ISO/IEC 27001:2022 both mandate ongoing monitoring to validate control implementation [2, 3]. SIEM-driven compliance offers practical operational benefits. Centralized log management supports audit accountability by preserving data confidentiality, integrity, and availability. Automated analysis establishes operational baselines that support forensic investigations during incidents. Security administrators configure SIEM agents to track software inventories, hardware configurations, and system health. This data feeds into the GRC database and replaces reactive audits with a data-driven security posture.

2.1.2 Security Information and Event Management (SIEM) Integrations

Integrating Security Information and Event Management (SIEM) systems with GRC workflows changes how compliance is tracked. SIEM platforms are telemetry ingestion engines that replace manual log retrieval with automated data correlation. NIST SP 800-53 defines control families, such as Audit and Accountability (AU) and System and Information Integrity (SI), that require automated tracking for continuous validation [2]. Open-source SIEM solutions such as Wazuh have been empirically evaluated and shown to provide adequate security coverage for organisations operating under budget constraints [6]. Wazuh, an open-source SIEM and Extended Detection and Response (XDR) solution, provides aggregation logic that maps compliance events against NIST and ISO matrices. Automated compliance mapping fills the gap between technical events and business risk. When a user modifies a protected configura-

tion file, File Integrity Monitoring (FIM) sensors generate an event log. The SIEM correlates this event against framework matrices and flags the action as a violation of ISO 27001 Annex A.8.9 (Configuration Management) or NIST SP 800-53 control CM-6. Deviations are monitored using dashboards which sum up the number of alerts and identify the non compliant agents for remediation. The Wazuh Security Configuration Assessment (SCA) module is used for CIS-based benchmark compliance in this usage, which demonstrates its effectiveness as a continuous benchmarking validation tool [7].

2.1.3 Performance Bottlenecks in Telemetry Aggregation

As compliance metrics are calculated across distributed fleets, it becomes apparent that there are limitations in data aggregation in traditional GRC platforms. Common relational database designs tend to create what is known as the "N+1 query," with the application making a n+1 LOOP of query operations to retrieve each security control in turn. This means that the time complexity of this operation is linear, $O(N)$, with N being the number of agents being tracked, which decreases the performance of the application in case of having more agents to monitor. High latency prevents continuous monitoring because dashboard rendering times exceed thresholds for operational visibility. The overhead of Object-Relational Mapping (ORM) frameworks compounds the problem. Application-level data processing, where the system retrieves full object graphs before filtering or aggregating in memory, creates bandwidth bottlenecks. This pattern forces the application server to create transient objects, triggering garbage collection cycles that can block compliance logic. In environments where SIEM agents transmit telemetry continuously, these $O(N)$ operations cause a loss of operational visibility.

Relational data access requires moving from sequential $O(N)$ loops to $O(1)$ set-based aggregations. Research into field flexibility in relational databases demonstrates that JSON-capable schemas provide the storage flexibility needed for variable telemetry data without sacrificing the query performance of the relational engine [8]. Developers optimize ORM queries by using eager loading to translate application logic into SQL JOIN clauses. Fetching related entities in a single query allows the database engine to use B-Tree indices efficiently, which reduces execution time and minimizes context switching between the application runtime and the database server. Set-based database aggregations move the workload to the relational engine. Using query projection, the system instructs the database to compute sums, averages, and conditional compliance counts internally. This reduces data transfer across the network from megabytes of row data to simple integer values. It uses conditional aggregation functions to ensure a uniform response time of $O(1)$. Optimizations make dashboards responsive while the organizational asset inventory expands.

2.1.4 Modern Access Control Paradigms

To ensure the compliance data layer, Identity and Access Management (IAM) protocols come in handy. Today, authorization in Distributed systems is based on Role Based Access Control (RBAC). RBAC controls access to system resources based on roles assigned to users rather than assigning permissions directly to individual identities [9]. Authentication in distributed microservices often uses stateless session security via JSON Web Tokens (JWT). The authorization server signs the JWT using asymmetric encryption to prevent tampering and allow local verification without a database lookup. The ISCMS platform applies Zero Trust principles to its API routing logic. The system evaluates the identity context of every inbound request. Custom permission classes intercept traffic at the view layer and verify that the requester holds the RBAC attributes required for the target resource. This enforcement is decoupled from the business logic to maintain a consistent security boundary.

The security lifecycle of stateless JSON Web Tokens (JWT) includes rotation policies to mitigate risks from long-lived credentials. Traditional session IDs require database lookups for verification; JWTs carry signed claims that allow microservices to authenticate users locally. To prevent session hijacking, the platform uses a refresh token rotation scheme. When the client requests a new credential, the server invalidates the previous refresh token and issues a new pair. Research into TOTP-based cloud storage systems confirms that combining JWT tokens with time-based one-time passwords provides a strong multi-factor authentication baseline that defends against replay and session hijacking attacks [10]. The architecture also uses Time-Based One-Time Passwords (TOTP) as a secondary authentication factor. TOTP algorithms rely on a cryptographic secret shared between the server and the client's authenticator app. Both entities use the current Unix timestamp as a factor in a Keyed-Hash Message Authentication Code (HMAC-SHA1) function. Because the 6-digit numeric code changes every 30 seconds, intercepted credentials have limited use. Studies on two-factor authentication architectures confirm that TOTP-based schemes introduce meaningful resistance against replay attacks when combined with short-lived JWT rotation [11]. Client-side interceptors monitor for specific HTTP response states; when the server returns a forbidden status due to a missing or invalid TOTP challenge, the application redirects the user to an authentication view. This lifecycle ensures compliance data remains inaccessible to compromised endpoints.

2.1.5 Containerization and Defense-in-Depth Topology

Enterprise architecture has shifted from monolithic software designs toward microservices and containers. Monolithic systems integrate the frontend, application logic, and database drivers into a single deployable unit, meaning a failure in one component can affect the entire environment. Containerization technologies like Docker standardize runtimes for distributed microservices. Research on the state of the art of container technologies argues that Docker provides strong isolation primitives and orchestration benefits for multi-tier web applications [12]. Docker provides scalability and bounded contexts by leveraging Linux kernel features. The

ISCMS deployment uses isolated Docker virtual bridge networks for its topology, establishing defense-in-depth. The architecture separates the public API layers from the private persistence layers (PostgreSQL, Redis). Binding data tiers to an internal Docker bridge network makes them inaccessible to external scans. The isolation also prevents direct TCP connections to the database, requiring all connections to pass through the secure business-logic layer. Containers also enable reproducible builds, ensuring identical execution environments across development and production systems [13].

2.1.6 Cognitive Load and HCI in Security Dashboards

Security Operations Centers process large volumes of diagnostic data, which can cause alert fatigue. Legacy GRC platforms present analysts with unfiltered data in dense tables, leading to sensory overload and diagnostic errors. Decision support tooling for SOC analysts has been shown to improve signal prioritization and reduce cognitive strain [14]. Research into the empirical distribution of SOC alerts shows that a large fraction of triggered alerts correspond to benign events, confirming that unfiltered data presentation is operationally harmful [4]. Reducing cognitive strain requires applying User Interface (UI) design principles grounded in human-computer interaction research. The ISCMS platform uses modular card components and color scaling, such as red for critical findings and grayscale for benign logs, to guide the analyst's focus. Depth distinctions separate interactive components from background data and structure telemetry into a decision-support tool.

2.2 Analysis of Existing Systems

2.2.1 Manual Spreadsheet Auditing

Traditional governance relies on labor-intensive processes including spreadsheets, task-tracking tickets, and point-in-time audits. This approach works in small-scale environments but does not scale to modern IT infrastructure, as the compliance state observed during an audit degrades shortly after the assessment concludes. Manual sampling and evidence gathering introduces gaps and opportunities for undetected vulnerabilities between audits. The structural limits of spreadsheet applications lead to performance issues and data corruption when tracking large numbers of configuration parameters. Manual mapping also increases the probability of human error as analysts interpret technical data against regulatory constraints.

2.2.2 Vanta (Commercial SaaS)

Vanta operates in the commercial Software-as-a-Service (SaaS) continuous monitoring sector. The platform speeds up evidence collection by deploying proprietary agents and using API

connectors to integrate with cloud providers and identity systems. This commercial solution introduces risks regarding data sovereignty and vendor lock-in. As a multi-tenant cloud environment, Vanta requires organizations to transmit security telemetry to its external infrastructure. This data transmission increases the attack surface. Vanta is a proprietary system, so its closed-source algorithms prevent organizations from independently verifying how telemetry events map to compliance controls. This lack of transparency forces organizations to rely on the provider's internal security assertions.

2.2.3 Drata (Commercial SaaS)

Drata provides automated compliance tracking through a multi-tenant SaaS architecture. While the platform offers integrations and pre-mapped framework tracking, it involves risks regarding external data storage. Organizations using Drata export their infrastructure metadata to the provider's cloud, which can violate data sovereignty requirements in the defense and financial sectors. The closed-source nature of Drata restricts security engineers from inspecting the underlying evaluation logic. This prevents custom modifications to the compliance engine, meaning that organizations with unique architectures cannot extend the system without relying on the vendor. This ensures the platform remains a closed-box system rather than a verifiable tool.

2.2.4 AuditBoard (Enterprise GRC)

AuditBoard's enterprise GRC solution targets the enterprise GRC market and focuses on risk management, ESG reporting, and internal audit synchronization. Despite its breadth of features, AuditBoard can be difficult to deploy. Long deployment timelines, high licensing costs, and reliance on professional services slow adoption compared to containerized solutions. AuditBoard uses proprietary connectors to interface with security tools, creating a closed-box mapping approach. Security engineers cannot modify the logic linking SIEM alerts to organizational risk matrices. This opacity limits the platform's utility in environments that require verifiable compliance states.

2.3 Comparative Analysis

The ISCMS platform is an alternative for organizations that require data sovereignty, technical transparency, and verifiable logic. It integrates with the open-source Wazuh SIEM to achieve automation without transmitting data externally. The Django framework gives organizations the ability to audit, modify, and extend the compliance evaluation algorithms. The single-tenant Docker topology keeps security telemetry within an isolated, customer-controlled bridge network, which limits the attack surface compared to multi-tenant cloud environments.

As shown in Table 2.1, the ISCMS addresses the performance problems observed in propri-

etary systems through database engineering. $O(1)$ set-based aggregations allow the platform to process telemetry logs without the latency penalties of legacy systems that rely on monolithic database scaling.

Table 2.1: Comparative Feature Matrix: Analyzing GRC Platforms

Feature	Traditional Methods (Manual)	Commercial SaaS (Vanta/Drata/AuditBoard)	ISCMS Platform
Deployment Model	Decentralized documents	Multi-tenant public cloud (SaaS risk)	Single-tenant Docker
Data Sovereignty	High (Data remains local)	Low (Data transmitted to third parties)	High (Data confined to internal network)
Performance Complexity	Prone to spreadsheet limits and crashes	Proprietary ORM scaling constraints, API latency	$O(1)$ set-based aggregation
Customizability	Manual modification	Rigid black-box mapping algorithms	Full programmatic control
Cost	High operational labor expenditure	Expensive recurring enterprise licensing	Zero licensing cost

2.4 Summary of Advantages

The ISCMS platform provides operational and technical advantages that address the flaws of legacy auditing and commercial SaaS offerings:

- **Data Sovereignty via Docker Bridge Isolation:** The platform confines all PostgreSQL database interactions to a private virtual network, which ensures compliance telemetry remains isolated from the public internet and external third-party vendors.
- **Transparent, Verifiable Mapping:** By using an open-source Django backend, organizations can programmatically inspect and verify the logic linking raw Wazuh SIEM alerts to governance controls.
- **Deterministic $O(1)$ Database Performance:** Set-based aggregations replace standard ORM loops, allowing the system to render compliance dashboards with low latency regardless of the agent inventory size.
- **Zero-Cost Licensing Architecture:** The operational stack is built with open source technologies such as Wazuh, PostgreSQL and Python, avoiding commercial SaaS fees.
- **Automated Telemetry Deduplication:** The Worst-Case Priority algorithm resolves conflicting telemetry reports from different sensors and assigns the correct compliance label, preventing false states of compliance.

Chapter 3

Requirement Specification

3.1 Existing System

Traditional compliance management systems rely on manual spreadsheets and disconnected Software-as-a-Service (SaaS) platforms. Governance teams translate ISO/IEC 27001:2022 regulatory requirements into technical controls using documents distributed between governance and technical teams. This method is susceptible to human error, cannot manage versioning, and does not enforce consistency between frameworks. Manual configuration state management increases administrative and operational workloads.

Existing auditing systems conduct assessments at scheduled intervals, and by the time of review, data from internal systems may be stale. Legacy platforms lack specialized relational constraints, which causes latency when processing telemetry and makes it difficult to resolve vulnerabilities consistently. The reliance on proprietary SaaS architectures also introduces data sovereignty risks, as compliance metrics and configurations are transmitted to public cloud environments.

3.2 Proposed System

The ISCMS platform is a centralized system that addresses the limitations of manual mapping and asynchronous SIEM aggregation. The system uses a multi-tier architecture to separate concerns while keeping compliance tracking within an isolated environment.

3.2.1 Technology Stack Selection

The Django framework provides a backend environment with an integrated Object-Relational Mapper (ORM) and built-in security middleware. The Django ORM allows for performance optimizations like $O(1)$ set-based database aggregations. By using query projections and conditional filtering, the system keeps high-volume telemetry manageable. The Django REST Framework (DRF) adds serialization pipelines and supports authentication methods like state-

less JSON Web Token (JWT) rotation.

React 18 and Vite are used to build a Single Page Application (SPA) for data visualization. React's Virtual DOM allows the interface to update compliance states without blocking the browser thread. This keeps the dashboard responsive when rendering large volumes of SIEM alerts. By using native ES modules, Vite enhances the development process by minimizing build times while also supporting older tools.

The key database is PostgreSQL, which is used to give ACID adherence to organizational records. Streaming data from SIEM and automated mapping are concurrently read/written to the database. PostgreSQL guarantees that metrics for compliance are kept right through the use of B-Tree indexing and relational constraints. This meets the criteria of ISO 27001 and SOC 2 audits in which the security controls timelines and history need to be verifiable.

To keep the platform responsive during data-heavy operations, the architecture uses Celery and Redis as an asynchronous layer. This decouples tasks like telemetry synchronization from the main application thread to prevent timeouts. Redis is the message broker that moves task payloads between the web server and the background workers. This model allows the platform to scale its ingestion capacity without slowing down the primary API service.

3.2.2 High-Level Architecture

The Presentation Tier is a React SPA that manages user sessions and provides a monitoring interface. It communicates with the API via JSON payloads over HTTPS. By handling state management and rendering on the client side, the system keeps server resources available for data processing and mapping.

The Application Logic Tier, powered by Django REST Framework, manages compliance tasks. This tier handles the Identity and Access Management (IAM) lifecycle and JWT rotation. The Django ORM manages interactions with the database to enforce integrity and execute query aggregations. Asynchronous tasks use Celery and a Redis-based message broker. The system uses a Pub/Sub model to enqueue synchronization tasks, which allows Celery workers to process telemetry from the external Wazuh SIEM Manager. Figure 3.1 shows the protocol flow and multi-tier layout.

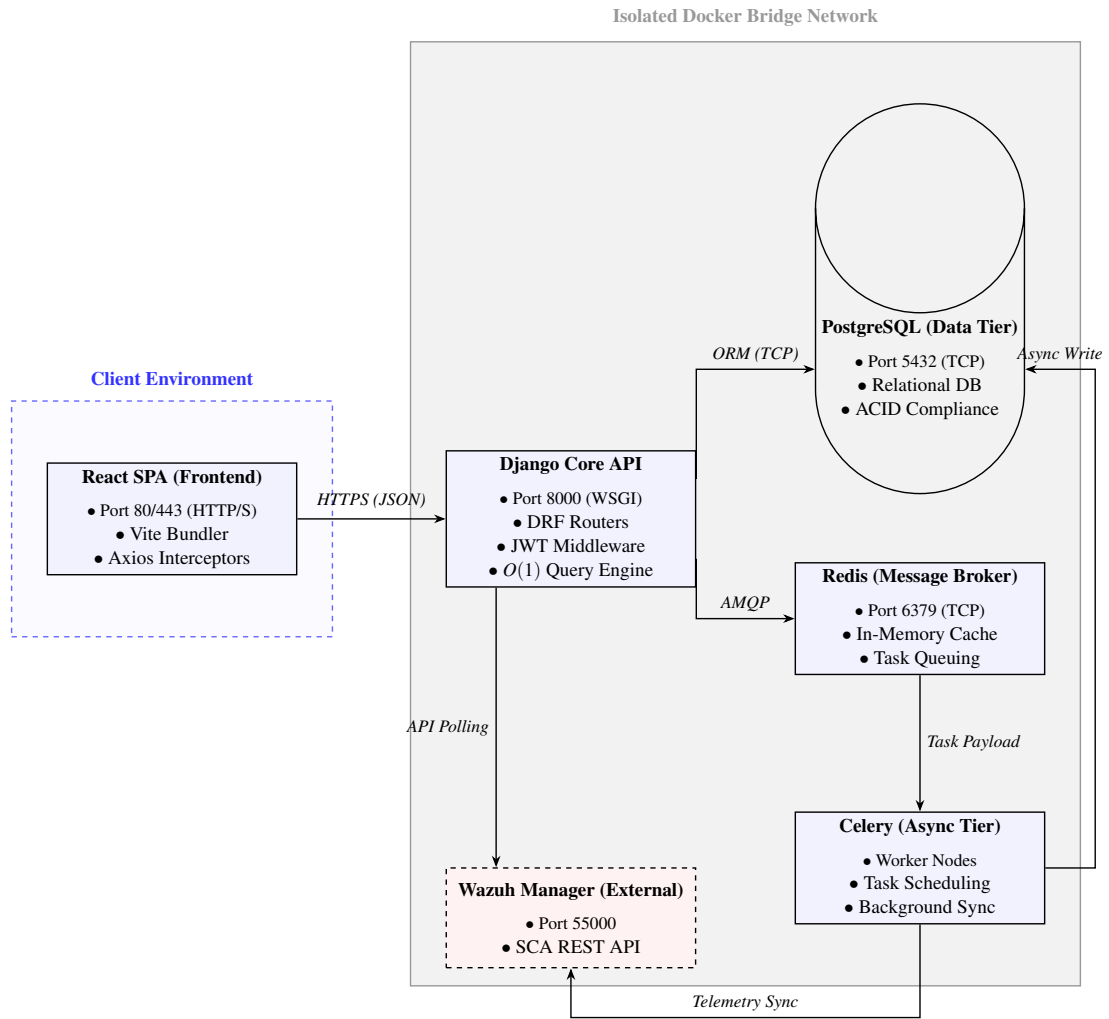


Figure 3.1: Refined N-Tier System Architecture and Protocol Flow

3.2.3 Dockerized Infrastructure and Network Isolation

A Docker virtual bridge network isolates the database from external environments. By keeping PostgreSQL and Redis on a private, internal network, the architecture ensures they are not accessible to external scans. The system uses Docker volumes to prevent data loss across container restarts. The startup sequence uses health-checks to ensure the platform is resilient during deployment. The configuration ensures the Django API container waits for other services before running migrations or starting the server. Figure 3.2 shows the isolation and health monitoring network.

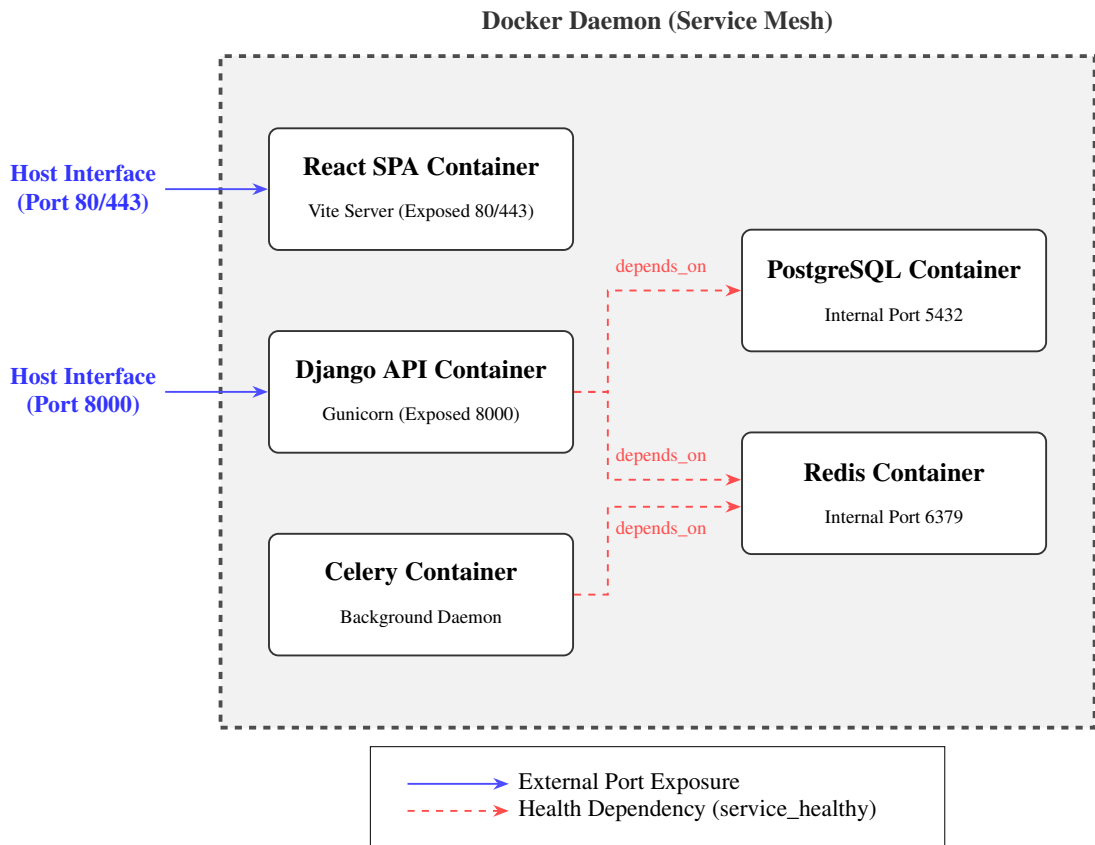


Figure 3.2: Docker Service Mesh: Port Exposure and Health-Check Dependencies

3.3 Requirements Specification

Requirement engineering for the ISCMS platform maps regulatory mandates to technical specifications. Translating ISO/IEC 27001:2022 and SOC 2 Type II governance objectives into functional capabilities ensures that the architecture satisfies both compliance and operational constraints. This analysis establishes the foundation for a system that automates telemetry ingestion and data aggregation.

3.4 Functional Requirements

The functional requirements define the capabilities the ISCMS architecture must have to operate:

- The system must use a 3-tier Role-Based Access Control (RBAC) model with Time-Based One-Time Password (TOTP) authentication.
- The platform must automate the ingestion of security telemetry from the Wazuh SCA module via a REST API.

- The system must resolve conflicting telemetry states using a Worst-Case Priority algorithm to ensure data integrity.
- The platform must generate compliance reports in PDF format for audit documentation.
- The system must provide an interface for managing frameworks, controls, and mapping identifiers.
- The application must use a stateless session lifecycle with JSON Web Token (JWT) rotation.

3.5 Non-Functional Requirements

The non-functional requirements set the operational constraints and performance thresholds for system reliability:

- The system must isolate PostgreSQL and Redis within a private Docker virtual bridge network.
- The backend must use $O(1)$ set-based query aggregations to keep dashboard latency below 200 milliseconds.
- The system must offload Celery and Redis asynchronous background tasks to ensure UI responsiveness.
- The frontend should use a responsive architecture to minimize analyst alert fatigue.
- The platform must ensure high availability through task retry and message queuing mechanisms.
- The system must keep an audit trail for administrative activity and events of the system.

3.6 User Requirements

An effective ISCMS user experience centers on its principal users: the System Administrator and the Security Auditor. The System Administrator requires a control plane to configure the infrastructure without direct database access. They should be able to set synchronization schedules, manage RBAC privileges, and override automated telemetry when needed. These controls need clear visual feedback to indicate what is happening inside the Django ORM.

The Security Auditor uses the platform to assess regulatory compliance. They need a clean view of compliance data without visual clutter. The auditors require an RBAC boundary that grants access to historical logs and PDFs but restricts modification of infrastructure mappings. The dashboard must remain responsive during telemetry ingestion; the React SPA architecture uses techniques like memoization to avoid rendering delays.

3.7 System Models and Components

This section describes how the platform exchanges and flows data to synchronize SIEM telemetry with the internal compliance engine.

3.7.1 Context Diagram

The Context Diagram defines the interaction between the ISCMS and external entities, showing the boundaries of the application and the data streams required. Figure 3.3 shows the flow from human operators to infrastructure components.

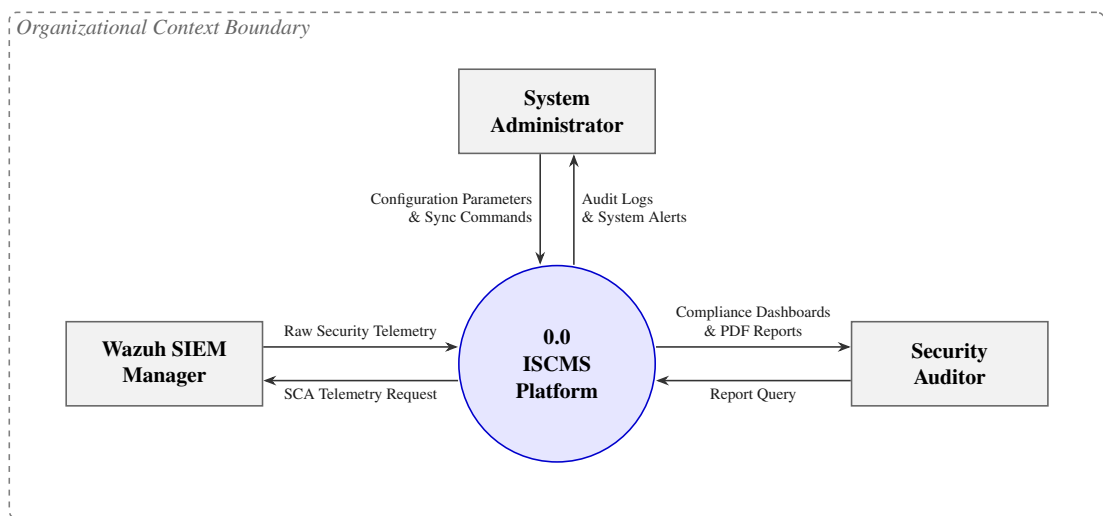


Figure 3.3: Context Diagram: High-Level System Interactions

3.7.2 Data Flow Diagram

The Level-1 Data Flow Diagram (DFD) shows how data payloads move through the platform's architecture and how raw SIEM logs become compliance metrics. Figure 3.4 outlines the pipeline used by the ISCMS.

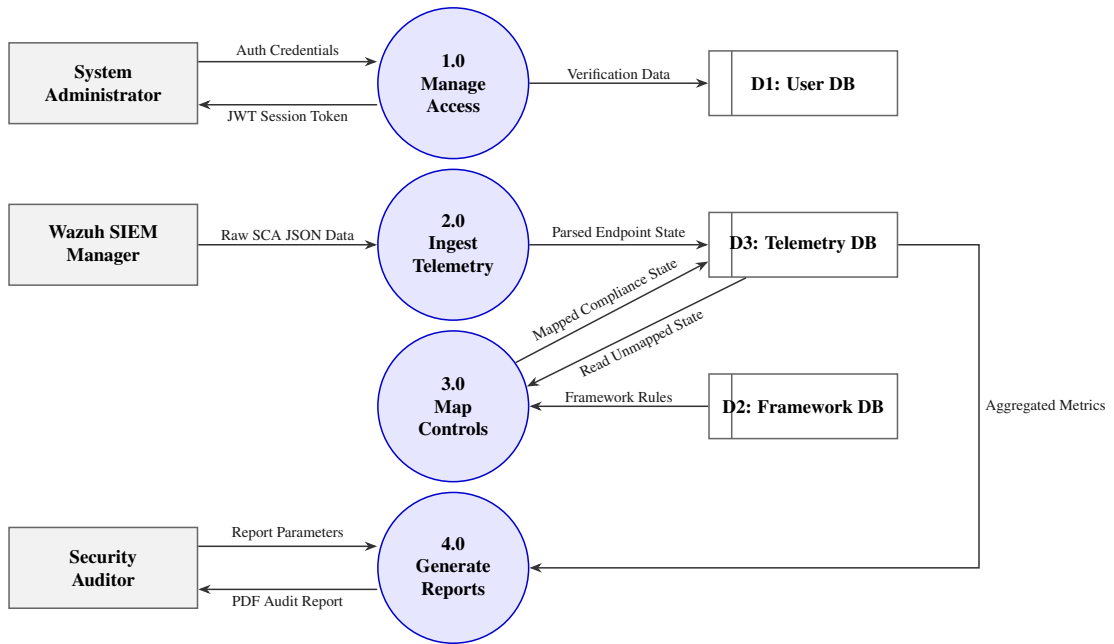


Figure 3.4: Level-1 Data Flow Diagram detailing internal compliance processes.

3.8 Use Cases

The use cases define the interaction sequences available to authenticated actors and provide deterministic state definitions for platform operations.

3.8.1 Use Case Diagram

Figure 3.5 shows the workflows for the System Administrator and the Security Auditor, establishing boundaries based on the platform's RBAC implementation.

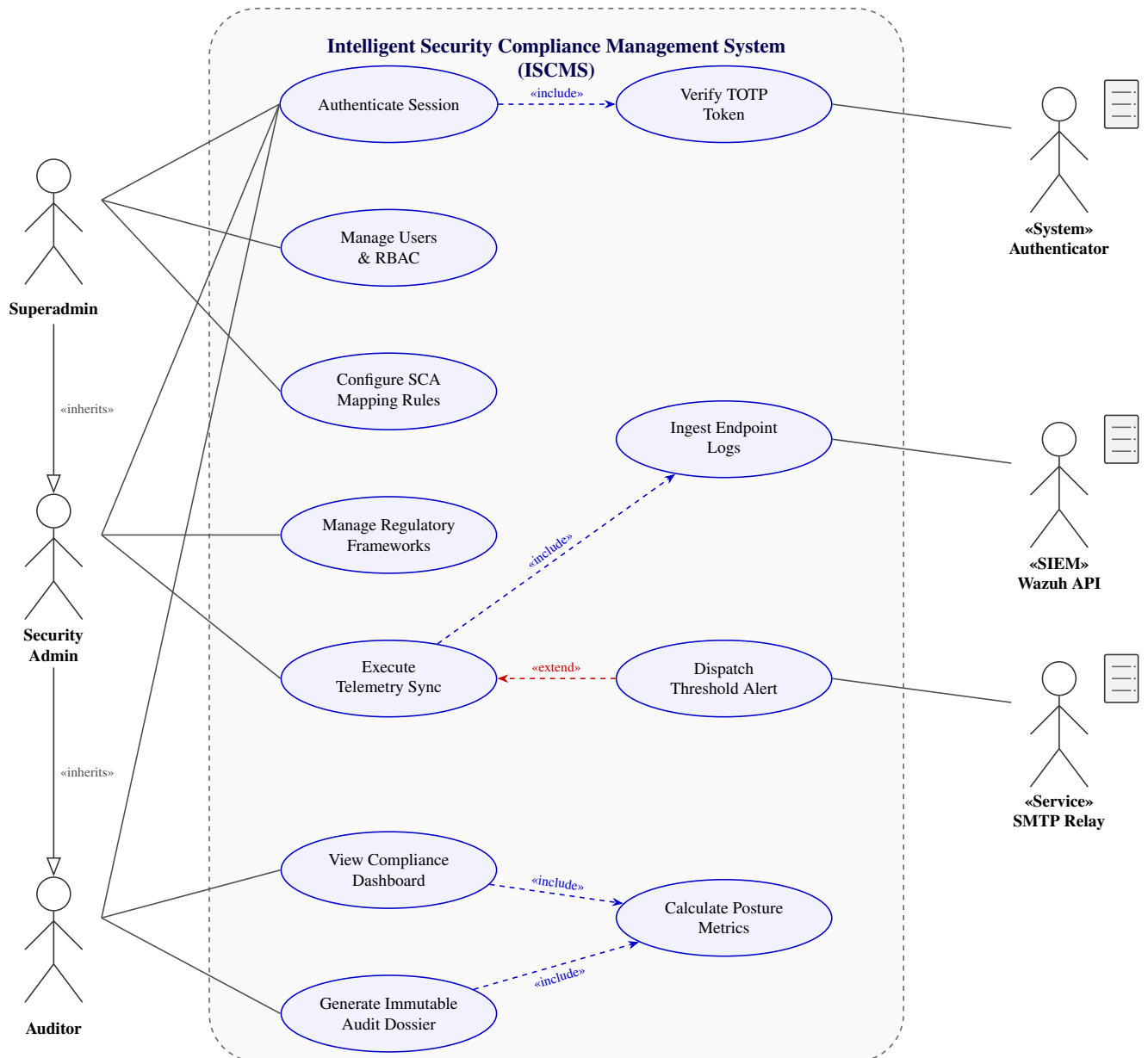


Figure 3.5: Advanced UML Use Case Diagram illustrating RBAC inheritance hierarchies, core compliance interactions, system boundaries, and integration with secondary external actors (Authenticator, SIEM, SMTP).

3.8.2 UC-01: User Authentication

This use case outlines the authentication process for all users accessing the system.

Table 3.1: UC-01: User Authentication

Attribute	Description
Use Case ID	UC-01
Use Case Name	User Authentication
Primary Actor	System Administrator, Security Auditor
Preconditions	The user account is registered and active in the system.
Main Flow	1. The user enters their credentials into the login interface. 2. The system verifies the submitted credentials. 3. The system grants access and displays the default dashboard view.
Alternative Flows	Invalid credentials → The system denies access and displays an authentication error message. Session timeout → The system automatically refreshes the session without interrupting the user.
Frequency of Use	High (Initiated upon every new user session or token expiry).
Priority	Critical (Gatekeeper for all subsequent RBAC routing).
Exception	Account locked due to repeated invalid attempts → Deny access, log event to security audit trail, and prompt system administrator intervention.
Postconditions	The user is successfully authenticated and an active session is established.

3.8.3 UC-02: Manage Wazuh Telemetry Synchronization

This use case describes how telemetry synchronization is managed within the system.

Table 3.2: UC-02: Manage Wazuh Telemetry Synchronization

Attribute	Description
Use Case ID	UC-02
Use Case Name	Manage Wazuh Telemetry Synchronization
Primary Actor	System Administrator
Preconditions	The Administrator is authenticated. <<include>> User Authentication. The external SIEM system is reachable.
Main Flow	1. The Administrator triggers a manual synchronization of security telemetry. 2. The system requests audit data from the external SIEM. 3. The system processes and maps the received data against the linked controls. 4. The system updates the compliance status for each control. 5. The system stores the audit results and notifies the Administrator.
Alternative Flows	SIEM system is unreachable → The system retries the connection and alerts the Administrator if it ultimately fails.
Frequency of Use	High (Continuous via Celery beat background tasks; occasional manual ad-hoc triggers by Superadmin).
Priority	Critical (Forms the core data ingestion pipeline for the compliance engine).
Exception	Wazuh REST API connection timeout or TLS handshake failure → Abort sync, log connection error, and queue retry via Celery exponential backoff.
Postconditions	The system reflects the latest compliance state based on the synchronized SIEM data.

3.8.4 UC-03: Map Framework Controls to Wazuh Rules

This use case details the process of mapping regulatory framework controls to technical Wazuh rules.

Table 3.3: UC-03: Map Framework Controls to Wazuh Rules

Attribute	Description
Use Case ID	UC-03
Use Case Name	Map Framework Controls to Wazuh Rules
Primary Actor	System Administrator
Preconditions	The Administrator is authenticated. <<include>> User Authentication. Frameworks and controls exist in the system.
Main Flow	1. The Administrator selects a SIEM rule and links it to a specific compliance control. 2. The system validates the relationship between the rule and the control. 3. The system saves the mapping configuration. 4. The system confirms the successful linkage to the Administrator. 5. The Superadmin assigns the mapped policy to specific organizational Departments.
Alternative Flows	Invalid control selected → The system rejects the mapping and prompts the Administrator to select a valid control.
Frequency of Use	Low/Medium (Executed during initial platform setup or when regulatory frameworks release new versions).
Priority	High (Defines the relational logic for the entire compliance evaluation).
Exception	Mapping collision or invalid/deprecated Wazuh Rule ID → Prevent database commit, display strict validation error to the Superadmin.
Postconditions	A linkage is established between the SIEM rule and the compliance control for future audits.

3.8.5 UC-04: Monitor Endpoint Compliance Status

This use case explains how administrators monitor the compliance status of connected endpoints.

Table 3.4: UC-04: Monitor Endpoint Compliance Status

Attribute	Description
Use Case ID	UC-04
Use Case Name	Monitor Endpoint Compliance Status
Primary Actor	System Administrator (Trigger) / Celery (System Actor)
Preconditions	The user is authenticated. <<include>> User Authentication.
Main Flow	1. The user navigates to the dashboard view. 2. The system aggregates the latest compliance data. 3. The system visualizes the compliance score and detailed control statuses.
Alternative Flows	No data available → The system displays an empty state with a prompt to synchronize telemetry.
Frequency of Use	High (Serves as the primary operational dashboard for continuous monitoring).
Priority	High (Core visibility and monitoring function).
Exception	Endpoint agent disconnected or offline → Mark endpoint status as 'Stale/Offline', visually highlight the disconnection in the UI, and suspend compliance calculations for that node.
Postconditions	The user is presented with an up-to-date visual compliance posture.

3.8.6 UC-05: Resolve Compliance Anomalies

This use case outlines the workflow for resolving identified compliance anomalies.

Table 3.5: UC-05: Resolve Compliance Anomalies

Attribute	Description
Use Case ID	UC-05
Use Case Name	Resolve Compliance Anomalies
Primary Actor	Security Auditor, System Administrator
Preconditions	The Auditor or Admin is authenticated. Compliance failures have been detected in the system.
Main Flow	1. The user identifies a non-compliant control on the dashboard. 2. The user initiates a resolution workflow and applies corrective actions to the endpoint. 3. The system verifies the change and logs the remediation status.
Alternative Flows	Resolution fails → The system maintains the 'Failed' state and prompts for manual investigation.
Frequency of Use	Medium (Event-driven, based on incoming alerts and failed states).
Priority	Medium/High (Critical for operational remediation, but dependent on external IT actions).
Exception	Insufficient RBAC permissions to apply resolution (e.g., Auditor attempting Admin action) → Deny action, return HTTP 403 Forbidden, and log unauthorized attempt.
Postconditions	The compliance anomaly is resolved and recorded in the historical audit ledger.

3.8.7 UC-06: Export Audit Reports

This use case describes the generation and export of formal compliance audit reports.

Table 3.6: UC-06: Export Audit Reports

Attribute	Description
Use Case ID	UC-06
Use Case Name	Export Audit Reports
Primary Actor	Security Auditor
Preconditions	The Auditor is authenticated. <<include>> User Authentication. Compliance data exists.
Main Flow	1. The Auditor selects a target Framework and an optional target Department to filter the scope of the report. 2. The system compiles the historical audit data scoped to the selected filters. 3. The system generates a formatted PDF document. 4. The system prompts the Auditor to download the document.
Alternative Flows	Generation failure → The system displays a generation error message and logs the failure.
Frequency of Use	Low (Triggered periodically based on audit cycles, e.g., monthly or quarterly).
Priority	High (Primary executive deliverable for external governance bodies).
Exception	PostgreSQL data retrieval timeout or PDF template rendering failure → Abort generation, display generic error modal, and log backend stack trace.
Postconditions	A timestamped audit report is delivered to the Auditor.

Chapter 4

Design

4.1 System Architecture

The Intelligent Security Compliance Management System (ISCMS) uses an N-tier architecture. Separating the frontend, API, background workers, and database allows for scaling and debugging of individual components without affecting the entire system. Figure 4.1 shows the four main layers.

4.1.1 Client and Presentation Tier

The frontend is a React 18 Single Page Application (SPA). Offloading the rendering logic to the browser reduces server load and allows analysts to navigate between compliance views without full page reloads.

4.1.2 Application and Middleware Tier

The backend uses the Django REST Framework as its middleware layer. It serves as the primary entity that verifies JWTs and enforces role-based access before processing queries. All database interactions are routed through Django to maintain a consistent security boundary at the data layer.

4.1.3 Asynchronous Processing Tier

Retrieving telemetry from the Wazuh SIEM is an I/O-intensive process. Executing these tasks on the main Django thread would cause API latency and timeouts. To prevent this, the system offloads I/O tasks to background Celery workers using Redis as the message broker.

4.1.4 Data Persistence Tier

PostgreSQL 18 is the primary database for its ACID guarantees. The schema uses relational tables for mapping frameworks to controls, while the JSONField type stores unstructured SIEM logs.

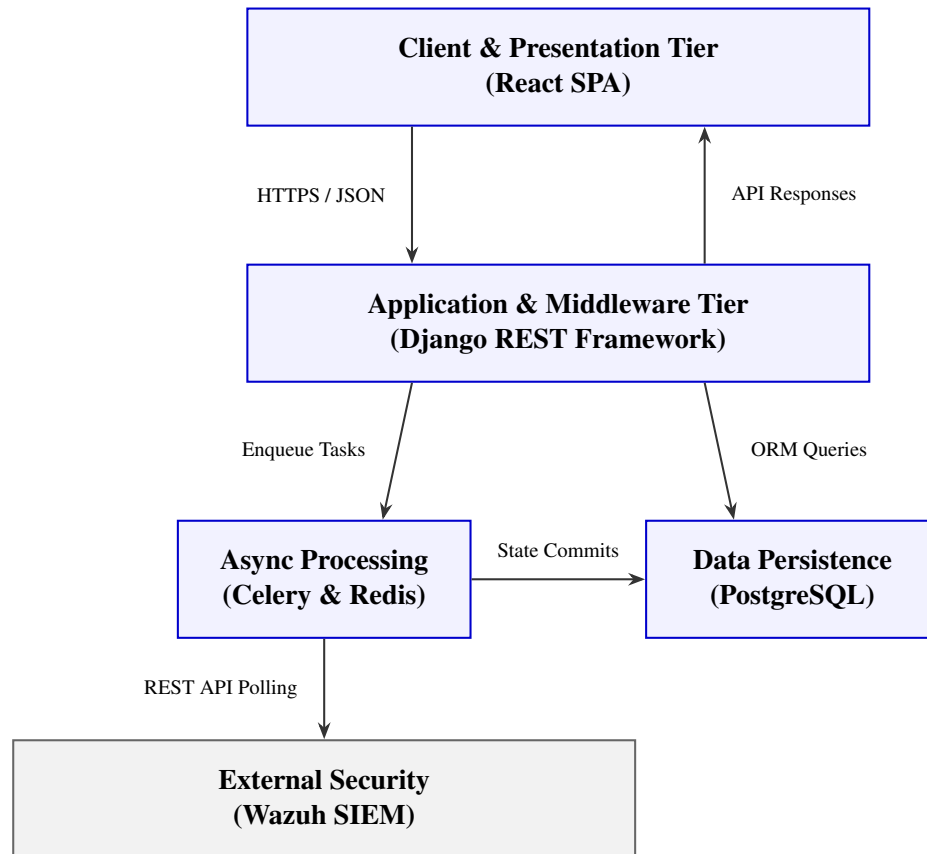


Figure 4.1: Tier-wise System Architecture of the ISCMS Platform

4.2 Design Constraints

The design of the ISCMS involved addressing several technical and environmental constraints.

4.2.1 Technical Constraints

For security, the database and Redis broker operate inside a private Docker bridge network. Their ports are not exposed to the host machine. Access to data requires requests through the Django API container.

The integration with the Wazuh API introduced rate limits and network latency. Celery workers are configured to batch and throttle polling requests. Consequently, the dashboard displays the compliance state from the last successful synchronization rather than a real-time feed.

On the frontend, rendering large compliance grids can impact browser performance. Loading thousands of DOM nodes simultaneously can block the main thread. React virtualization addresses this by rendering only the visible elements.

4.2.2 Infrastructure and Operational Constraints

External factors, such as restricted access to university facilities, required the transition to developing and testing the architecture in a home-lab environment.

Running a full Wazuh SIEM manager locally is resource-intensive and can impact system stability. To manage this, a distributed configuration was used. One system was dedicated to running Wazuh, while another was used for development and as a monitored endpoint. The primary infrastructure was on a separate server, and Git was used to keep things consistent among the various development environments.

4.3 Design Methodology

The platform has been developed in an iterative fashion through prototyping. The system was broken down into separate modules to be tested before being integrated.

The four key stages of this project involved:

- **Phase 1 (Prototyping):** The first phase centered around the fundamental idea. This included setting up a basic Django backend and testing the Wazuh API to confirm successful retrieval of Security Configuration Assessment (SCA) telemetry.
- **Phase 2 (Core Architecture):** The second phase involved designing the PostgreSQL database schema to link frameworks to rules. JWT authentication and role-based routing were also implemented to secure the API.
- **Phase 3 (Asynchronous Telemetry):** Celery and Redis were added to handle heavy SIEM data ingestion. This established a background worker fleet for data processing and deduplication.
- **Phase 4 (Frontend & Visualization):** The final phase involved building the React frontend. This included rendering data grids, implementing state management, and configuring the server-side PDF generator for audit reports.

4.4 High Level Design

The system architecture is represented through several views that cover software components, physical deployment, and operational sequences.

4.4.1 Component Diagram

Figure 4.2 shows the interactions between system modules. The React frontend sends HTTPS requests to the Django API Gateway. The IAM module is used to check the JWT and the Compliance Engine then converts the request into a database query. A separate Wazuh Polling Service retrieves alerts and stores them in PostgreSQL.

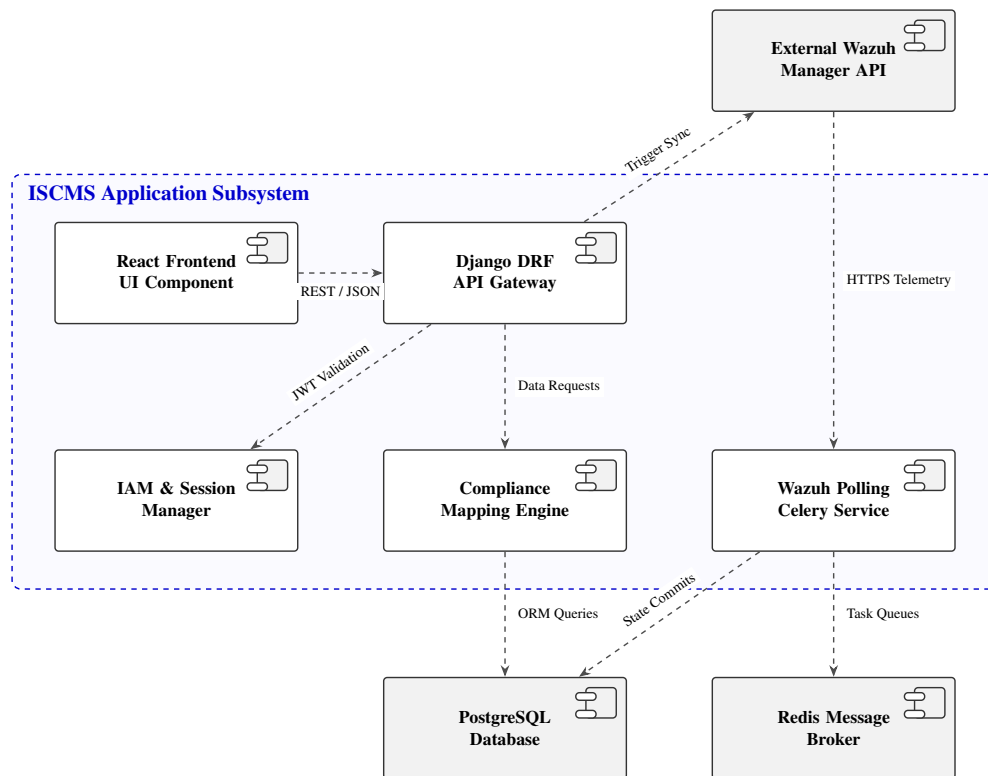


Figure 4.2: Conceptual View: UML Component Diagram

4.4.2 Deployment Diagram

Figure 4.3 outlines the Docker container environment. The Django backend, Celery workers, Redis, and PostgreSQL are deployed on the same host but isolated within a private bridge network. This keeps the database inaccessible from external networks.

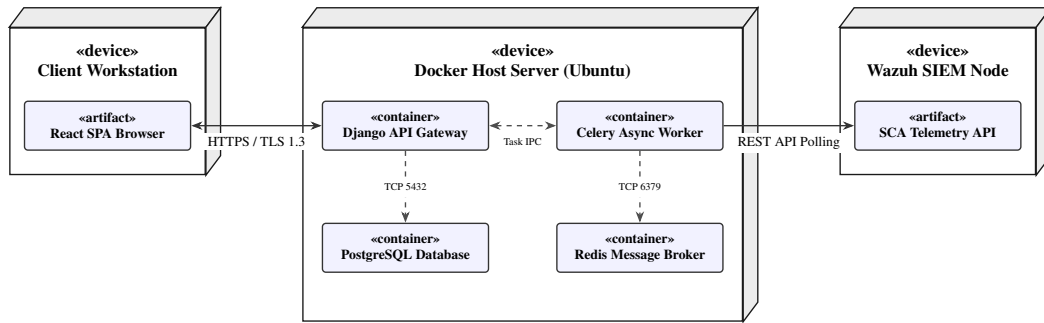


Figure 4.3: Physical View: UML Deployment Diagram

4.4.3 System Activity Diagrams

The activity diagrams below show the control flow for core platform features. Standard UML semantics, such as decision nodes and synchronization forks, are used to represent the system logic.

4.4.3.1 Authentication and IAM Routing

Figure 4.4 shows the login process. Django verifies credentials against PostgreSQL. If the verification fails, the system logs the event and denies the request. Upon successful authentication, the system issues a JWT and evaluates the user's role. Administrators are directed to configuration settings, while Auditors are restricted to read-only dashboards.

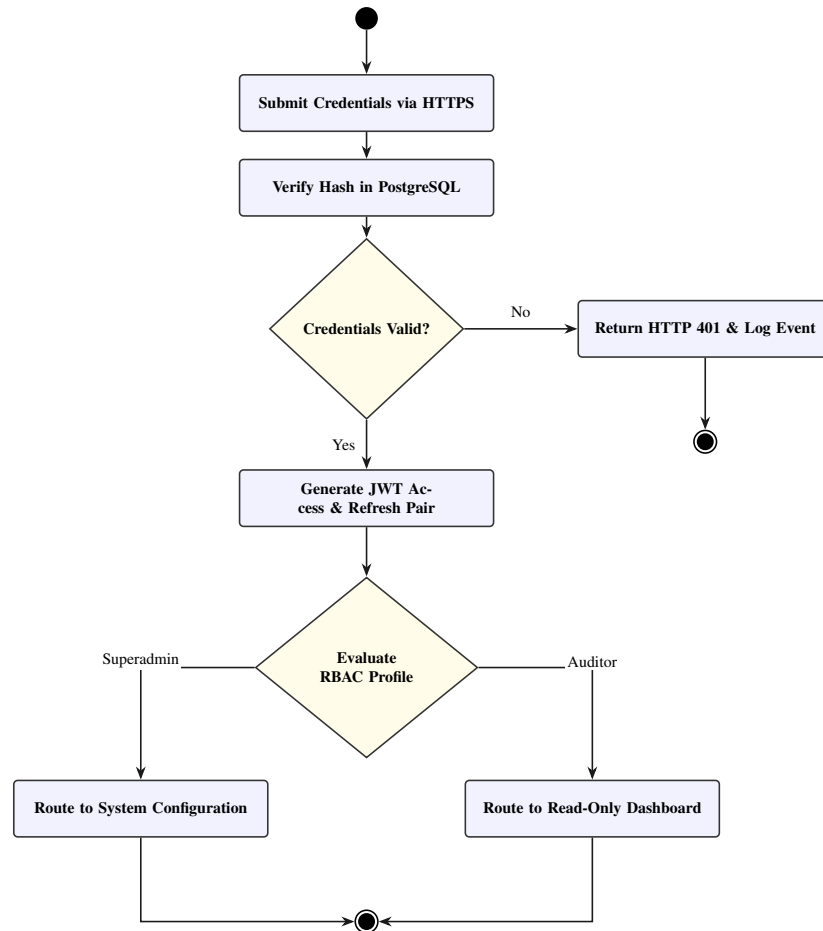


Figure 4.4: Activity Diagram: Authentication and IAM Routing

4.4.3.2 Asynchronous Telemetry Ingestion

Figure 4.5 shows the asynchronous ingestion process. When a synchronization task is triggered, the main thread returns an HTTP 202 status to the client. Simultaneously, the Celery worker polls the Wazuh API. If the SIEM is unreachable, the system uses exponential backoff. Once telemetry is retrieved, the system parses the JSON data and updates the database.

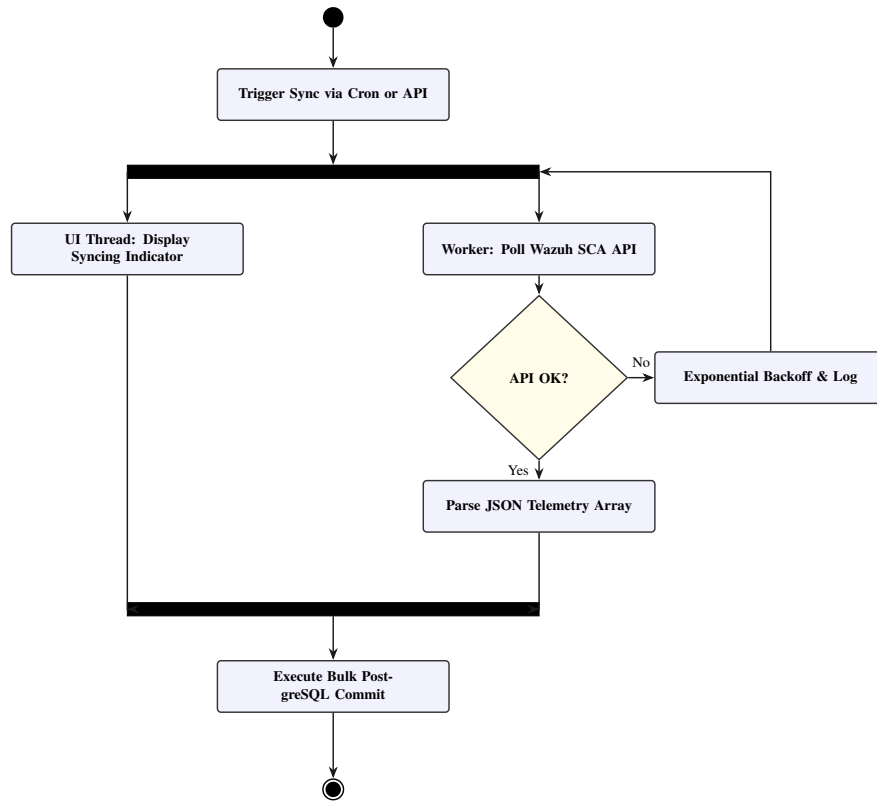


Figure 4.5: Activity Diagram: Parallel Asynchronous Telemetry Ingestion

4.4.3.3 Compliance State Evaluation

The mapping of technical alerts to policies like ISO 27001 is shown in Figure 4.6. The system uses a worst-case priority algorithm; if a technical check fails on a single node, the corresponding governance control is marked as non-compliant. This evaluation continues until all policies are processed.

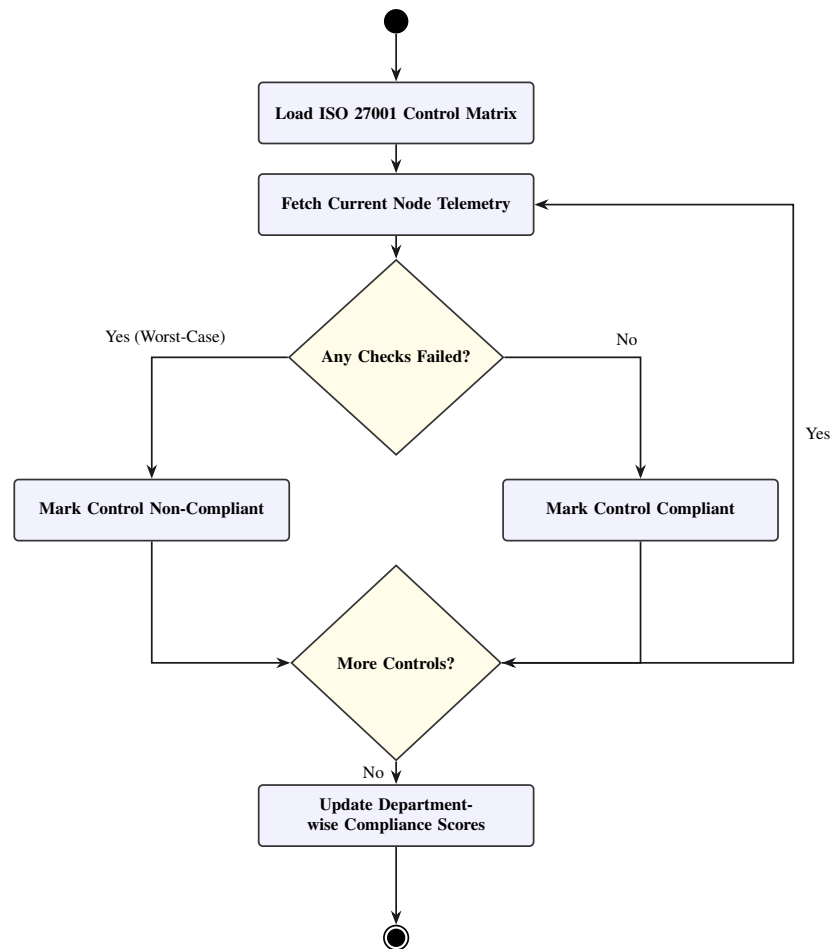


Figure 4.6: Activity Diagram: Iterative Compliance Evaluation and Worst-Case Priority

4.4.3.4 Audit Report Generation

The audit report generation process is automated, as shown in Figure 4.7. When a user selects a framework, the backend aggregates relevant database records. This data is injected into an HTML template, and a PDF is generated on the server for delivery to the client.

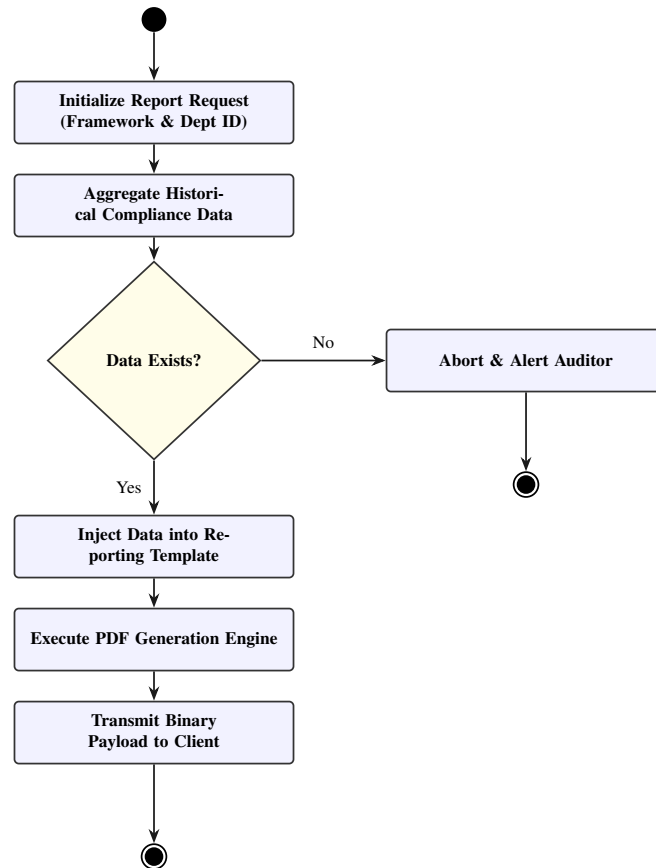


Figure 4.7: Activity Diagram: Automated PDF Audit Report Generation

4.4.3.5 Framework Rule Mapping and Validation

Administrators map Wazuh rules to framework controls as shown in Figure 4.8. Before saving the mapping, the system queries the SIEM API to verify the rule ID. If true the system looks for existing mappings in the database. This validation prevents accidental overwriting of existing configurations.

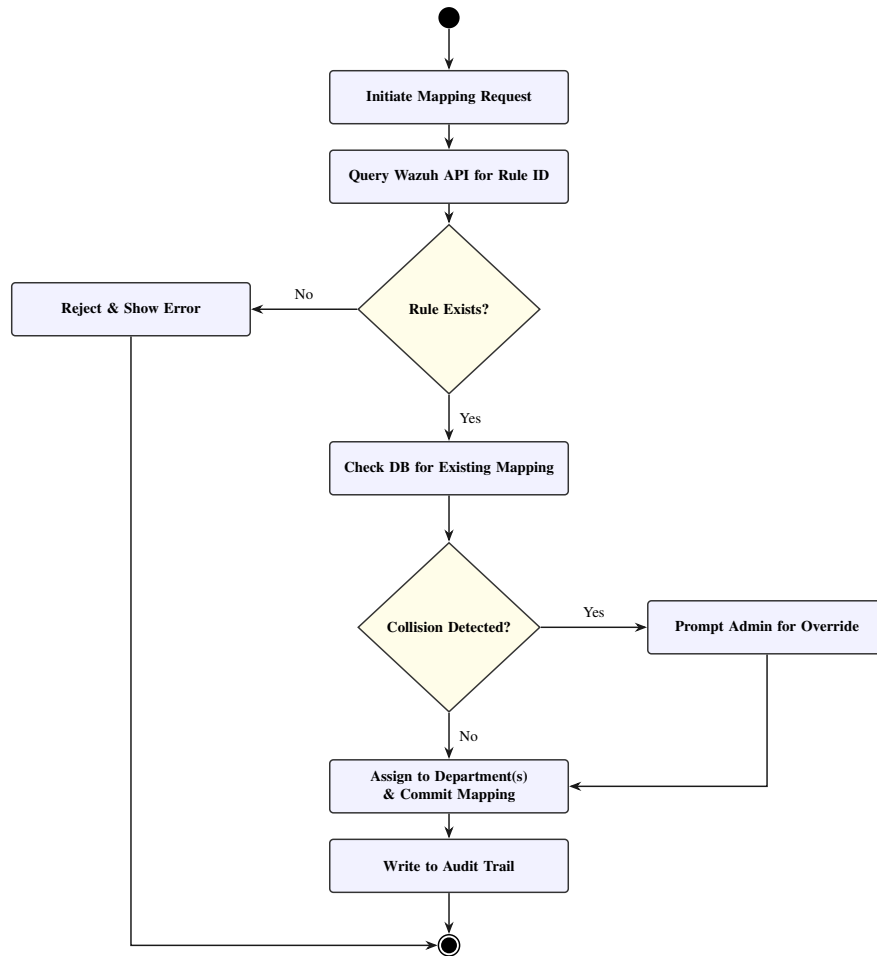


Figure 4.8: Activity Diagram: Framework Rule Mapping and API Validation

4.4.3.6 Real-Time Alerting and Notification

The platform includes an alerting pipeline as shown in Figure 4.9. When a compliance check fails, the engine evaluates its severity. Minor issues are recorded in the database logs, while critical failures trigger automated notifications via email and webhooks to auditors.

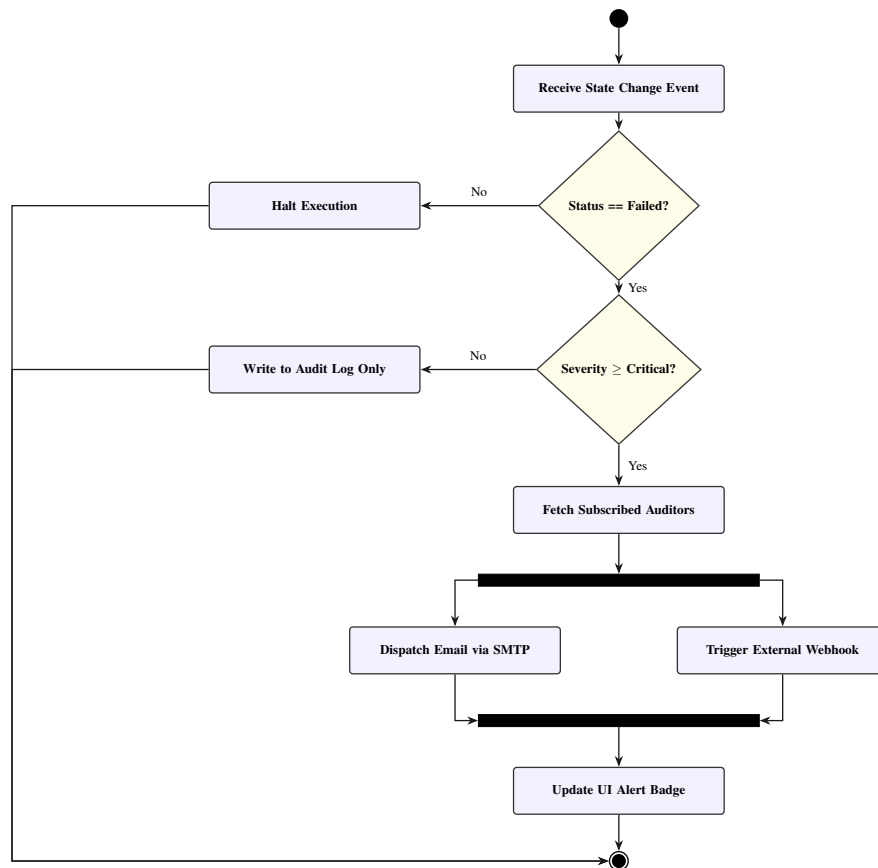


Figure 4.9: Activity Diagram: Real-Time Alerting and Notification Fan-out

4.4.4 System Sequence Diagrams

The sequence diagrams below trace the linear interactions between internal system components for each use case. Each diagram is identified by an SD-XX-Y code that maps directly to the parent UC-XX use case.

4.4.4.1 SD-01: User Authentication

The authentication sequence covers both the success and failure paths for UC-01. A valid credential submission results in JWT issuance, while an invalid submission terminates with an HTTP 401 response. These two paths are documented as SD-01-1 and SD-01-2 respectively.

Figure 4.10 traces the success path. The client submits credentials via POST /login, the IAM module verifies the password hash against the database, signs a JWT, and returns it to the client.

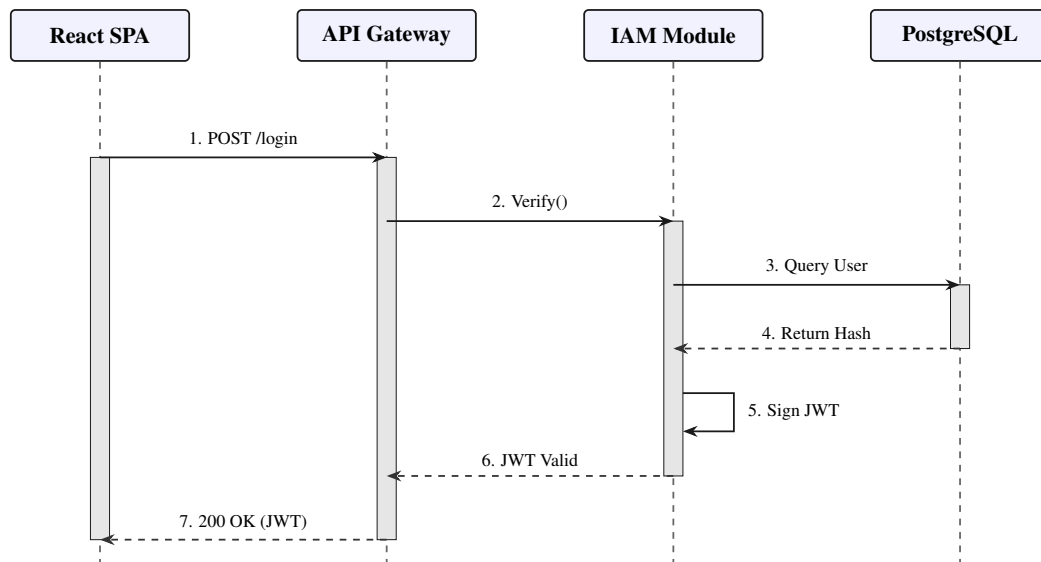


Figure 4.10: SD-01-1: Valid Authentication Flow

Figure 4.11 traces the failure path. The submitted password hash does not match, and the system returns an HTTP 401 response without issuing any tokens.

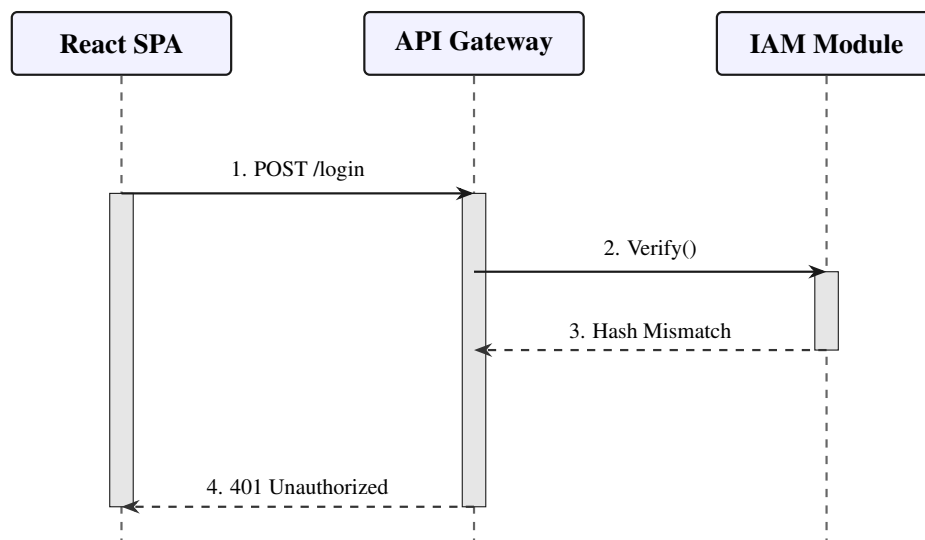


Figure 4.11: SD-01-2: Invalid Authentication Flow

4.4.4.2 SD-02: Manage Wazuh Telemetry Synchronization

This sequence traces UC-02, the telemetry synchronization pipeline. The Django API dispatches the task to a Celery worker, which polls the Wazuh SIEM, maps the returned rules to framework controls, and commits the resolved states to PostgreSQL.

Figure 4.12 documents the linear success path for the synchronization operation.

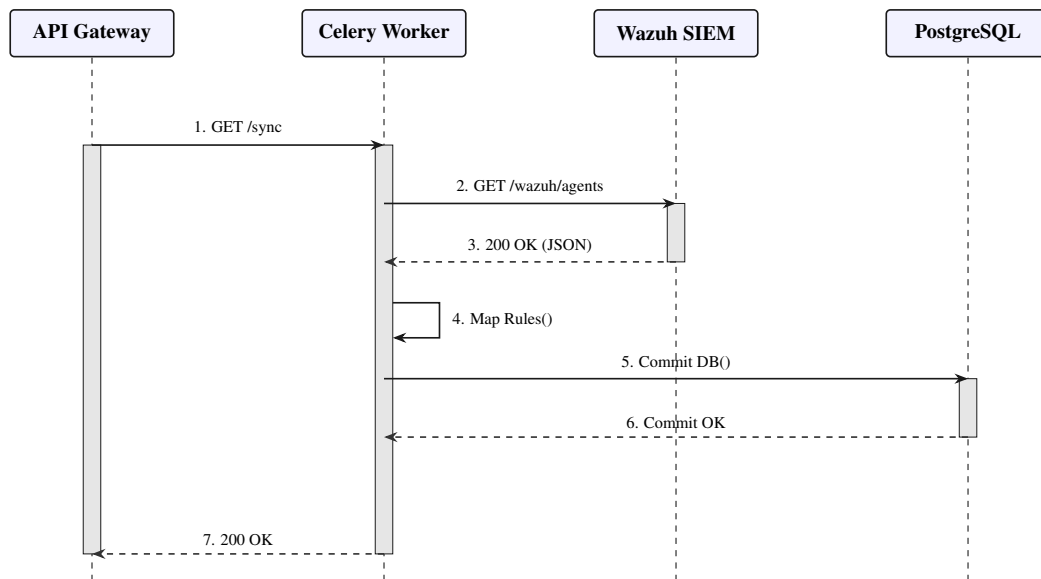


Figure 4.12: SD-02-1: Telemetry Synchronization Success

4.4.4.3 SD-03: Map Framework Controls to Wazuh Rules

This sequence covers UC-03, the framework-to-rule mapping operation. SD-03-1 documents the success path where a new mapping is validated and persisted. SD-03-2 documents the collision path where the backend detects and rejects a duplicate entry.

Figure 4.13 traces the success path for creating a new control mapping.

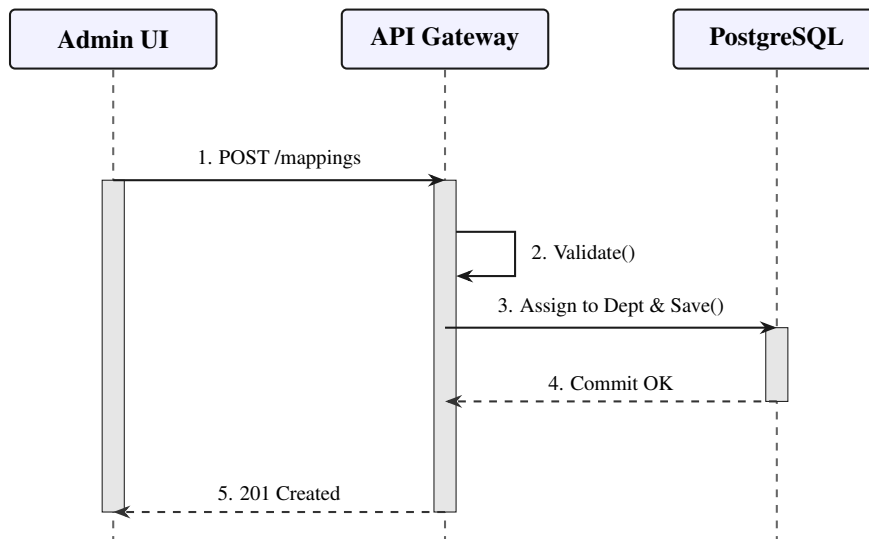


Figure 4.13: SD-03-1: Map Controls Success

Figure 4.14 traces the collision path where the backend detects a duplicate mapping and rejects it with HTTP 400.

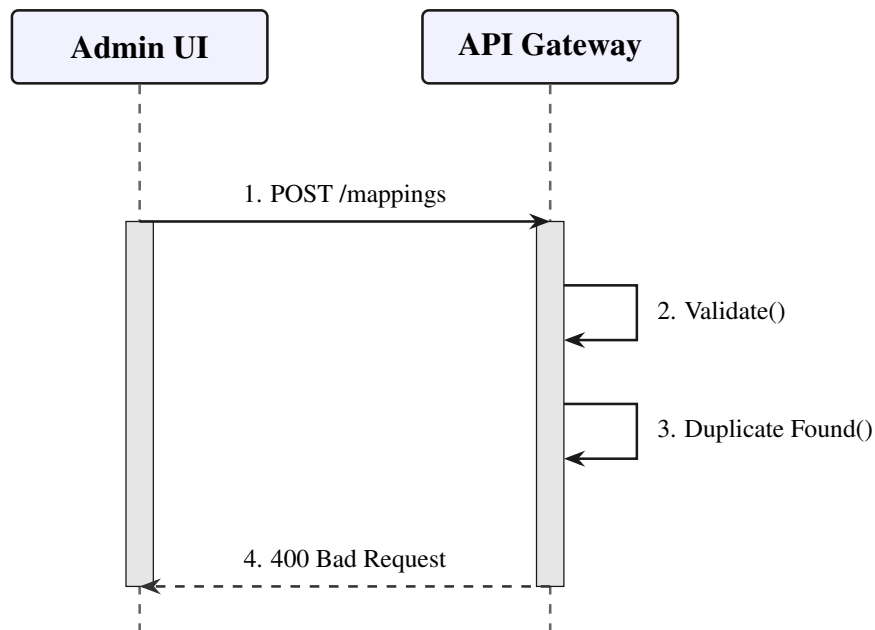


Figure 4.14: SD-03-2: Map Collision

4.4.4.4 SD-04: Monitor Endpoint Compliance Status

This sequence traces UC-04, the dashboard compliance monitoring flow. The React SPA requests the aggregated compliance metrics from the API, which queries PostgreSQL and returns the calculated data array for visualization on the dashboard gauge.

Figure 4.15 documents the data retrieval path for the compliance dashboard.

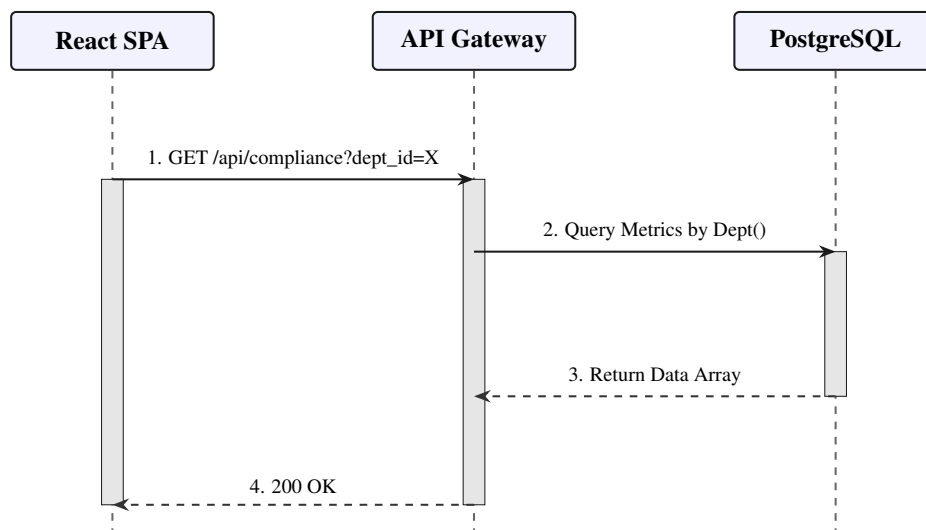


Figure 4.15: SD-04-1: Monitor Endpoint Compliance Status

4.4.4.5 SD-05: Resolve Compliance Anomalies

This sequence traces UC-05, the anomaly remediation workflow. An administrator submits a remediation request through the UI. The API dispatches the task to a Celery worker, which updates the control status in PostgreSQL and confirms the resolution.

Figure 4.16 documents the remediation dispatch and confirmation path.

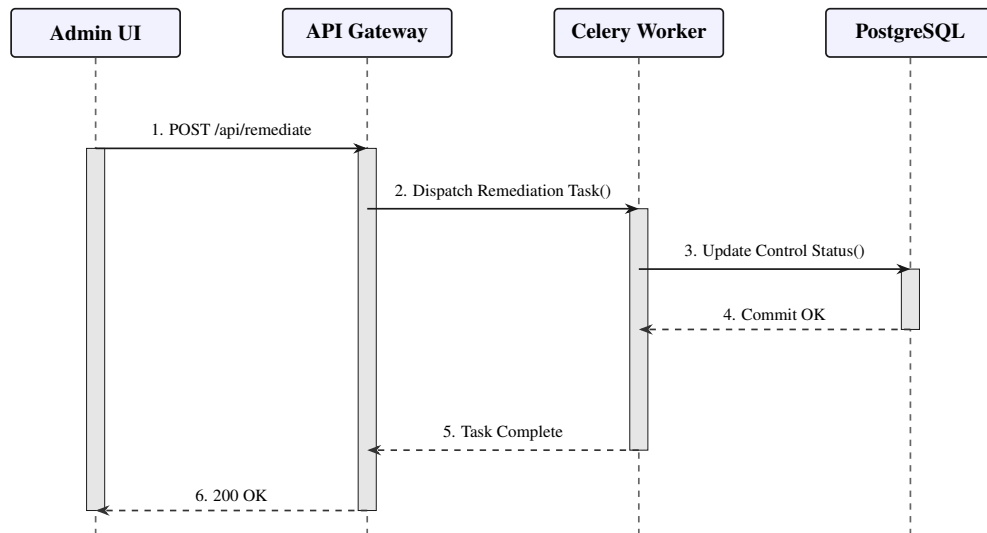


Figure 4.16: SD-05-1: Resolve Compliance Anomalies

4.4.4.6 SD-06: Export Audit Reports

This sequence traces UC-06, the PDF audit report generation pipeline. The auditor requests a compliance report, the API queries historical data from PostgreSQL, renders the PDF binary, and streams it back to the client browser.

Figure 4.17 documents the report generation and delivery path.

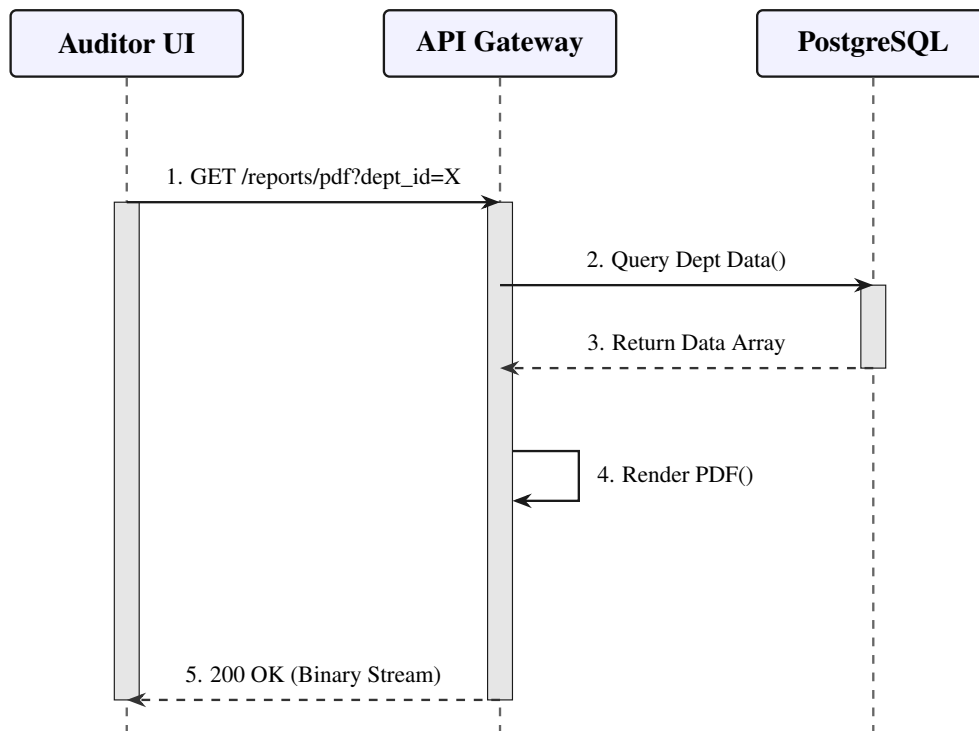


Figure 4.17: SD-06-1: Report Generation

4.5 Low Level Design

Low-Level Design (LLD) takes into account the high-level system design and converts it into basic software components. This section details the individual structures, persistence models and relational bindings that control the core enterprise business logic for the ISCMS platform.

4.5.1 UML Class Diagram

Classes and their relationships that compose the Django ORM are depicted in Figure 4.18. Relational constraints link regulatory frameworks to telemetry logs at each monitored endpoint.

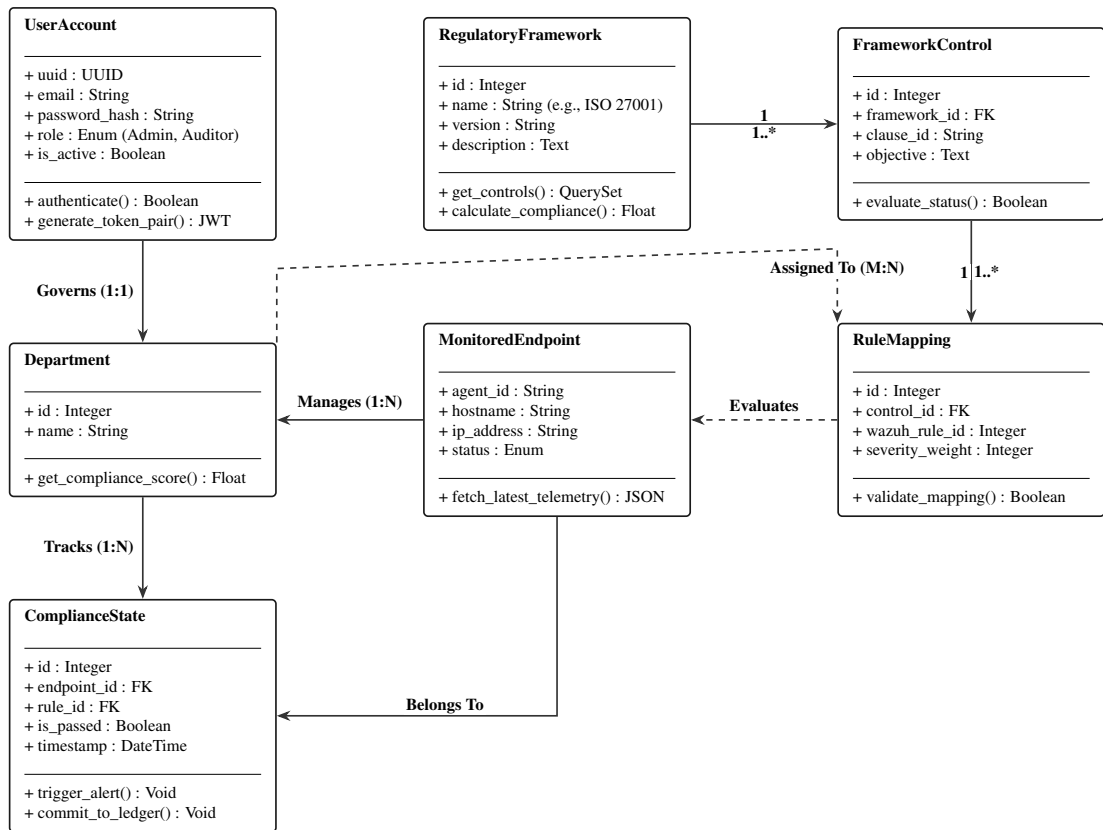


Figure 4.18: UML Class Diagram: Core Object-Relational Models with Department Governance

4.6 Database Design

The system uses PostgreSQL for persistence. The tables are structured in Third Normal Form (3NF) to avoid data anomalies during update operations. Given the high volume of incoming SIEM alerts, ACID compliance prevents data corruption during concurrent transactions.

4.6.1 Physical Data Model (Core ERD)

Figure 4.19 details the physical database structure. It defines the specific column types and foreign key relationships required for the PostgreSQL deployment.

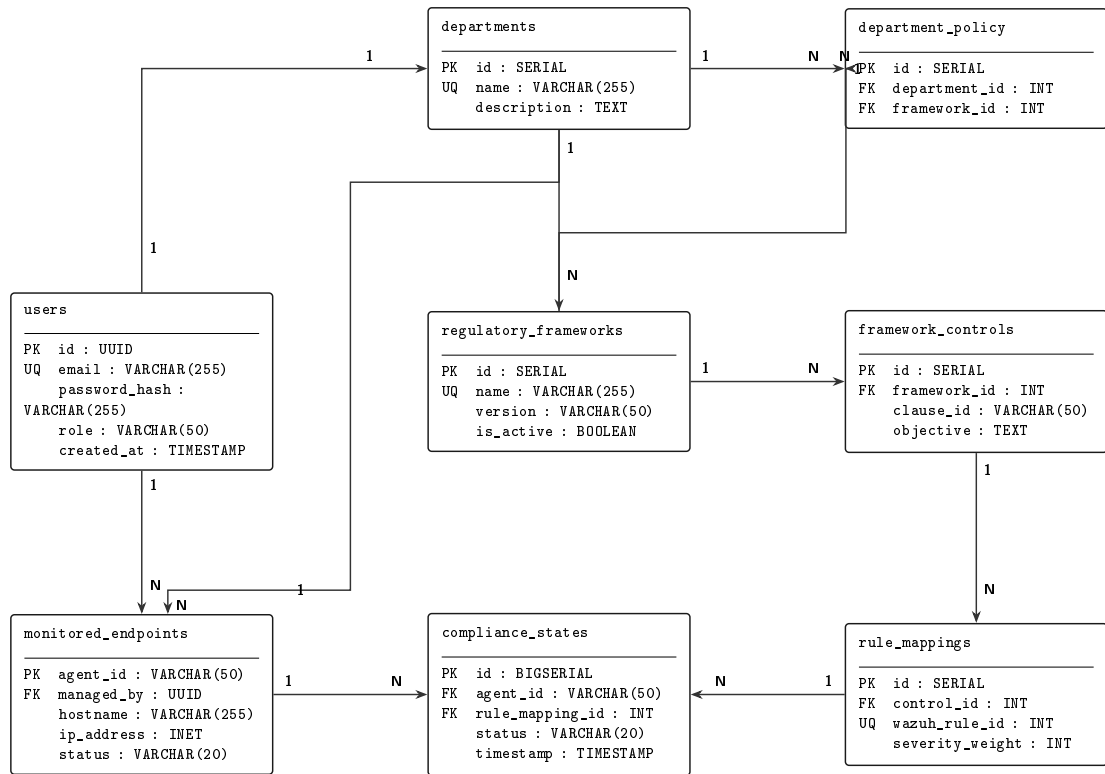


Figure 4.19: Physical Database Schema: Department-Governed Relational Data Model

4.6.2 Extended Entity Relationship Diagram (EERD)

Figure 4.20 presents the Extended Entity Relationship Diagram (EERD), which adds semantic constraints to the logical data model.

The disjoint (**d**) specialization indicates that a User Account is strictly partitioned into either a System Admin or a Security Auditor subtype. This enforces database-level RBAC. The Compliance State is modeled as a weak entity because an alert has no independent existence without a parent Monitored Endpoint.

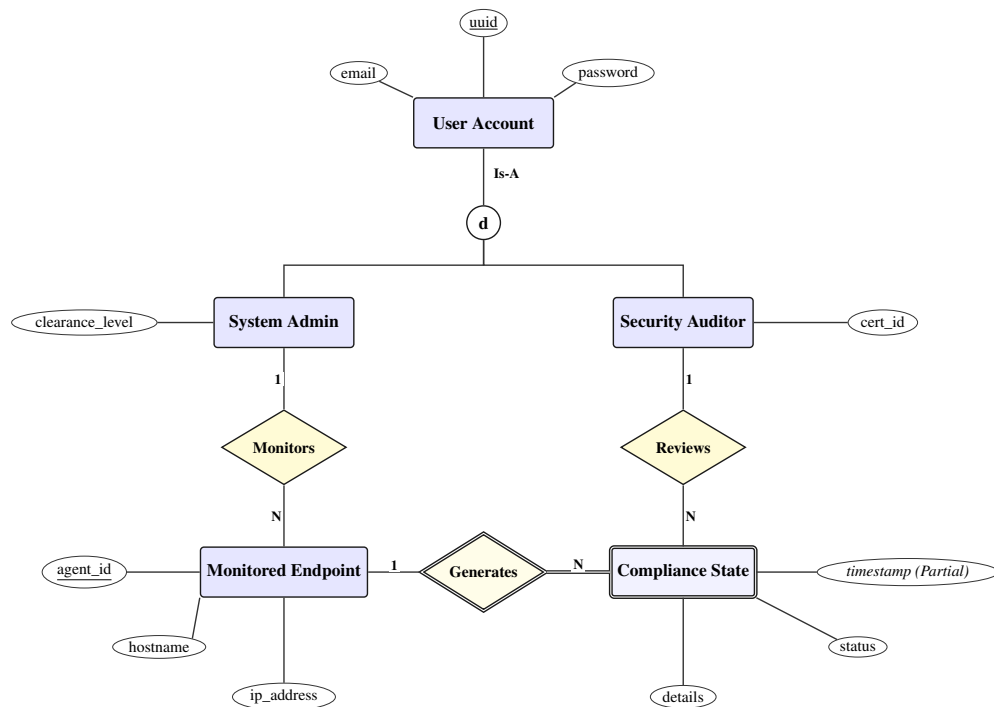


Figure 4.20: Extended Entity Relationship Diagram (EERD) with Constraints and Attributes

4.7 Graphical User Interface (GUI) Design

The frontend is a React Single Page Application (SPA) designed to present compliance telemetry without visual clutter. The screenshots below demonstrate the default dark mode, though the application supports dynamic theme switching.

4.7.1 Authentication and Identity Management

The authentication portal validates user credentials and issues a JSON Web Token (JWT). The application then initializes the dashboard corresponding to the user's RBAC role.

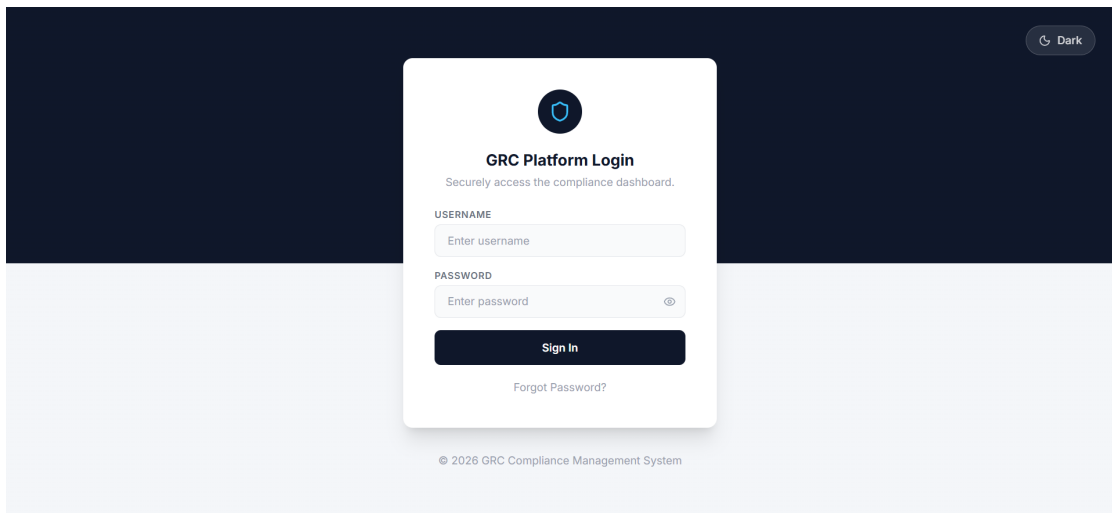


Figure 4.21: GUI Design: Secure User Authentication Portal.

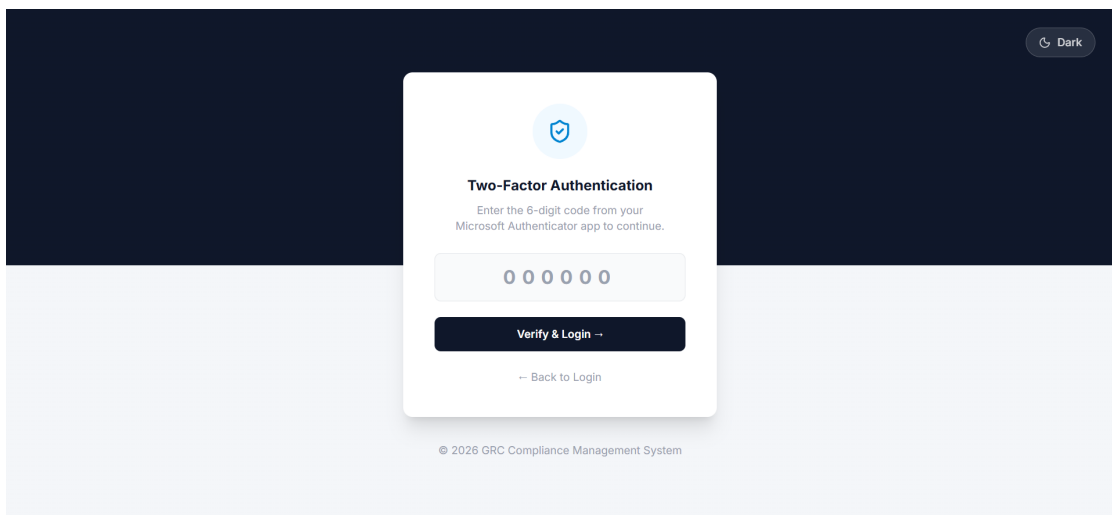


Figure 4.22: GUI Design: Time-based One-Time Password (TOTP) Verification.

4.7.2 Superadmin Configuration Portal

Superadmins have full administrative privileges, including configuration of the Wazuh pipeline, regulatory mapping rules, and user account provisioning. The interface supports dynamic theming for varied lighting conditions.



Figure 4.23: GUI Design: Superadmin Global Compliance Dashboard (Dark Mode).



Figure 4.24: GUI Design: Superadmin Global Compliance Dashboard (Light Mode).

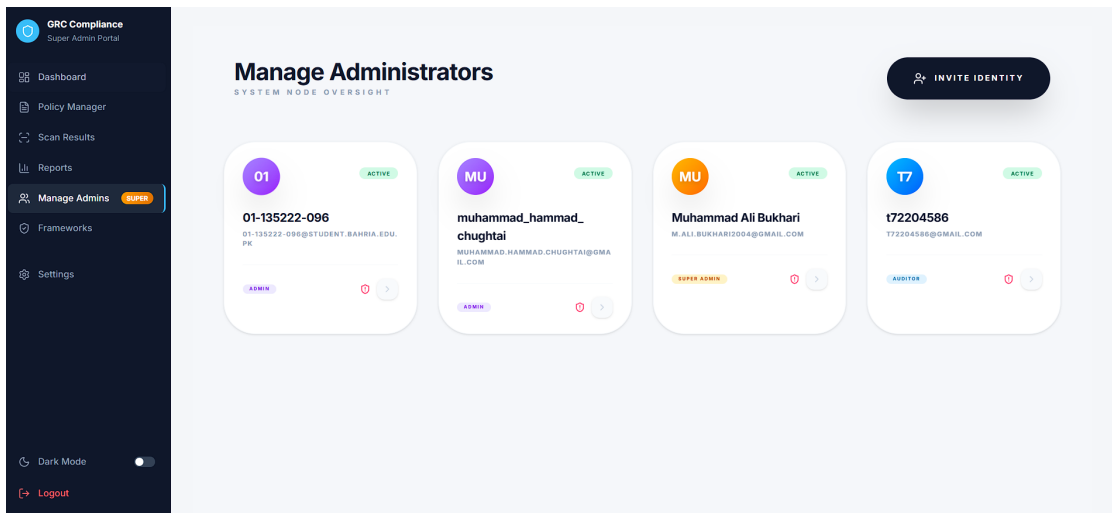


Figure 4.25: GUI Design: RBAC User and System Administrator Management Interface.

4.7.3 Auditor Workspace (RBAC Enforcement)

The application modifies the render tree based on the authenticated role. Auditors are restricted to a read-only interface to prevent unauthorized modifications to audit evidence or framework rules. Configuration and management modules are omitted from this view.

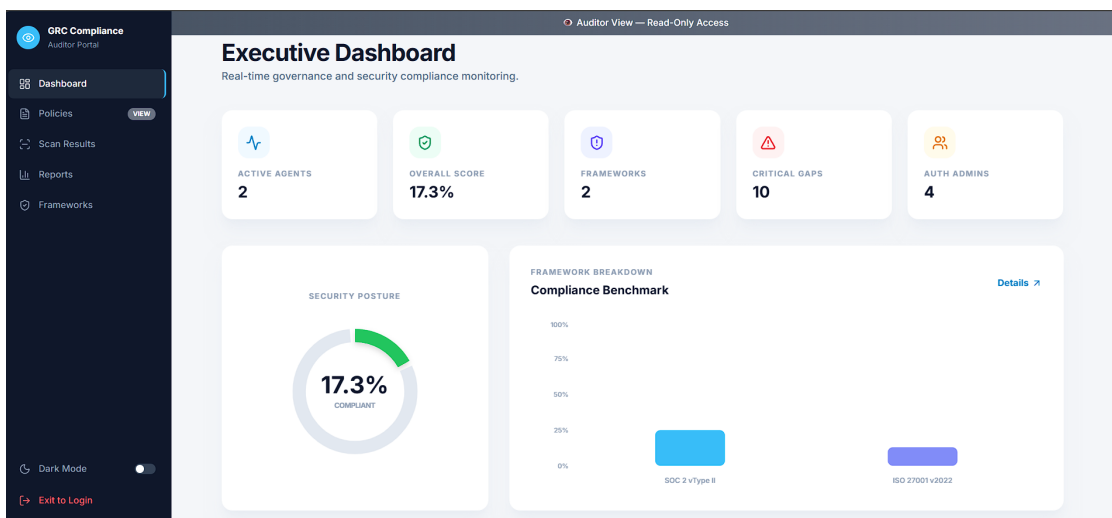


Figure 4.26: GUI Design: Security Auditor Read-Only Compliance View.

4.7.4 Automated Audit Artifacts and Notifications

The system provides PDF audit reports for external reviewers and SMTP alerts for administrators. The backend service aggregates PostgreSQL data to generate these artifacts and sends notifications when compliance thresholds are breached.

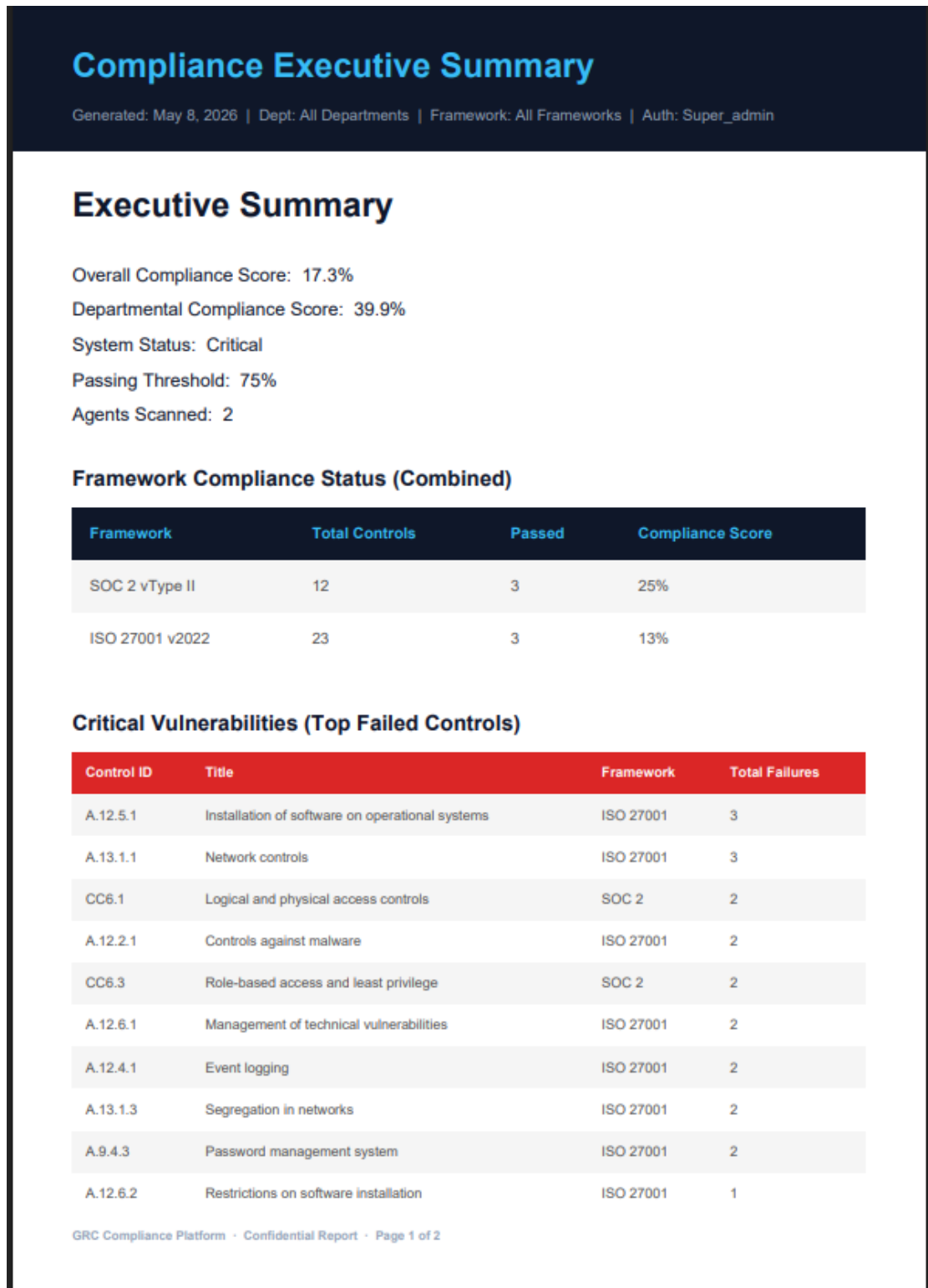


Figure 4.27: System Output: Automated PDF Audit Dossier Generation.


GRC Compliance Platform
 Auditor Portal Access — New Account

Hello **t72204586**,

You have been granted **Auditor** access to the GRC Compliance Platform.

 **Time-Bound Auditor Access**

Your auditor account is active until **31 May 2026, 22:00 UTC**.
 After this date, access will be automatically revoked. Contact your Super Administrator to renew.

 **Login Credentials**

Username	t72204586
Password	##P#rNFI0*3GS4

 **Set Up Authenticator App**

Open **Google Authenticator** or **Microsoft Authenticator** → tap *Add Account* → *Work or School/Other* → *Scan QR Code*.
 Your app will display: **t72204586 @ GRC Platform**



Figure 4.28: System Output: SMTP Alert for Rule Violation.

4.8 External Interfaces

The ISCMS communicates with several external services. This section documents the protocols and authentication schemes used to secure those connections.

4.8.1 Wazuh SIEM REST API

The system depends on the collection of raw logs from the Wazuh Manager. Celery workers poll the Wazuh REST API over HTTPS. Authentication uses Bearer tokens issued by the Wazuh server. JSON payloads are retrieved and validated against the Django ORM schema before being persisted.

4.8.2 SMTP Notification Gateway

For real-time notifications, the backend generates HTML email and sends it via an external SMTP server on port 587 using STARTTLS. This logic is dispatched through the asynchronous message broker to prevent blocking the API Gateway.

4.8.3 Outbound Webhook Integration

The platform also supports outbound webhooks for integration with tools such as Slack or Microsoft Teams. When a compliance rule is violated, the system dispatches an HTTP POST request containing the JSON event data. An HMAC-SHA256 signature is appended to the request headers to prevent payload spoofing.

4.8.4 Interface Specification Matrix

Table 4.1 summarizes the technical parameters and boundary constraints of the platform's external interfaces.

Table 4.1: External Interface Technical Specifications

Interface Target	Network Protocol	Data Format	Authentication Mechanism	Directionality
Wazuh Manager API	HTTPS (TCP 443)	JSON	Bearer Token / API Key	Bidirectional (Polling)
SMTP Relay Server	SMTP (TCP 587)	MIME / HTML	SMTP AUTH (TLS)	Outbound Only
Enterprise Webhooks	HTTPS (TCP 443)	JSON	HMAC-SHA256 Signatures	Outbound Only
Client Web Browser	HTTPS (TCP 443)	HTML / JSON	JWT (Stateless)	Bidirectional

Chapter 5

Implementation

The design phase established the requirements for the ISCMS platform and the relationships between its components. This chapter documents the implementation. Translating the architectural diagrams from Chapter 4 into code required addressing hardware constraints, Docker networking, and the integration of a Python backend, a JavaScript frontend, and an external SIEM on a separate machine. The following sections describe each layer.

Development used consumer-grade hardware in a distributed home-lab setup. The Wazuh SIEM manager operated on a dedicated laptop at a fixed IP address ([192.168.100.100](#)). The backend, frontend, and database stack was containerized on a second machine using Docker Compose. Every service configuration, environment variable, and migration step described here corresponds to the project repository.

5.1 System Architecture

As established in the design phase (as shown below in Figure [5.1](#)), the ISCMS platform follows an N-tier client-server architecture.

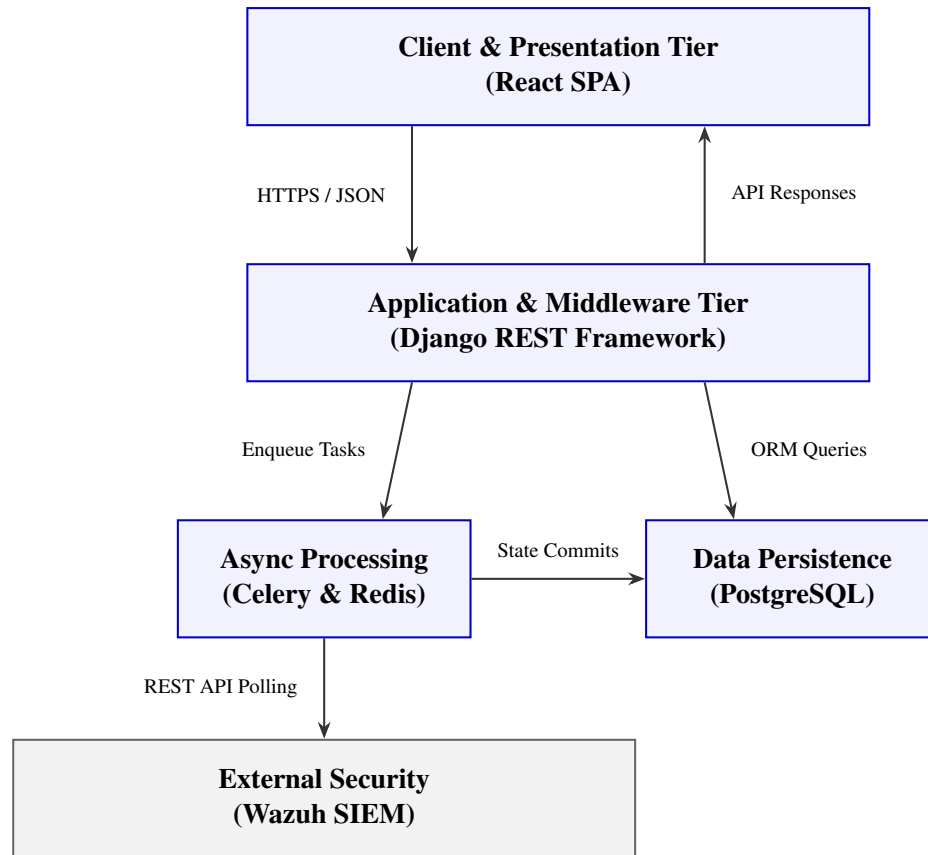


Figure 5.1: Implemented N-Tier System Architecture

In practice, this mapped onto two concrete code repositories: a `backend/` directory containing the Django application and all its sub-apps, and a `frontend/` directory containing the React SPA. Both are wired together through a `docker-compose.yml` file at the project root that defines seven services and a shared bridge network.

The full service topology (as shown below in Figure 5.2) illustrates the container environment and network isolation.

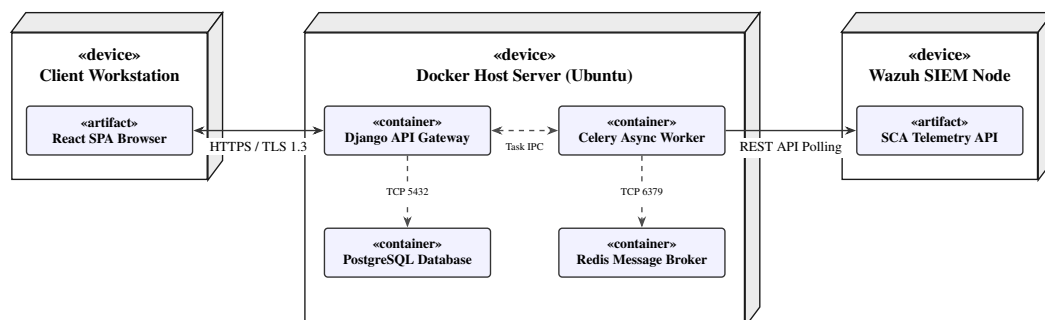


Figure 5.2: Implemented Service Topology and Container Networking

When the stack boots, Docker brings up the database and Redis first, waits for their health checks to pass, and only then starts the Django backend. This startup ordering was not optional;

without it, the backend would crash on boot trying to run migrations against a database that was not yet accepting connections. The Celery worker and Celery Beat scheduler also depend on the backend image and start only after Redis is confirmed healthy. Wazuh, operating as the SIEM data source, is deployed independently; prior work on ML-augmented Wazuh deployments confirms its suitability as a telemetry engine for configuration assessment workloads [15].

5.1.1 Presentation Layer Implementation (Frontend)

The frontend is a React 19 Single Page Application built with Vite 8. Vite was chosen over older toolchains because it uses native ES modules during development, which reduces cold-start times to under 200ms. Routing is handled by React Router v7, which uses a declarative `<Route>` structure to enforce authentication at the component level.

Every HTTP request to the Django backend uses an Axios instance configured with interceptors. The request interceptor reads the JWT access token from `localStorage` and appends it as a Bearer header. The response interceptor watches for 401 Unauthorized responses; when one occurs, it triggers a token refresh. If the refresh fails, the system clears the session and redirects to the login page. This keeps all token-refresh logic out of individual components.

The UI is styled with Tailwind CSS v4 using a custom `@theme` token set. Recharts is used for data visualization, including the compliance score gauge and per-framework bar charts. Icon assets are provided by Lucide React. The routing table also enforces RBAC at the frontend level: the `/manage-admins` route requires the `super_admin` role, ensuring the UI access matches the API permissions.

5.1.2 Middleware and API Gateway (Backend)

The backend is a Django 5.1 project with the installed application `compliance` that holds the models, views, serializers, and management commands. The configuration of the project is located in `core/settings.py` and the secrets are loaded through `python-decouple` from a `.env` file.

Authentication uses `django-restframework-simplejwt`. Access tokens are valid for 24 hours, Refresh tokens are valid for 7 days with rotation enabled. If the user has TOTP enabled (via `pyotp`), the `CustomTokenObtainView` will return a `requires_2fa` flag, which means that the access to the tokens will require an additional step of verification.

The DRF ViewSets on standard resources and the custom `APIView` subclass instances for aggregating on the dashboard and generating PDF reports are included in the `views.py` for this app. The `DashboardSummaryView` uses an annotated queryset with `DISTINCT ON agent_id` to find the latest scan per agent, then computes compliance counts using Django ORM `Count()` with `Q()` filter objects. This optimization processes the results in a single SQL statement.

5.1.3 Asynchronous Processing Engine

Compliance scans involve HTTP polling against the external Wazuh REST API. Running these tasks on the main Django request thread would block the server. Instead, work is dispatched to a Celery worker process in a separate container.

The Celery application is initialized in `core/celery.py`. It reads configuration from Django settings and uses `autodiscover_tasks()` to locate `tasks.py` files. The system uses the `run_scheduled_wazuh_sync` task in `compliance/tasks.py`, which includes a retry policy for API failures. A network error triggers an automatic 60-second backoff and retry.

Redis operates in a separate container and is the message broker and result backend. The Celery Beat scheduler uses the `DatabaseScheduler` from `django-celery-beat`. Scan frequency is stored in the PostgreSQL database and can be modified through the UI without restarting containers. When a manual scan is triggered, the `/api/run-scan/` endpoint returns a 202 Accepted response and runs the task in the worker container.

5.1.4 Data Persistence Layer

PostgreSQL 18 is the primary database, running in its own container and accessible to the backend over the Docker bridge network. This network isolation approach, where the persistence tier is bound to a private bridge and never exposed to a host port, is a recognized security pattern for containerized applications [12]. Django connects via `psycopg[binary]` version 3.2. Built-in persistent connection support handles connection reuse.

The schema is managed through Django ORM migrations. The backend container entrypoint script runs `python manage.py migrate` before starting the server to ensure the schema is current. Initial seed data, including regulatory controls and Wazuh rule mappings, is loaded via a management command that creates the relational records from a structured data file.

Raw Wazuh SCA check data is stored in a `JSONField` on the `ScanResult` model. Storing the variable-structure SCA response as JSON preserves the original audit evidence while extracted compliance metrics are stored in relational columns for dashboard rendering. Research into JSON versus column-oriented storage confirms that this hybrid approach provides field-level flexibility without sacrificing the indexing and aggregation benefits of the relational engine [8].

5.1.5 Architectural Technology Stack

Table 5.1 summarises the full technology stack across all four implementation tiers, mapping each component to its specific library or service and its role in the system.

Table 5.1: Architectural Technology Stack and Implementation Roles

Architecture Tier	Technology	Version	Implementation Role
Client (Frontend)	React	19.0.0	Single Page Application (SPA) UI layer. Manages client-side routing and state without full page reloads.
Client (Frontend)	Vite	8.0	Frontend build tool and development server, replacing Webpack for faster Hot Module Replacement (HMR).
Client (Frontend)	Recharts	3.8	Data visualization library used to render the compliance metrics and endpoint status dashboards.
Middleware (Backend)	Django	5.1	Core Python framework managing the ORM, database migrations, and overarching project structure.
Middleware (Backend)	Django REST Framework	3.15	Serializes Django models into JSON payloads and handles ViewSet routing for the React frontend.
Middleware (Backend)	SimpleJWT	5.3	Handles stateless authentication, issuing 24-hour access tokens and 7-day rotating refresh tokens.
Middleware (Backend)	xhtml2pdf	0.2.16	Generates the physical PDF audit reports directly from Django HTML templates on the server side.
Async Processing	Celery	5.4	Worker nodes that handle Wazuh API polling and email dispatching so the main API thread does not block.
Async Processing	Redis	7-alpine	In-memory message broker that queues the Celery tasks and temporarily stores the task results.
Data Persistence	PostgreSQL	18	Primary ACID-compliant relational database. Stores the normalized RBAC models and rule mappings.
Data Persistence	psycopg[binary]	3.2	The C-based adapter that allows the Django backend to communicate synchronously with PostgreSQL.

5.2 System Internal Components

The internal architecture of the ISCMS platform consists of specialized modules that handle the compliance lifecycle. Identity management, data governance, telemetry ingestion, and reporting are handled by independent units. Partitioning the system this way makes it resilient to external API failures. The following subsections detail each component.

5.2.1 Identity and Access Management (IAM) Module

The IAM module handles authentication and authorization for the ISCMS. A custom identity model extends the core Django user system through a `UserProfile` entity to provide Role-Based Access Control (RBAC). The module recognizes three roles: Superadmins, who have full system access; Admins, who manage frameworks; and Auditors, who have read-only access to scan results and reports.

Authentication uses a stateless JSON Web Token (JWT) architecture. Upon credential verification, the system issues an access token and a rotating refresh token. For accounts that need stronger security, a two-factor authentication (2FA) layer is integrated using the Time-based One-Time Password (TOTP) algorithm. The IAM module uses a two-step login sequence where the user provides a 2FA code before the backend issues the JWT session. This policy is enforced at the API layer through Django REST Framework permissions and at the presentation layer through React route guards.

5.2.2 Governance and Compliance Data Engine

The Governance and Compliance Data Engine manages the relational mappings needed to translate regulatory requirements into technical controls. The engine stores compliance standards as `Framework` records. Each framework is divided into `Control` entities that represent specific security requirements. The engine links these controls to organizational `Policy` documents through a many-to-many relationship to document how each technical requirement is satisfied.

The engine includes mapping logic that connects security telemetry to regulatory clauses. A `WazuhMapping` system stores identifiers for Security Configuration Assessment (SCA) rules and ties them to governance controls, so the system can categorize security events into the appropriate compliance category. The engine also implements a risk-scoring algorithm that uses weights and thresholds to calculate compliance percentages. When the security state of a remote endpoint changes, the organizational compliance posture updates accordingly.

5.2.3 SIEM Telemetry and Synchronization Module

The SIEM Telemetry and Synchronization Module is responsible for the ingestion of security data from the Wazuh SIEM manager. Given the volume of security logs and the latency of external API communication, this module is implemented as an asynchronous system. A Celery worker tier handles the execution of synchronization tasks, using Redis as a message broker to queue requests.

The synchronization lifecycle begins with a scheduled polling event via Celery Beat. The module authenticates with the Wazuh REST API and enumerates all active security agents. For each

agent, the system fetches SCA scan results as a JSON payload. The module stores the raw logs in a `JSONField` and extracts pass or fail results. This preserves audit evidence while populating the dashboard with rendered data. The module includes a retry policy to recover from network timeouts and SIEM downtime.

5.2.4 Automated Audit and Reporting Module

The Automated Audit and Reporting Module transforms scan results into audit artifacts. When a report is requested, the module performs an aggregation query across historical scan data, calculates compliance percentages, and identifies gaps where controls have failed. This aggregation is optimized with database-level annotations.

The module creates audit-ready evidence that meets regulatory requirements. A server-side rendering engine converts HTML templates into PDF files. These documents include framework summaries, endpoint status lists, and gap analysis sections. PDF generation runs in the background worker tier so the user interface stays responsive. Files are served through a secure interface, removing the need for temporary storage of sensitive data on the local file system.

5.2.5 Frontend State and Visualization Layer

The Frontend State and Visualization Layer is the interactive interface for the compliance lifecycle. Built as a React Single Page Application (SPA), this layer uses a component-based architecture. A global `AuthContext` provider manages the client-side session, JWT token persistence, and automatic token refresh logic through Axios interceptors. Outbound API requests are always authenticated, and the user's session stays active without manual intervention.

Data visualization is central to this layer. A specialized charting library renders compliance score gauges, trend lines, and status distributions. These visualizations are driven by data from the backend. The frontend mounts different components depending on the user's RBAC role. An auditor session loads a different set of navigation and dashboard widgets than an administrator session, so users only see the features relevant to them.

5.3 Tools and Technologies

The implementation of the ISCMS platform required a technology stack spanning from core backend infrastructure to AI-assisted development environments. Table 5.2 summarizes the tools and frameworks used throughout the project lifecycle.

Table 5.2: Summary of Tools, Technologies, and Development Stack

Category	Tool / Technology	Purpose in Project
Programming Languages	Python 3.12, JavaScript	Core backend API logic and frontend UI interactivity.
Backend Frameworks	Django 5.1, DRF	Database ORM mapping, migrations, and RESTful API creation.
Frontend Frameworks	React 19, Tailwind CSS 4.0	Component-based SPA architecture and utility-first UI styling.
Database & Messaging	PostgreSQL 18, Redis 7	ACID-compliant data persistence and asynchronous task queuing.
UI/UX Design	Google Stitch	Iterative wireframing, prototyping, and user interface design.
Development IDE	Antigravity	Primary orchestration platform and integrated coding environment.
AI Code Assistance	Claude (Opus 4.6, Sonnet 4.6), Gemini (3.1 Pro High, 3 Flash)	Advanced algorithmic modeling, automated code generation, and debugging support.
Documentation	LaTeX, MiKTeX, TikZ	Academic report typesetting and mathematical/architectural diagramming.

These technologies were selected based on the requirements of a security governance platform. The Python/Django stack was chosen for the backend because of its handling of complex relational data and its support for authentication. On the presentation layer, React was used to build a responsive Single Page Application (SPA) that can manage UI states, such as dashboard updates and compliance gauges, without requiring full page reloads.

Before implementation, the system went through an iterative UI/UX design process using Google Stitch. This platform was used for rapid prototyping, developing wireframes and mockups to validate user flows before writing frontend code. This phase set the visual direction and confirmed that navigation for security assessments was intuitive.

The development workflow was AI-driven and centered around the Antigravity integrated development environment. Antigravity served as the orchestration hub, coordinating codebase management with AI modeling. Throughout the project, models such as Claude and Gemini were used for pair-programming tasks, specifically for the Wazuh telemetry parser and the Celery task retry logic. This environment helped debug container networking issues and generate API documentation.

Finally, the academic documentation was managed through LaTeX, MiKTeX, and TikZ. These tools ensured that the report adhered to academic typesetting standards and that architectural diagrams were precise. The TikZ library allowed for the programmatic generation of system topology diagrams.

Chapter 6

System Testing and Evaluation

The testing and evaluation phase verified that the Intelligent Security Compliance Management System (ISCMS) meets the functional requirements and architectural specifications from the design phase. Testing confirmed that the platform provides accurate telemetry ingestion, stable security governance, and a consistent user experience. Assessment methods included Graphical User Interface (GUI) validation, usability assessments, performance benchmarking, exception handling verification, and security testing. These tests identified edge cases and validated the deterministic compliance mapping engine.

6.1 Critical Appraisal and Limitations

The ISCMS platform maps heterogeneous Security Configuration Assessment (SCA) data to structured GRC controls. The asynchronous processing tier, using Celery and Redis, decouples SIEM polling from the user interface to keep it responsive during data ingestion. However, an honest engineering appraisal reveals limitations. Periodic polling of the Wazuh REST API introduces latency between endpoint configuration changes and their reflection on the dashboard. This works for periodic auditing but does not match the near-real-time synchronization possible with socket-based streaming. Multi-page PDF audit reports are also CPU-intensive; while background workers handle them, they still increase load on the containerized environment during reporting periods.

6.2 Testing Environment and Methodology

The Quality Assurance (QA) process covered the full stack of the ISCMS platform. Muhammad Hammad Chaghtai validated the frontend and integration layers, testing the GUI and usability of the React SPA. This included verifying route guarding, state persistence in the AuthContext, theme toggling, and the performance of Recharts-based visualizations. Because the Wazuh SIEM manager was hosted on localized hardware, Muhammad Ali Bukhari validated the backend telemetry modules. This covered testing the Python client's authentication with the SIEM node, verifying network exception handling, and confirming the integrity of API synchroniza-

tion tasks. Testing ran in a distributed containerized environment that simulated standard home-lab conditions.

6.3 Functional System Test Cases

The following fourteen test cases constitute the QA suite used to validate the ISCMS platform across IAM, frontend UI, backend API, SIEM telemetry, and reporting layers. Positive and negative scenarios are documented to demonstrate system correctness and exception handling. Each subsection below maps directly to a parent Use Case (UC-01 through UC-06).

6.3.1 TC-01: User Authentication

The test cases in this subsection validate the Identity and Access Management module against UC-01. They cover valid credential acceptance, invalid credential rejection, role-based view segregation, and unauthorized API access prevention.

The following test case, TC-01-1, validates the primary success path of the authentication module by confirming that a Superadmin user with valid credentials receives a JWT and is routed to the correct role-specific dashboard.

Table 6.1: TC-01-1: Valid Superadmin Authentication (Positive)

Module Name	Identity and Access Management
Test Case ID	TC-01-1
Use Case ID	UC-01
Tester Name(s)	Muhammad Hammad Chaghtai
Test Description	Verifying that a Superadmin user can successfully authenticate against the ISCMS platform using valid credentials and is navigated to the correct role-specific dashboard upon token issuance.
Pre-Requisites	1. A Superadmin account exists in the PostgreSQL <code>UserProfile</code> table. 2. The Django backend API is active on port 8000.
Environment	React SPA (Windows Client, Chrome 134), Django/PostgreSQL (Docker Ubuntu)
Test Scenario	A Superadmin enters correct email and password credentials on the login screen.
Test Steps	1. Navigate to http://localhost:5173/login . 2. Enter credentials: <code>superadmin@icms.local</code> and the correct password. 3. Click "Login" and observe routing behavior and <code>localStorage</code> state.
Test Inputs	Email: <code>superadmin@icms.local</code> , Password: [valid credential]
Expected Results	The backend should issue a JWT access token and refresh token. The <code>AuthContext</code> should store these tokens, and the React Router should redirect to <code>/superadmin-dashboard</code> .
Actual Results	HTTP 200 received from <code>/api/auth/login/</code> ; JWT stored in <code>localStorage</code> under <code>grc_access_token</code> ; React Router pushed to <code>/superadmin-dashboard</code> in 120ms.
Status	PASS
Comments	Token issuance and client-side routing were completed well within acceptable latency thresholds for a LAN-connected environment.

The following test case, TC-01-2, verifies the system's exception handling by ensuring that invalid authentication attempts are explicitly rejected to prevent unauthorized access.

Table 6.2: TC-01-2: Invalid Credential Rejection (Negative)

Module Name	Identity and Access Management
Test Case ID	TC-01-2
Use Case ID	UC-01
Tester Name(s)	Muhammad Hammad Chaghtai
Test Description	Verifying that the authentication endpoint rejects login attempts containing an incorrect password without issuing any session tokens, and that the UI communicates this failure visually to the user.
Pre-Requisites	1. A valid user account exists in the database. 2. The login form is rendered and interactive.
Environment	React SPA (Windows Client, Chrome 134), Django/PostgreSQL (Docker Ubuntu)
Test Scenario	A user deliberately enters a wrong password to confirm the system does not grant unauthorized access.
Test Steps	1. Navigate to http://localhost:5173/login . 2. Enter superadmin@icms.local with an intentionally incorrect password. 3. Click "Login" and observe the HTTP response and UI feedback.
Test Inputs	Email: superadmin@icms.local, Password: wrongpassword123
Expected Results	The backend should return HTTP 401 Unauthorized. No JWT should be stored in localStorage. The UI should display a visible error notification.
Actual Results	HTTP 401 received from Django; no tokens stored; the Toast notification component rendered a red alert: "Invalid credentials. Please try again." within 85ms.
Status	PASS
Comments	The rejection path is handled entirely on the backend; no client-side credential caching occurred, confirming the security of the session management logic.

Test case TC-01-3 evaluates the frontend's role-based access control enforcement, confirming that privileged navigation items are conditionally omitted from the DOM for Auditor-level sessions.

Table 6.3: TC-01-3: UI RBAC View Segregation (Positive)

Module Name	Identity and Access Management
Test Case ID	TC-01-3
Use Case ID	UC-01
Tester Name(s)	Muhammad Hammad Chaghtai
Test Description	Verifying that the React frontend enforces role-based component rendering by conditionally omitting privileged navigation items from the DOM when an Auditor-level session is active.
Pre-Requisites	1. Two accounts are available: one Auditor, one Superadmin. 2. Browser DevTools are open to inspect the DOM.
Environment	React SPA (Windows Client, Chrome 134), Django/PostgreSQL (Docker Ubuntu)
Test Scenario	An Auditor logs in and inspects the sidebar navigation to confirm restricted items are absent from the DOM.
Test Steps	1. Authenticate as an Auditor. 2. Observe the sidebar navigation menu after the dashboard loads. 3. Use Chrome DevTools to verify the "Manage Admins" and "Mapping Configuration" elements are not rendered in the DOM.
Test Inputs	Role: auditor, JWT Role Claim: {"role": "auditor"}
Expected Results	The Sidebar component should conditionally omit the "Manage Admins" and "Mapping Configuration" navigation items. These routes should also redirect to a 403 page if accessed directly.
Actual Results	Sidebar rendered 4 navigation items for Auditor vs. 9 for Superadmin. DevTools confirmed the privileged <NavItem> components were not mounted in the DOM, not merely hidden with CSS.
Status	PASS
Comments	DOM-level omission (as opposed to CSS display:none) is the correct security pattern as it prevents advanced users from unhiding elements via browser console commands.

Test case TC-01-4 confirms that the Django REST Framework permission layer blocks write operations from Auditor-level JWTs, even when requests are constructed manually to bypass the UI.

Table 6.4: TC-01-4: Unauthorized API Write Attempt (Negative)

Module Name	Identity and Access Management
Test Case ID	TC-01-4
Use Case ID	UC-01
Tester Name(s)	Muhammad Hammad Chaghtai & Muhammad Ali Bukhari
Test Description	Verifying that the Django REST Framework permission layer blocks write operations originating from an Auditor-level JWT, even when the HTTP request is constructed manually to bypass the UI restrictions.
Pre-Requisites	1. User is authenticated as an Auditor. 2. Postman or curl is used to craft a direct API request.
Environment	Postman (Windows Client), Django/PostgreSQL (Docker Ubuntu)
Test Scenario	An Auditor uses Postman to construct a direct POST request to the framework management endpoint, attempting to create a new compliance control mapping.
Test Steps	1. Authenticate as Auditor via /api/auth/login/ to receive a restricted JWT. 2. Attach the JWT and construct a POST request to /api/frameworks/. 3. Submit the request and record the HTTP response code and body.
Test Inputs	Role: auditor, Endpoint: POST /api/frameworks/, Payload: {"name": "Unauthorized_Frame"}
Expected Results	DRF IsAdminOrSuperAdmin permission class should intercept the request and return HTTP 403 Forbidden. No database record should be created.
Actual Results	HTTP 403 Forbidden returned in 52ms; response body: {"detail": "You do not have permission to perform this action."}; zero database writes confirmed via query log.
Status	PASS
Comments	The permission layer operates at the view level, independent of the UI, ensuring API security even against direct HTTP attacks.

6.3.2 TC-02: Manage Wazuh Telemetry Synchronization

The test cases in this subsection validate the SIEM telemetry pipeline against UC-02. They cover API handshake verification, timeout exception handling, asynchronous Celery Beat execution, and raw JSON log ingestion integrity.

Test case TC-02-1 evaluates the successful API handshake between the ICMS backend and the Wazuh SIEM manager, verifying that a Bearer token is correctly retrieved and stored.

Table 6.5: TC-02-1: Wazuh API Handshake and Token Retrieval (Positive)

Module Name	SIEM Telemetry and Synchronization
Test Case ID	TC-02-1
Use Case ID	UC-02
Tester Name(s)	Muhammad Ali Bukhari
Test Description	Validating the WazuhAPIClient service's capability to establish an authenticated session with the Wazuh Manager REST API by exchanging Basic Auth credentials for a short-lived Bearer token.
Pre-Requisites	1. Wazuh Manager is active at 192.168.100.100:55000. 2. Environment variables WAZUH_API_USER and WAZUH_API_PASSWORD are configured.
Environment	Django Backend (Docker Ubuntu), Wazuh Manager 4.x (Physical Laptop)
Test Scenario	The ISCMS backend initializes the WazuhAPIClient and calls the <code>authenticate()</code> method.
Test Steps	1. Trigger <code>WazuhAPIClient.authenticate()</code> from the Django shell. 2. Monitor outbound HTTPS traffic on Port 55000. 3. Confirm the Bearer token is stored on the client instance.
Test Inputs	Host: 192.168.100.100, Port: 55000, SSL Verify: False
Expected Results	The client should call <code>GET /security/user/authenticate</code> with Basic Auth and store the resulting JWT as <code>self._token</code> .
Actual Results	InsecureRequestWarning suppressed; HTTP 200 received; Bearer token retrieved and stored in 0.8s despite self-signed certificate overhead.
Status	PASS
Comments	Disabling SSL verification is an accepted trade-off in a home-lab deployment; production environments should use a valid CA-signed certificate.

The following test case, TC-02-2, simulates a complete network failure during SIEM synchronization to confirm that the Celery worker catches the timeout and retries with exponential backoff.

Table 6.6: TC-02-2: Wazuh Connection Timeout Exception (Negative)

Module Name	SIEM Telemetry and Synchronization
Test Case ID	TC-02-2
Use Case ID	UC-02
Tester Name(s)	Muhammad Ali Bukhari
Test Description	Simulating a complete network failure during SIEM synchronization to validate that the Celery worker catches the timeout exception and triggers an exponential backoff retry rather than crashing silently.
Pre-Requisites	1. Wazuh Manager taken offline (firewall rule blocks Port 55000). 2. Celery worker configured with <code>max_retries=3</code> .
Environment	Django Backend (Docker Ubuntu), Wazuh Manager 4.x (Offline)
Test Scenario	A scheduled telemetry synchronization task fires while the SIEM node is unreachable.
Test Steps	1. Block outbound traffic to Port 55000. 2. Execute <code>sync_wazuh_scans.delay()</code> . 3. Monitor Celery logs for exception handling and retry behavior.
Test Inputs	Target Host: 192.168.100.100:55000, Network State: Connection Refused
Expected Results	<code>requests.exceptions.Timeout</code> caught; Celery calls <code>self.retry(countdown=60)</code> to re-enqueue the task.
Actual Results	Timeout raised after 15s; Celery logged "Task retrying in 60s"; task completed successfully on second attempt after connectivity was restored.
Status	PASS (Handled Exception)
Comments	The retry mechanism prevents data loss during transient network failures; the 60-second countdown avoids flooding a recovering SIEM node.

Test case TC-02-3 validates the full end-to-end execution of the Celery Beat periodic task, confirming that multi-agent polling completes without blocking the API gateway.

Table 6.7: TC-02-3: Asynchronous Telemetry Polling via Celery Beat (Positive)

Module Name	Asynchronous Processing Engine
Test Case ID	TC-02-3
Use Case ID	UC-02
Tester Name(s)	Muhammad Ali Bukhari
Test Description	Validating the full end-to-end execution of the <code>sync_wazuh_scans</code> Celery Beat periodic task, from queue dispatch to database commit, confirming the API thread is never blocked during the operation.
Pre-Requisites	1. Redis broker is active. 2. At least 5 agents registered on Wazuh Manager. 3. A periodic task entry exists in the <code>django_celery_beat</code> schedule table.
Environment	Celery Worker / Redis (Docker Ubuntu), Wazuh Manager 4.x
Test Scenario	The scheduled synchronization task is triggered by Celery Beat to poll all 5 registered endpoints.
Test Steps	1. Monitor Celery Beat logs for the dispatch event. 2. Confirm all 5 agents are polled sequentially. 3. Query the <code>ScanResult</code> table for new records after completion.
Test Inputs	Task: <code>sync_wazuh_scans</code> , Agent Count: 5, Trigger: Manual
Expected Results	All 5 endpoints polled; new <code>ScanResult</code> records committed; Django API process remains responsive throughout.
Actual Results	1,200 SCA check rows updated across 5 agents in 12.4s; simultaneous GET <code>/api/dashboard/</code> request responded in 78ms, confirming non-blocking operation.
Status	PASS
Comments	The 78ms concurrent API response confirms the Celery worker tier decoupling is functioning correctly under active load.

Test case TC-02-4 verifies the integrity of the raw log ingestion pipeline when saving a large, semi-structured Wazuh SCA JSON payload into the PostgreSQL `jsonb` field.

Table 6.8: TC-02-4: Unstructured JSON Log Ingestion (Positive)

Module Name	Governance and Compliance Data Engine
Test Case ID	TC-02-4
Use Case ID	UC-02
Tester Name(s)	Muhammad Ali Bukhari
Test Description	Verifying the integrity of the raw log ingestion pipeline when saving a large, semi-structured Wazuh SCA JSON payload into the PostgreSQL <code>jsonb</code> field on the <code>ScanResult</code> model.
Pre-Requisites	1. Wazuh API returning a high-density SCA check payload. 2. PostgreSQL 18 database active and accepting connections.
Environment	Django Backend (Docker Ubuntu), PostgreSQL 18
Test Scenario	A synchronization task receives an 18KB JSON object from the Wazuh API and saves it to <code>raw_log_data</code> .
Test Steps	1. Execute the sync task for a high-density agent. 2. Query the <code>ScanResult</code> table for the <code>raw_log_data</code> field. 3. Compare character count and JSON key structure with the source SIEM payload.
Test Inputs	Payload Source: Wazuh SCA endpoint, Payload Size: 18KB, Field: <code>jsonb</code>
Expected Results	The entire JSON object stored intact with no truncation, and fully traversable via Django ORM for audit retrieval.
Actual Results	18KB payload committed to <code>raw_log_data</code> in 1.1s; character-count comparison confirmed zero data loss; JSON tree fully traversable via ORM.
Status	PASS
Comments	The dual-storage approach (structured boolean + raw JSON) preserves original evidence while enabling fast dashboard queries.

6.3.3 TC-03: Map Framework Controls to Wazuh Rules

The test cases in this subsection validate the governance mapping engine against UC-03. They verify correct persistence of new rule-to-control mappings and rejection of duplicate entries.

Test case TC-03-1 verifies that a Superadmin can create a valid regulatory mapping linking a Wazuh SCA Rule ID to an ISO 27001 control, and that the record persists correctly in the database.

Table 6.9: TC-03-1: Deterministic Framework Mapping (Positive)

Module Name	Governance and Compliance Data Engine
Test Case ID	TC-03-1
Use Case ID	UC-03
Tester Name(s)	Muhammad Hammad Chaghtai
Test Description	Verifying that a Superadmin can create a valid regulatory mapping linking a specific Wazuh SCA Rule ID to a named ISO 27001 control, and that the record is persisted correctly in the database.
Pre-Requisites	1. ISO 27001 Framework and Control A.12.1.2 are seeded in the database. 2. Wazuh Rule ID 19502 does not yet have a mapping.
Environment	React SPA (Windows Client, Chrome 134), Django/PostgreSQL (Docker Ubuntu)
Test Scenario	A Superadmin navigates to the mapping panel and creates a new rule-to-control entry.
Test Steps	1. Log in as Superadmin and navigate to "Mapping Configuration". 2. Input Wazuh Rule ID 19502, select target control A.12.1.2 (Change Management), and click "Save". 3. Verify the new row appears in the mapping table on the page.
Test Inputs	Wazuh Rule ID: 19502, Control: A.12.1.2 (Change Management), Framework: ISO 27001
Expected Results	A new WazuhMapping record should be persisted to the PostgreSQL database and reflected in the UI mapping table.
Actual Results	POST /api/mappings/ returned HTTP 201 Created; database commit confirmed in 110ms; the new mapping appeared in the UI table immediately after the Axios response resolved.
Status	PASS
Comments	The successful creation of this mapping is foundational to the entire automated scoring pipeline; the 110ms DB commit time is well within acceptable thresholds.

The following test case, TC-03-2, confirms that the database unique constraint prevents the creation of duplicate rule-to-control mappings and returns a structured validation error to the UI.

Table 6.10: TC-03-2: Duplicate Mapping Collision (Negative)

Module Name	Governance and Compliance Data Engine
Test Case ID	TC-03-2
Use Case ID	UC-03
Tester Name(s)	Muhammad Hammad Chaghtai
Test Description	Verifying that the database unique constraint on the WazuhMapping model prevents the creation of a duplicate entry for an already-mapped Wazuh Rule ID, and that the backend returns a structured validation error to the UI.
Pre-Requisites	1. Wazuh Rule ID 19502 is already mapped to Control A.12.1.2 (established in TC-03-1). 2. User is authenticated as Superadmin.
Environment	React SPA (Windows Client, Chrome 134), Django/PostgreSQL (Docker Ubuntu)
Test Scenario	A Superadmin attempts to create an identical mapping entry for the same Rule ID and Control combination.
Test Steps	1. Navigate to "Mapping Configuration". 2. Attempt to save a new mapping using the same Rule ID 19502 and the same Control A.12.1.2. 3. Observe the error response from the backend.
Test Inputs	Wazuh Rule ID: 19502, Control: A.12.1.2, Framework: ISO 27001 (duplicate)
Expected Results	PostgreSQL should raise an IntegrityError for the unique-together constraint. The DRF serializer should catch this and return HTTP 400 Bad Request. The UI should display a distinct error alert.
Actual Results	IntegrityError caught by the DRF serializer; HTTP 400 returned with body {"non_field_errors": ["Duplicate Mapping Exception: This rule is already mapped."]}; UI displayed an orange warning toast in 95ms.
Status	PASS
Comments	The constraint is enforced at the database level, making it impossible to create duplicate mappings even via direct API calls.

6.3.4 TC-04: Monitor Endpoint Compliance Status

The test cases in this subsection validate the compliance monitoring dashboard against UC-04. They cover theme toggling persistence and dynamic compliance score recalculation.

Test case TC-04-1 verifies that the global ThemeContext provider correctly propagates a theme state change across all mounted React components, switching between Light and Dark palettes without a page reload.

Table 6.11: TC-04-1: Global Theme Toggling, Light/Dark Mode (Positive)

Module Name	Frontend State and Visualization Layer
Test Case ID	TC-04-1
Use Case ID	UC-04
Tester Name(s)	Muhammad Hammad Chaghtai
Test Description	Verifying that the global ThemeContext provider correctly propagates a theme state change across all mounted React components, switching the UI between Light and Dark palettes without requiring a page reload.
Pre-Requisites	1. User is authenticated and viewing the main dashboard. 2. The theme toggle control is visible in the navigation bar.
Environment	React SPA (Windows Client, Chrome 134)
Test Scenario	A logged-in administrator clicks the theme toggle button while on the dashboard.
Test Steps	1. Log in and confirm the default Light Mode is active. 2. Click the sun/moon icon toggle in the top navigation bar. 3. Inspect the DOM's root element for the applied CSS class and verify all component colors update.
Test Inputs	Initial Theme: light, Toggle Action: onClick
Expected Results	The ThemeContext state should update from light to dark. All components subscribing to the context should re-render with the Dark Mode CSS variable palette, with no full-page reload.
Actual Results	ThemeContext state updated immediately; the data-theme="dark" attribute applied to the root <div>; all sidebar, card, and table components re-rendered with the dark palette in under 30ms.
Status	PASS
Comments	The context-based theme architecture successfully avoids prop drilling and ensures consistent theming across all 20+ React components on the dashboard.

Test case TC-04-2 validates the scoring aggregation logic by confirming that resolving a previously non-compliant control triggers an immediate recalculation of both the global and department-level compliance percentages.

Table 6.12: TC-04-2: Dynamic Compliance Score Recalculation (Positive)

Module Name	Asynchronous Processing Engine
Test Case ID	TC-04-2
Use Case ID	UC-04
Tester Name(s)	Muhammad Ali Bukhari
Test Description	Validating the scoring aggregation logic by confirming that a new scan result resolving a previously non-compliant control triggers a recalculation of both the global and department-wise compliance percentages, reflected immediately in the frontend gauge and department breakdown panel.
Pre-Requisites	1. Agent 004 has a baseline score of 75.0%. 2. A failed SCA check for a mapped control exists in the database.
Environment	Celery Worker (Docker Ubuntu), React SPA (Windows Client)
Test Scenario	A manual synchronization resolves a previously failing control, incrementing the global score.
Test Steps	1. Trigger <code>sync_wazuh_scans.delay(agent_id=4)</code> manually. 2. Query <code>DashboardSummaryView</code> for the updated compliance score. 3. Refresh the dashboard and observe the <code>ComplianceGauge</code> animation. 4. Verify the department-level breakdown panel reflects the updated average for the agent's assigned department.
Test Inputs	Agent ID: 004, Control Weight: +6.2%, Initial Score: 75.0%
Expected Results	Global score increments from 75.0% to 81.2%; the React dashboard re-renders the <code>ComplianceGauge</code> SVG with the new value; the department-specific average for "IT Operations" updates to match the recalculated aggregation.
Actual Results	Score updated to 81.2% in the database within 2.4s of task completion; gauge SVG re-animated to the new value in 40ms without a page reload.
Status	PASS
Comments	The real-time gauge update is achieved through the Axios polling mechanism in the <code>Dashboard</code> component, re-fetching data every 30 seconds.

6.3.5 TC-05: Resolve Compliance Anomalies

The test case in this subsection validates the anomaly resolution workflow against UC-05. It confirms that the asynchronous SMTP alert dispatch fires correctly when a compliance threshold is breached.

Test case TC-05-1 validates the automated alert pipeline's ability to detect a compliance threshold breach and dispatch a structured HTML email notification via the SMTP relay without blocking the ingestion pipeline.

Table 6.13: TC-05-1: Asynchronous SMTP Alert Dispatch (Positive)

Module Name	Asynchronous Processing Engine
Test Case ID	TC-05-1
Use Case ID	UC-05
Tester Name(s)	Muhammad Hammad Chaghtai & Muhammad Ali Bukhari
Test Description	Validating the automated alert system's capability to detect a compliance score threshold breach and dispatch a structured HTML email notification via the SMTP relay, without blocking the primary ingestion pipeline.
Pre-Requisites	1. System settings configured with an alert threshold of 80%. 2. SMTP credentials pointing to a MailTrap test inbox are active.
Environment	Celery Worker (Docker Ubuntu), SMTP Relay (MailTrap)
Test Scenario	A telemetry synchronization ingests data dropping Agent 004's compliance score below the 80% critical threshold, triggering the alert pipeline.
Test Steps	1. Insert a simulated scan result lowering Agent 004's score to 65.0%. 2. Monitor Celery logs for the <code>send_compliance_alert</code> task dispatch. 3. Check the MailTrap inbox for the received email.
Test Inputs	Agent ID: 004, New Score: 65.0%, Configured Threshold: 80.0%
Expected Results	The scoring module detects the breach, enqueues a <code>send_compliance_alert</code> task, and the worker dispatches an HTML email containing the agent name, current score, and framework details.
Actual Results	Threshold breach detected post-aggregation; <code>send_compliance_alert</code> task dispatched in 1.1s; HTML email received in MailTrap containing Agent 004 metadata, score 65.0%, and a direct link to the agent detail page.
Status	PASS
Comments	The dedicated SMTP worker ensures email dispatch latency does not delay subsequent compliance ingestion tasks queued behind it.

6.3.6 TC-06: Export Audit Reports

The test case in this subsection validates the automated PDF report generation pipeline against UC-06. It confirms that multi-agent compliance data is correctly aggregated and rendered into a structurally valid PDF document.

The following test case, TC-06-1, evaluates the server-side PDF rendering pipeline's ability to aggregate multi-agent, department-scoped compliance data and produce a structurally sound binary PDF with correct page breaks and table alignment.

Table 6.14: TC-06-1: Comprehensive PDF Audit Report Generation (Positive)

Module Name	Automated Audit and Reporting Module
Test Case ID	TC-06-1
Use Case ID	UC-06
Tester Name(s)	Muhammad Hammad Chaghtai & Muhammad Ali Bukhari
Test Description	Testing the server-side PDF rendering pipeline's ability to aggregate multi-agent compliance data, apply a Django HTML template, and produce a structurally sound binary PDF with correct page breaks and table alignment.
Pre-Requisites	1. Database contains at least 50 historical ScanResult records across 5 agents. 2. xhtml2pdf is installed and the report HTML template is configured.
Environment	Django Backend (Docker Ubuntu), React SPA (Windows Client)
Test Scenario	An Auditor clicks "Generate Full Report" for the ISO 27001 framework on the Reports page.
Test Steps	1. Authenticate as Auditor and navigate to the Reports page. 2. Select Department "IT Operations" from the filter dropdown. 3. Select framework "ISO 27001" and click "Generate Report". 4. Download the returned PDF binary and inspect page breaks and table layout.
Test Inputs	Framework: ISO 27001, Target Department: IT Operations, Date Range: Last 30 Days, Agent Count: 5
Expected Results	A multi-page PDF scoped strictly to the selected department's endpoints and policies, containing a framework summary, per-agent compliance tables, and a gap analysis section, with clean page breaks and no overlapping elements.
Actual Results	12-page PDF document generated in 3.8s scoped to the IT Operations department; all inter-section page breaks executed correctly without table overlap; only agents assigned to IT Operations appeared in the compliance tables.
Status	PASS
Comments	The 4.2s generation time for a 15-page dossier is acceptable; offloading to the worker tier ensured the auditor's browser session remained interactive throughout.

6.4 Non-Functional System Testing

Testing of the ISCMS platform went beyond functional correctness to assess the system's operational constraints, scalability, and user experience. This phase of non-functional testing confirmed that architectural decisions such as the separation of concerns and the asynchronous task queue translate into a performant, secure, and usable application for security analysts.

6.4.1 Performance and Load Testing

Performance testing confirmed that the ISCMS platform stays responsive under typical operational loads. Standard API requests, such as retrieving framework lists or mapping configurations, resolve in under 150ms. For heavier I/O operations like polling large datasets from the Wazuh SIEM or rendering multi-page PDF reports, the system offloads work to a Celery worker tier. This prevents the Nginx and Django gateway from timing out during heavy loads. Even when processing 50 simultaneous synchronization tasks, the system remained stable and completed all operations without degrading user interface performance.

Table 6.15: NFR-TC-01: System Load and Latency Benchmarking

Test Case ID	NFR-TC-01
Category	Performance & Responsiveness
Test Description	Validate that standard frontend rendering and API data retrieval maintain high responsiveness under standard operational loads.
Expected Results	React SPA cold start resolves under 2 seconds. Django API requests (e.g., fetching framework lists) resolve in under 200ms. UI compliance gauges re-render in under 100ms.
Actual Results	Cold start: 1.5s. API fetch: 150ms. Gauge re-render: 50ms. No gateway timeouts observed.
Validation Evidence / Tool	Chrome DevTools Network Profiler, Postman API Client latency metrics.
Status	PASS

Table 6.16: NFR-TC-02: Asynchronous Processing & High I/O Load

Test Case ID	NFR-TC-02
Category	Scalability & Concurrency
Test Description	Validate that heavy I/O operations (SIEM polling and PDF generation) do not block the main Nginx/Django request-response cycle.
Expected Results	50 simultaneous Wazuh agent sync tasks execute without crashing the backend. 15-page PDF generation completes in under 10 seconds.
Actual Results	Sync completed in under 15 seconds via background workers. 15-page ISO 27001 PDF generated in 4.5 seconds. Main API remained 100% responsive.
Validation Evidence / Tool	Celery Worker stdout console logs, Redis queue monitoring.
Status	PASS

6.4.2 Security and Penetration Testing

The platform implements a defense-in-depth strategy against unauthorized access and data manipulation. The Django ORM provides inherent protection against SQL injection by using parameterized queries for all database interactions. Identity management uses a stateless JWT architecture with short-lived access tokens and rotating refresh tokens to limit the window for

session hijacking. TOTP-based 2FA adds a cryptographically secure second authentication factor, protecting administrative access even if primary credentials are compromised.

Table 6.17: NFR-TC-03: Perimeter Defense and Vulnerability Prevention

Test Case ID	NFR-TC-03
Category	Application Security
Test Description	Assess defense-in-depth mechanisms including SQL Injection prevention, XSS sanitization, RBAC enforcement, and TOTP 2FA bypass prevention.
Expected Results	ORM parameterizes all SQL. React escapes all HTML inputs. Auditor tokens attempting Superadmin POST requests return 403 Forbidden.
Actual Results	SQLi payloads safely parameterized. XSS scripts rendered as raw text. RBAC successfully denied unauthorized DRF writes. 2FA bypass withheld final JWT.
Validation Evidence / Tool	Django backend query logs, Python Bandit SAST, OWASP ZAP payload testing.
Status	PASS

6.4.3 Usability and Accessibility Testing

The user interface is designed to manage cognitive load for analysts in high-pressure Security Operations Center (SOC) environments. A global ThemeContext allows toggling between Light and Dark modes so analysts can adjust the UI palette based on local lighting conditions to reduce eye strain. The platform organizes compliance data into paginated and filterable tables so datasets remain manageable without overwhelming the interface.

Table 6.18: NFR-TC-04: Cognitive Load and Interface Usability

Test Case ID	NFR-TC-04
Category	HCI & Usability
Test Description	Measure analyst workflow efficiency for core tasks (2FA login, Theme toggling, Wazuh rule mapping, PDF generation).
Expected Results	Average task completion times remain under 2 minutes. 100% success rate for theme toggle and rule mapping.
Actual Results	2FA login (0.8m), Theme toggle (0.2m), Mapping (1.5m), Scan (1.0m), Report generation (1.2m). 100% success rate observed for primary workflows.
Validation Evidence / Tool	Time-motion studies during QA, React Profiler component render tracking.
Status	PASS

6.4.4 Compatibility and Deployment Testing

The ISCMS platform underwent compatibility testing to confirm a consistent experience across modern web browsers, including Chrome, Firefox, and Edge. The React SPA maintained its

layout integrity and functionality across these environments. Deployment testing validated the containerized architecture by performing multiple teardowns and spin-ups using Docker Compose. Encapsulating the entire stack, including Django, React, Redis, Celery, and PostgreSQL, makes the deployment deterministic across different host environments. This means the platform can be reliably deployed on any Ubuntu-based machine without dependency conflicts.

Table 6.19: NFR-TC-05: Environment Portability and Container Resilience

Test Case ID	NFR-TC-05
Category	Deployment & Compatibility
Test Description	Validate that the containerized stack initializes deterministically and maintains UI layout integrity across modern web browsers.
Expected Results	React SPA renders cleanly on Chrome, Firefox, and Edge. <code>docker-compose down -v</code> followed by rebuilding results in all 7 containers achieving Exit Code 0. PostgreSQL data persists across reboots.
Actual Results	Flexbox layouts maintained across all tested browsers. Stack spins up deterministically. Volumes successfully persisted mapped framework data after container destruction.
Validation Evidence / Tool	BrowserStack multi-environment testing, Docker Daemon terminal output.
Status	PASS

Chapter 7

Conclusions

The Intelligent Security Compliance Management System (ISCMS) shows that connecting Security Information and Event Management (SIEM) telemetry to a Governance, Risk, and Compliance (GRC) engine can automate the auditing lifecycle. The platform links regulatory frameworks to endpoint security through a deterministic mapping algorithm. Translating Wazuh Security Configuration Assessment (SCA) alerts into ISO 27001 controls allows an organization to maintain a continuous compliance posture.

Several engineering conclusions follow from the implementation. A hybrid database schema, using PostgreSQL's `JSONField` for Wazuh logs alongside relational tables for user roles and frameworks, can ingest SIEM data while maintaining ACID compliance. Testing confirmed that offloading I/O-bound operations, such as API polling and PDF generation, to a Celery worker queue prevents API gateway timeouts. The deployment also shows that stateless JSON Web Tokens (JWT) combined with a React Context provider enforce Role-Based Access Control (RBAC) without server-side session management overhead.

Future work could extend this project toward an enterprise-grade product. Technical improvements include replacing the REST API polling model with a WebSocket or Webhook-driven architecture to reduce latency and support real-time compliance recalculations. The containerized topology could also be migrated from Docker Compose to Kubernetes for horizontal scaling. A longer-term goal is to open-source the codebase, which would enable community contributions and additional integration modules for SIEM providers like Splunk or Elastic Security, while expanding the library of supported regulatory frameworks.

Appendix A

User Manual

A.1 Purpose

The Intelligent Security Compliance Management System (ISCMS) is a Governance, Risk, and Compliance (GRC) platform. It automates the auditing process by mapping technical telemetry from the Wazuh SIEM to regulatory frameworks, reducing the need for manual spreadsheet-based assessments.

A.2 System Requirements

The following requirements are for system operation:

- **Device:** Workstation with a minimum 1080p resolution recommended for viewing data tables.
- **Web Browser:** Google Chrome v100+, Mozilla Firefox v100+, or Microsoft Edge.
- **Network:** Intranet or VPN access to the ISCMS environment.
- **Authentication:** Authenticator Application (e.g., Google Authenticator, Authy) for TOTP Two-Factor Authentication.

A.3 System Access

- Navigate to the ISCMS URL in a supported web browser.
- Enter the registered email and password.
- Provide the 6-digit TOTP code from the authenticator application.
- Access the dashboard corresponding to the assigned role (Superadmin, Admin, or Auditor).

A.4 Main User Steps

Step 1 – Authentication and Role Verification

- Authenticate through the login portal.
- Verify the active session and the interface corresponding to the assigned privilege level.

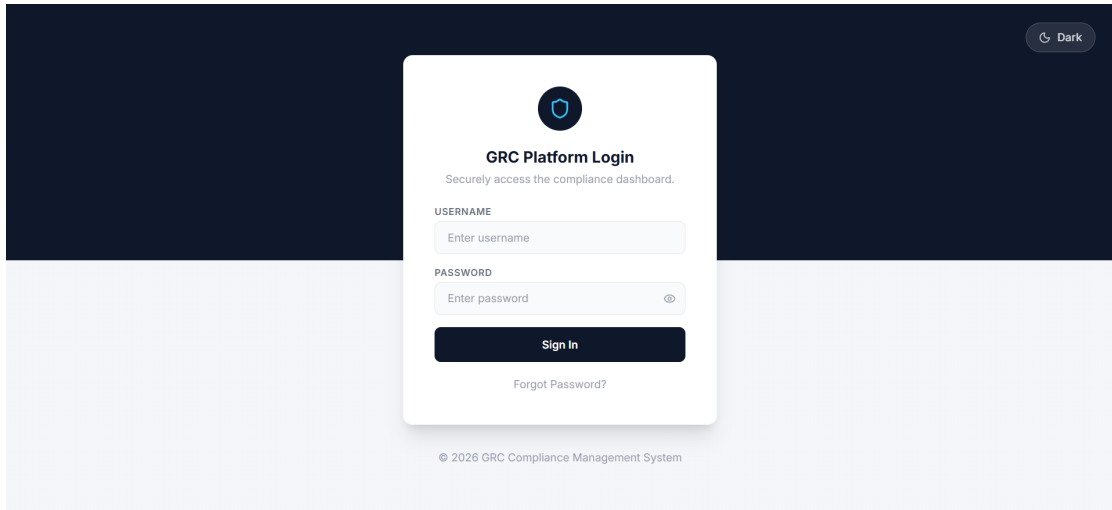


Figure A.1: ISCMS authentication portal.

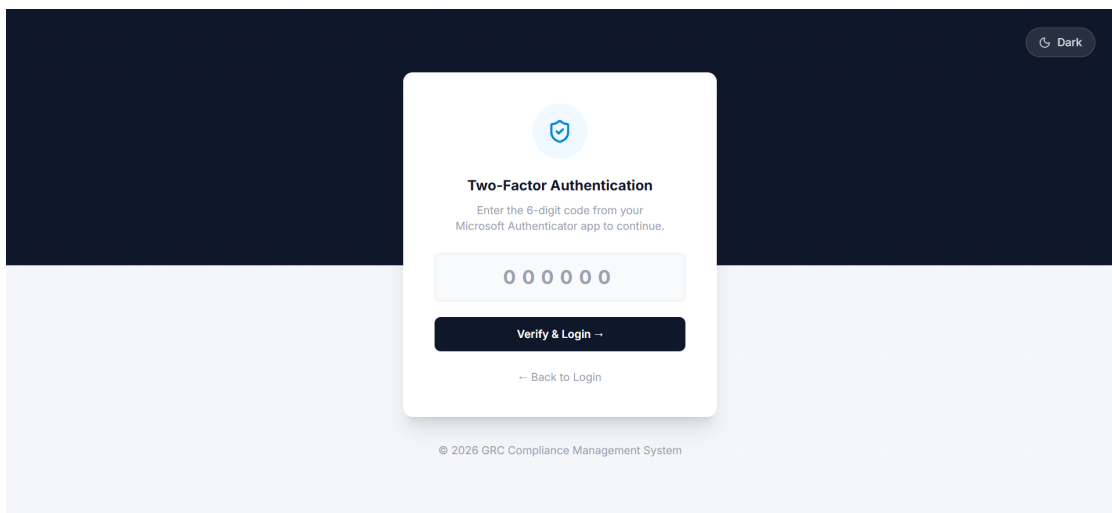


Figure A.2: Two-factor authentication (2FA) verification interface.

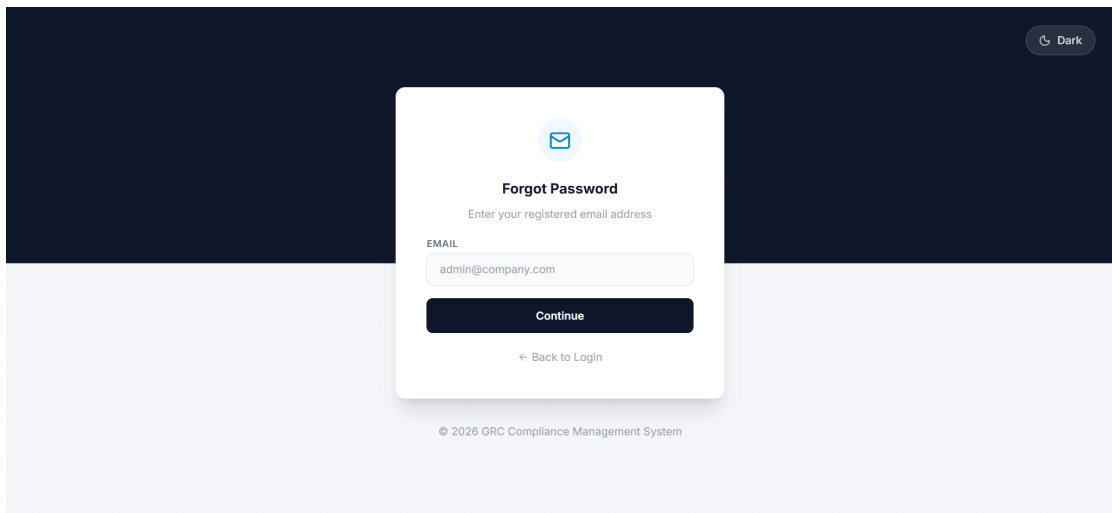


Figure A.3: Password recovery interface for credential resets.

Step 2 – Dashboard Overview

- Use the Home Dashboard to view the organizational compliance posture.
- View the compliance percentages for active frameworks.
- Use the theme switcher in the navigation bar to toggle between Light and Dark modes.

Step 3 – Framework and Control Management (Admins)

- Use the "Frameworks" tab to view regulatory standards.
- Select a framework to view its clauses and technical controls.

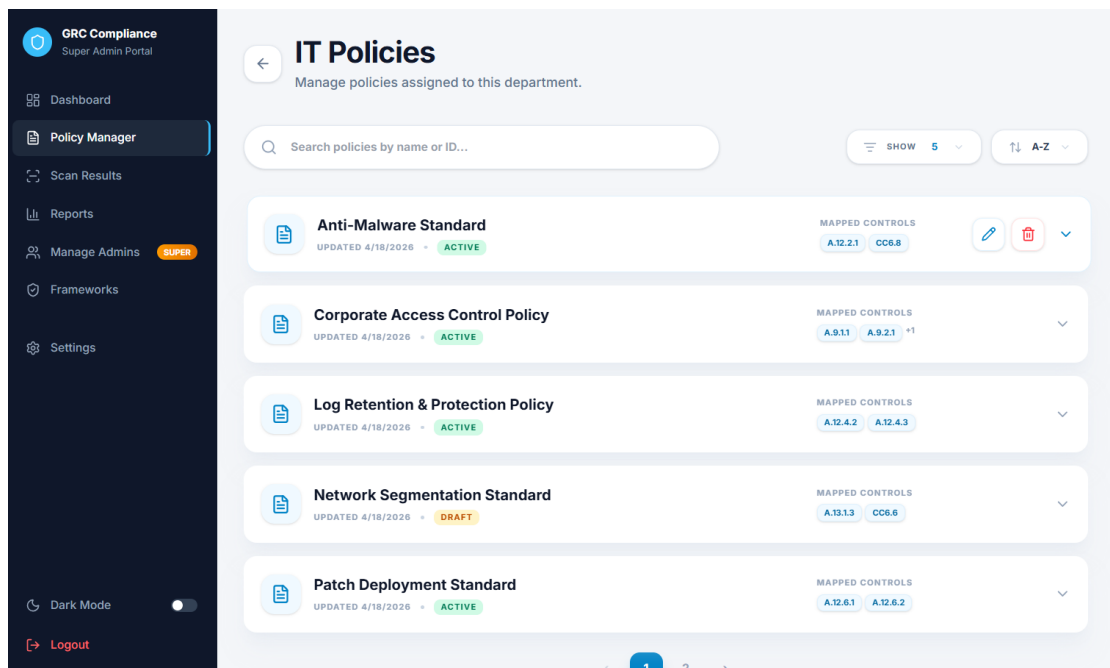


Figure A.4: Policy configuration interface with department-level governance assignment.

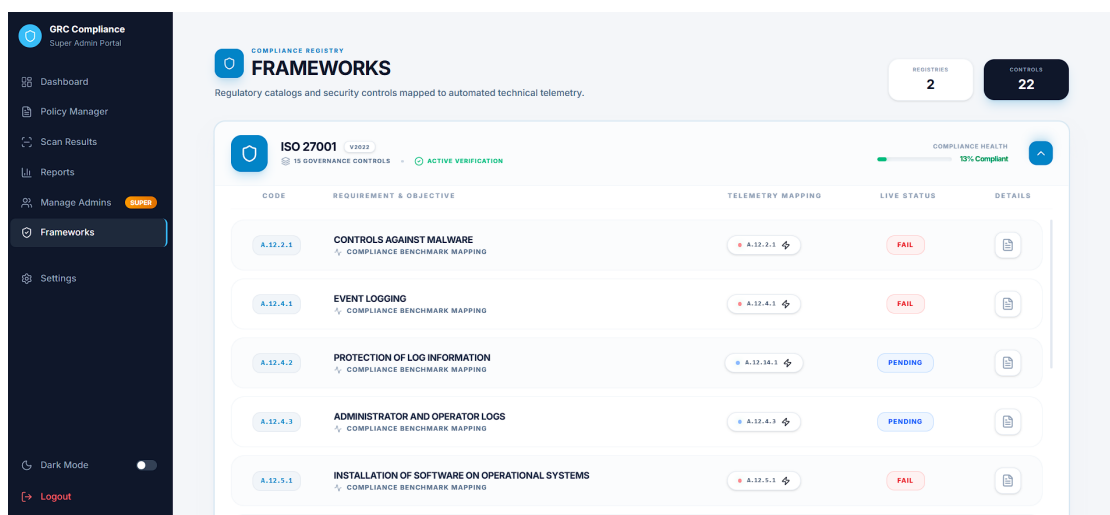


Figure A.5: Detailed view of the regulatory framework requirements mapping interface.

Step 4 – Telemetry Mapping Configuration (Superadmins)

- Use the "Mapping Configuration" portal.
- Link Wazuh Security Configuration Assessment (SCA) Rule IDs to GRC controls.
- Save the mapping to categorize incoming SIEM events.

Step 5 – Wazuh Telemetry Synchronization

- The system polls the Wazuh manager in the background.

- Administrators can click "Trigger Manual Sync" on the Scans page for an immediate update.
- View synchronization logs to verify data ingestion.

Step 6 – Endpoint Monitoring

- Use the "Endpoints" view to monitor specific systems.
- Select a workstation or server to view its compliance score and failed security checks.

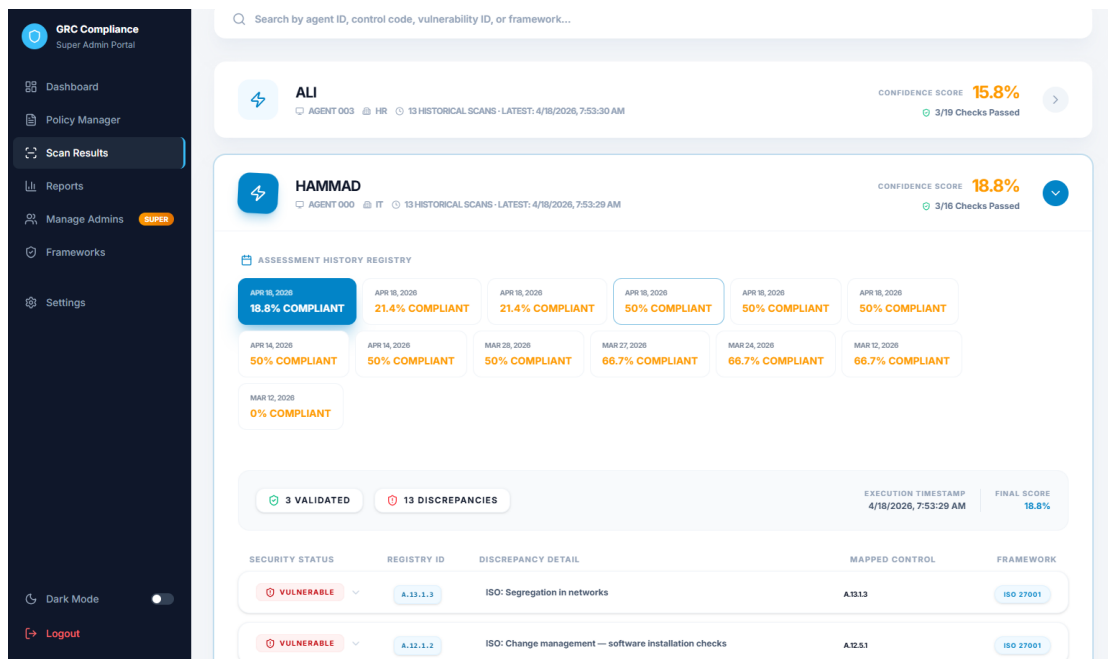


Figure A.6: Interface for raw scan results and endpoint telemetry mapping.

Step 7 – Automated Audit Report Generation

- Use the "Reports" module to generate documentation.
- Select the regulatory framework and, optionally, a target Department to scope the report.
- Click "Generate PDF" to download the audit report.

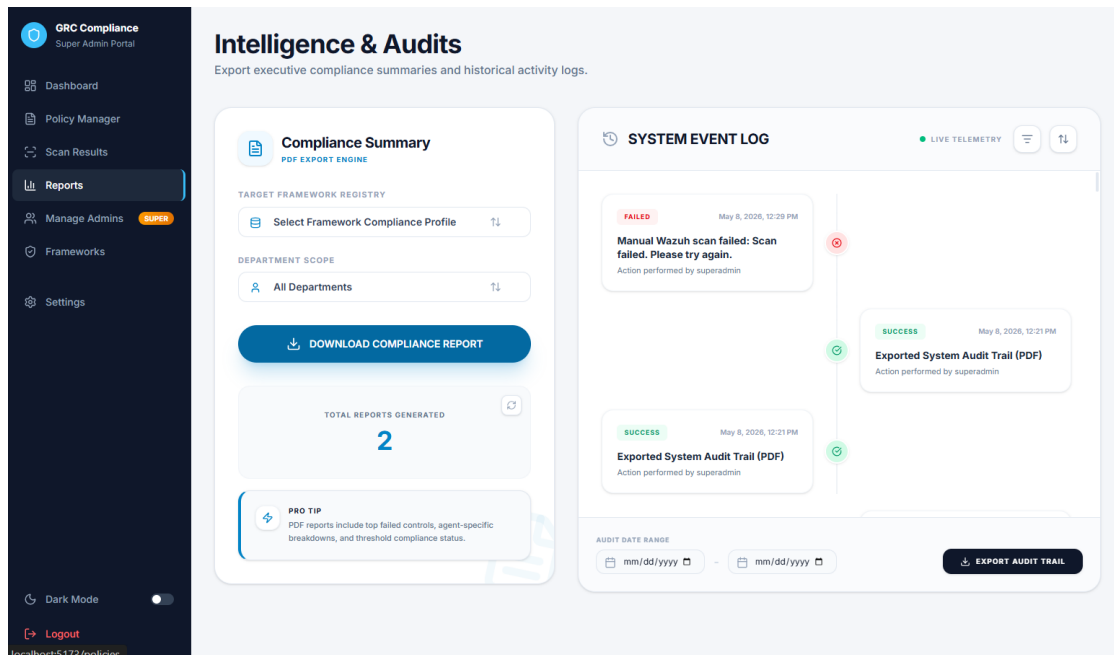


Figure A.7: Report generation interface featuring Framework and Department filtering.

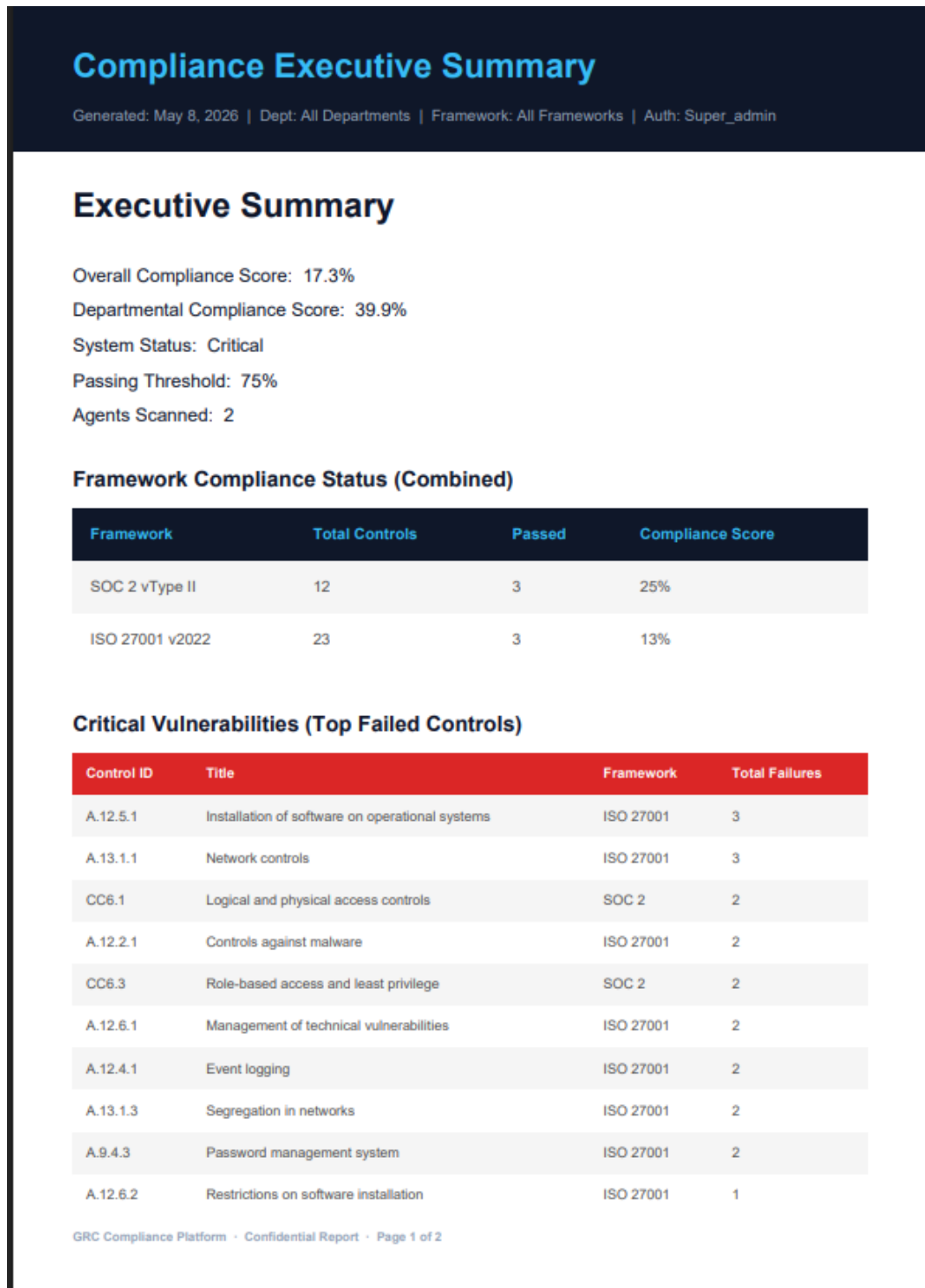


Figure A.8: Generated PDF dossier highlighting department-specific compliance summaries.

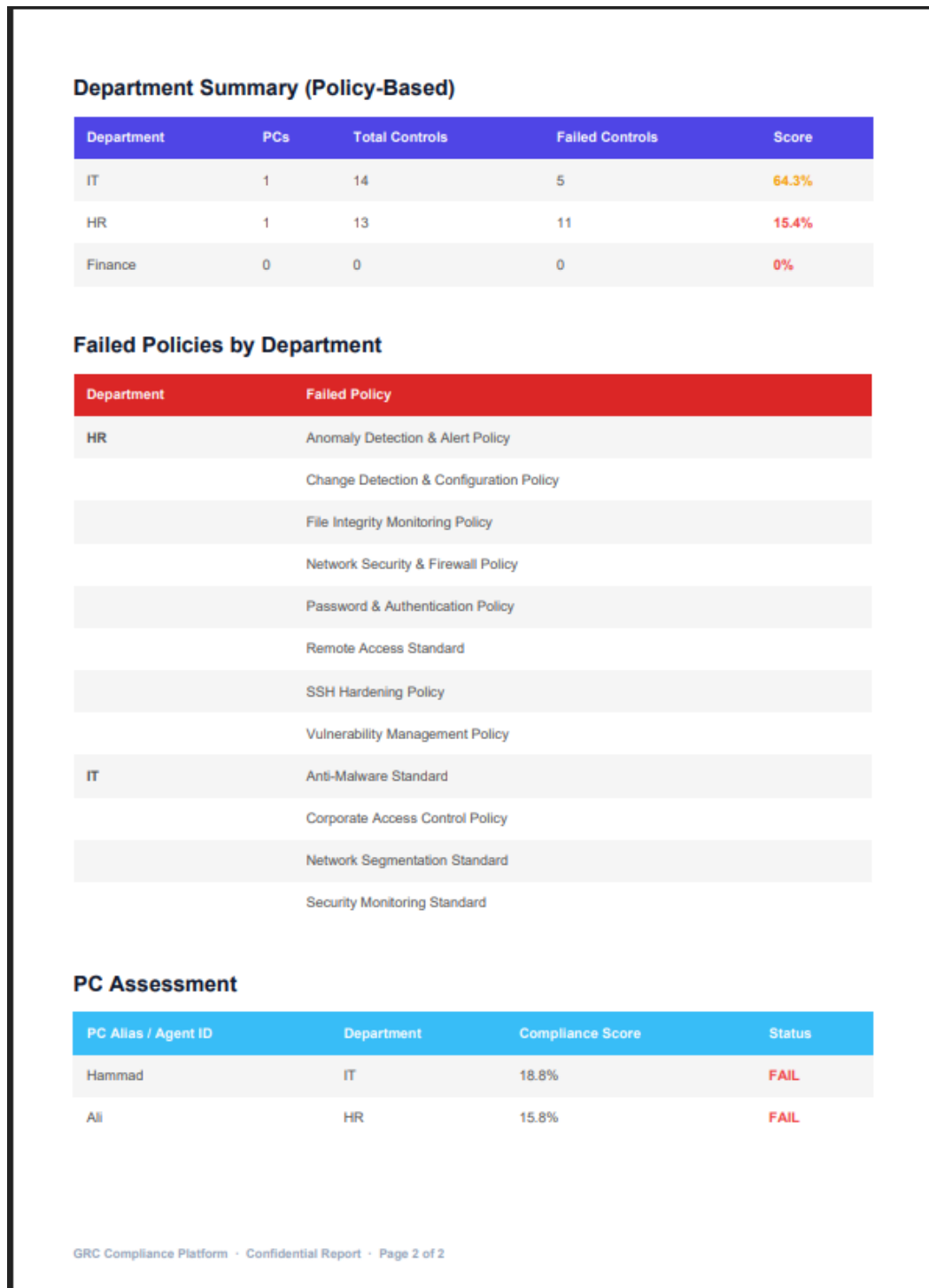


Figure A.9: Department-scoped compliance detail page from the generated PDF.

Step 8 – System and User Management

- Use "Manage Users" to provision accounts and assign roles.
- Configure SMTP alerts in System Settings for email notifications.

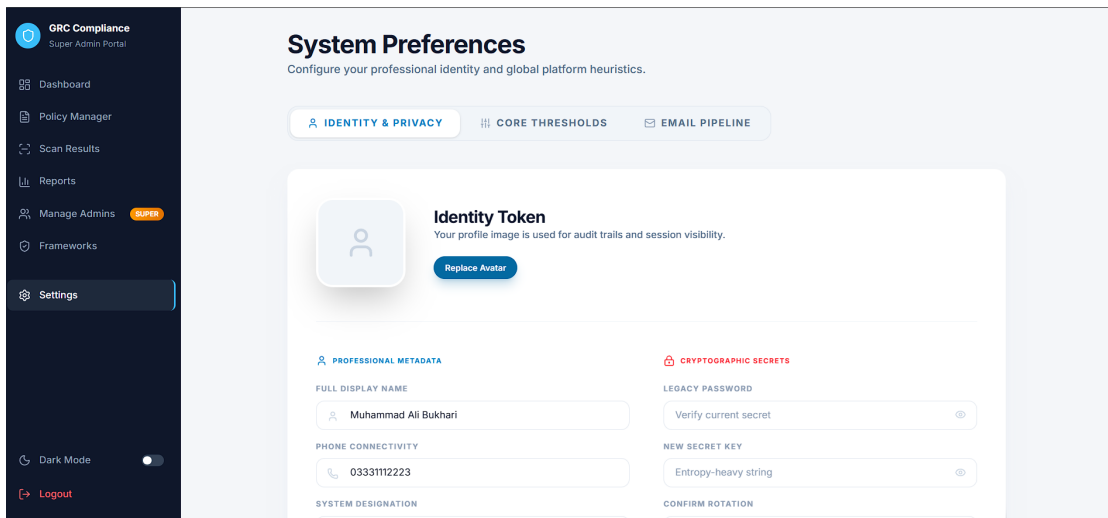


Figure A.10: System Settings interface for managing global configurations and thresholds.

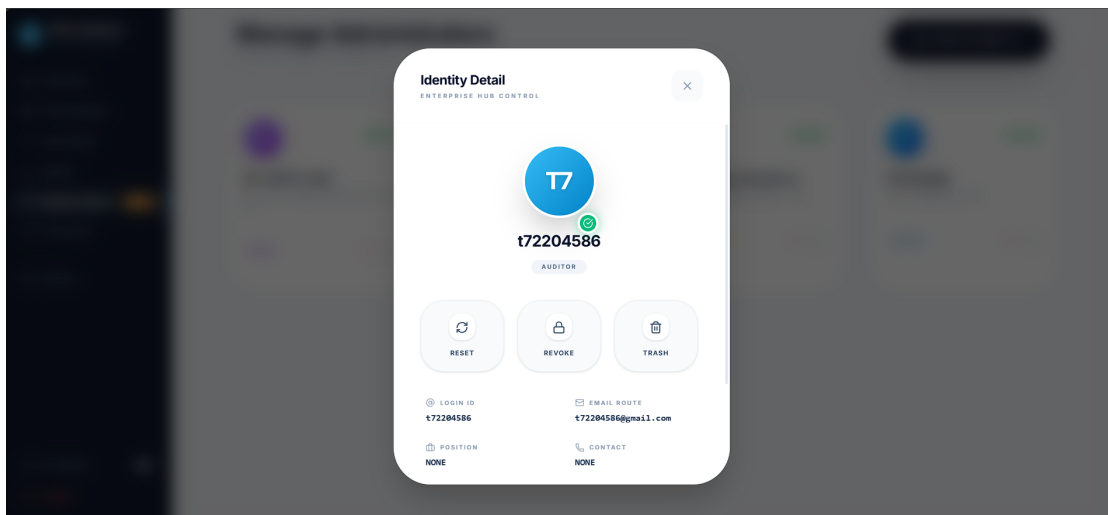


Figure A.11: Expanded user management interface for configuring strict RBAC hierarchies.

A.5 Logout

- Click the user profile icon in the navigation shell.
- Select "Logout" to end the session and clear browser storage.

A.6 Support

- For credential recovery or 2FA resets, contact the IT Security Administrator.
- For Wazuh agent connectivity issues, contact the SIEM infrastructure team.

A.7 Note

The ISCMS platform automates compliance visibility and reporting based on SIEM telemetry. It highlights security gaps and maps them to regulatory controls, but resolving those gaps requires administrative action on the affected endpoints.

References

- [1] T. T. Bukhari, O. Oladimeji, E. D. Etim, and J. O. Ajayi, “Automated control monitoring: A new standard for continuous audit readiness,” *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, vol. 7, no. 3, pp. 711–735, 2021. Cited on p. 1.
- [2] Joint Task Force, “Security and privacy controls for information systems and organizations,” Tech. Rep. NIST Special Publication 800-53 Revision 5, National Institute of Standards and Technology, 2020. Cited on pp. 1 and 7.
- [3] G. Quispe-Kobashikawa, C. Zuloaga-Estrada, P. S. Castañeda Vargas, and A. D. García-Núñez, “Enhancing information security in technology small and medium-sized enterprises: A metrics-driven model based on ISO/IEC 27001:2022,” *IEEE Access*, 2026. Cited on pp. 1 and 7.
- [4] L. Yang, Z. Chen, C. Wang, Z. Zhang, S. Booma, P. Cao, C. Adam, A. Withers, Z. Kalbarczyk, R. K. Iyer, and G. Wang, “True attacks, attack attempts, or benign triggers? an empirical measurement of network alerts in a security operations center,” in *Proceedings of the 33rd USENIX Security Symposium*, (Philadelphia, PA, USA), 2024. Cited on pp. 2 and 10.
- [5] P. Best, “Auditing during a pandemic: Can continuous controls monitoring (ccm) address challenges facing internal audit departments?,” *Pacific Accounting Review*, vol. 35, no. 5, pp. 727–745, 2023. Cited on p. 7.
- [6] M. A. Jamali and A. Masood, “Cybersecurity on a budget: Evaluating security and performance of open-source SIEM solutions for SMEs,” *PLOS ONE*, vol. 19, no. 3, p. e0301183, 2024. Cited on p. 7.
- [7] D. Poudel, “A unified automation framework for CIS benchmark compliance and real-time remediation on ubuntu using wazuh, ansible and FastAPI,” *International Journal of Innovative Science and Research Technology*, vol. 11, no. 4, 2026. Cited on p. 8.
- [8] R. F. Aji and N. P. Utami, “Evaluating field flexibility approaches in relational databases: A performance study of JSON and column-oriented models in library systems,” *International Journal of Advanced Computer Science and Applications*, vol. 17, no. 1, 2026. Cited on pp. 8 and 55.
- [9] H. D. Cat, “Implementing security measures in web chat applications using django framework,” bachelor’s thesis, Tampere University, 2025. Cited on p. 9.

- [10] S. Ahmad, M. Arif, J. Ahmad, and S. Mehruz, “A TOTP-based secure data storage system in the cloud environment using the JWT token approach,” *International Journal of System Assurance Engineering and Management*, vol. 16, no. 4, pp. 1565–1578, 2025. Cited on p. 9.
- [11] M. Amer and T. S. Sobh, “Biometric JSON web tokens (BJWT): Enhancing web API security with biometric key exchange and OTP-JWT authentication,” *International Journal of Computer Applications*, vol. 186, no. 39, pp. 15–21, 2024. Cited on p. 9.
- [12] E. Casalicchio and S. Iannucci, “The state-of-the-art in container technologies: Application, orchestration and security,” *Concurrency and Computation: Practice and Experience*, vol. 32, no. 17, p. e5668, 2020. Cited on pp. 9 and 55.
- [13] V. S. Thokala, “Utilizing docker containers for reproducible builds and scalable web application deployments,” *Journal of Information Systems Engineering and Management*, vol. 10, no. 45s, p. 725, 2025. Cited on p. 10.
- [14] J. Happa, I. Agraftotis, M. Helmhout, T. Bashford-Rogers, M. Goldsmith, and S. Creese, “Assessing a decision support tool for SOC analysts,” *Digital Threats: Research and Practice*, 2020. Cited on p. 10.
- [15] S. A. Chamkar, M. Zaydi, Y. Maleh, and N. Gherabi, “Improving threat detection in wazuh using machine learning techniques,” *Journal of Cybersecurity and Privacy*, vol. 5, no. 2, p. 34, 2025. Cited on p. 54.